Jetpack

Generated by Doxygen 1.13.2

1 Namespace Index 1
1.1 Namespace List . . . 1

2 Hierarchical Index 3
2.1 Class Hierarchy . . . 3

3 Class Index 5
3.1 Class List . . . 5

4 File Index 7
4.1 File List . . . 7

5 Namespace Documentation 11
5.1 Math Namespace Reference . . . 11
5.1.1 Function Documentation . . . 11
5.1.1.1 operator∗() . . . 11

6 Class Documentation 13
6.1 AMaterial Class Reference . . . 13
6.1.1 Constructor & Destructor Documentation . . . 14
6.1.1.1 AMaterial() . . . 14
6.1.1.2 ∼AMaterial() . . . 14
6.1.2 Member Function Documentation . . . 14
6.1.2.1 getAmbient() . . . 14
6.1.2.2 getColorAt() . . . 14
6.1.2.3 getDiffuse() . . . 15
6.1.2.4 getDiffuseColor() . . . 15
6.1.2.5 getName() . . . 15
6.1.2.6 getRefractionIndex() . . . 15
6.1.2.7 getShininess() . . . 15
6.1.2.8 getSpecular() . . . 15
6.1.2.9 getSpecularColor() . . . 15
6.1.2.10 getTransparency() . . . 15
6.1.2.11 loadFromMtlLine() . . . 15
6.1.2.12 setDiffuseColor() . . . 16
6.1.2.13 setRefractionIndex() . . . 16
6.1.2.14 setShininess() . . . 16
6.1.2.15 setSpecularColor() . . . 16
6.1.2.16 setTransparency() . . . 17
6.2 AmbientLight Class Reference . . . 17
6.2.1 Detailed Description . . . 18
6.2.2 Constructor & Destructor Documentation . . . 18
6.2.2.1 AmbientLight() . . . 18
6.2.3 Member Function Documentation . . . 18

6.2.3.1 getAmbientComponent() . . . 18
6.2.3.2 getColor() . . . 18
6.2.3.3 getDiffuseComponent() . . . 19
6.2.3.4 getDirection() . . . 19
6.2.3.5 getIntensity() . . . 19
6.2.3.6 getPosition() . . . 19
6.2.3.7 getSpecularComponent() . . . 20
6.2.3.8 getType() . . . 20
6.3 Camera Class Reference . . . 20
6.3.1 Detailed Description . . . 21
6.3.2 Constructor & Destructor Documentation . . . 21
6.3.2.1 Camera() . . . 21
6.3.3 Member Function Documentation . . . 21
6.3.3.1 getFov() . . . 21
6.3.3.2 getHeight() . . . 22
6.3.3.3 getPosition() . . . 22
6.3.3.4 getRay() . . . 22
6.3.3.5 getWidth() . . . 23
6.3.3.6 rotate() . . . 23
6.3.3.7 setPosition() . . . 23
6.3.3.8 setRay() . . . 23
6.3.3.9 shear() . . . 24
6.3.3.10 translate() . . . 24
6.4 ChessboardMaterial Class Reference . . . 24
6.4.1 Detailed Description . . . 25
6.4.2 Constructor & Destructor Documentation . . . 25
6.4.2.1 ChessboardMaterial() [1/2] . . . 25
6.4.2.2 ChessboardMaterial() [2/2] . . . 25
6.4.2.3 ~ChessboardMaterial() . . . 26
6.4.3 Member Function Documentation . . . 26
6.4.3.1 getColorAt() . . . 26
6.5 Color Class Reference . . . 26
6.5.1 Detailed Description . . . 27
6.5.2 Constructor & Destructor Documentation . . . 27
6.5.2.1 Color() . . . 27
6.5.3 Member Function Documentation . . . 28
6.5.3.1 applyIntensity() . . . 28
6.5.3.2 clamp() . . . 28
6.5.3.3 getB() . . . 29
6.5.3.4 getG() . . . 29
6.5.3.5 getR() . . . 29
6.5.3.6 operator*() [1/2] . . . 29

6.5.3.7 operator∗() [2/2] . . . 30
6.5.3.8 operator∗=() [1/2] . . . 30
6.5.3.9 operator∗=() [2/2] . . . 30
6.5.3.10 operator+() . . . 31
6.5.3.11 operator+=() . . . 31
6.5.3.12 operator-() . . . 31
6.5.3.13 operator/() . . . 32
6.5.3.14 operator/=() . . . 32
6.5.3.15 setColor() . . . 32
6.6 Cone Class Reference . . . 33
6.6.1 Constructor & Destructor Documentation . . . 33
6.6.1.1 Cone() [1/2] . . . 33
6.6.1.2 Cone() [2/2] . . . 34
6.6.1.3 ∼Cone() . . . 34
6.6.2 Member Function Documentation . . . 34
6.6.2.1 getColor() . . . 34
6.6.2.2 getIntersectBase() . . . 35
6.6.2.3 getIntersectCone() . . . 35
6.6.2.4 getIntersection() . . . 35
6.6.2.5 getMaterial() . . . 36
6.6.2.6 getType() . . . 36
6.6.2.7 intersect() . . . 36
6.6.2.8 intersect_base() . . . 36
6.6.2.9 intersectCone() . . . 37
6.6.2.10 normal_at() . . . 37
6.7 Core Class Reference . . . 38
6.7.1 Detailed Description . . . 38
6.7.2 Constructor & Destructor Documentation . . . 38
6.7.2.1 Core() . . . 38
6.7.2.2 ∼Core() . . . 39
6.7.3 Member Function Documentation . . . 39
6.7.3.1 checkSceneFile() . . . 39
6.7.3.2 createGuiThread() . . . 39
6.7.3.3 createRenderThread() . . . 39
6.7.3.4 init() . . . 39
6.7.3.5 loop() . . . 40
6.7.3.6 notifyRenderThread() . . . 40
6.7.3.7 reloadScene() . . . 40
6.7.3.8 setCtrl() . . . 40
6.7.3.9 setHandleRotation() . . . 41
6.7.3.10 setHandleTranslation() . . . 41
6.8 Cursor Class Reference . . . 41

6.8.1 Constructor & Destructor Documentation . . . 42
6.8.1.1 Cursor() . . . 42
6.8.1.2 ~Cursor() . . . 42
6.8.2 Member Function Documentation . . . 42
6.8.2.1 getCursorLine() . . . 42
6.8.2.2 getCursorTitleText() . . . 42
6.8.2.3 getPosCursor() . . . 43
6.8.2.4 getRectangle() . . . 43
6.8.2.5 getText() . . . 43
6.8.2.6 getType() . . . 43
6.8.2.7 initShape() . . . 43
6.8.2.8 initText() . . . 43
6.8.2.9 isCursorSelected() . . . 44
6.8.2.10 mouseClickedOnCursor() . . . 44
6.8.2.11 moveCursor() . . . 44
6.8.2.12 moveRectangleShape() . . . 44
6.8.2.13 setCursorSelected() . . . 45
6.8.2.14 setCursorText() . . . 45
6.8.2.15 updateCursor() . . . 45
6.9 Cylindre Class Reference . . . 46
6.9.1 Constructor & Destructor Documentation . . . 46
6.9.1.1 Cylindre() [1/2] . . . 46
6.9.1.2 Cylindre() [2/2] . . . 46
6.9.1.3 ~Cylindre() . . . 47
6.9.2 Member Function Documentation . . . 47
6.9.2.1 getColor() . . . 47
6.9.2.2 getIntersection() . . . 47
6.9.2.3 getMaterial() . . . 48
6.9.2.4 getType() . . . 48
6.9.2.5 intersect() . . . 48
6.9.2.6 intersection_cap() . . . 48
6.9.2.7 normal_at() . . . 49
6.10 Default Class Reference . . . 49
6.10.1 Constructor & Destructor Documentation . . . 50
6.10.1.1 Default() . . . 50
6.11 DirectionalLight Class Reference . . . 50
6.11.1 Detailed Description . . . 51
6.11.2 Constructor & Destructor Documentation . . . 51
6.11.2.1 DirectionalLight() . . . 51
6.11.3 Member Function Documentation . . . 52
6.11.3.1 getAmbientComponent() . . . 52
6.11.3.2 getColor() . . . 52

6.11.3.3 getDiffuseComponent() . . . 52
6.11.3.4 getDirection() . . . 52
6.11.3.5 getIntensity() . . . 53
6.11.3.6 getPosition() . . . 53
6.11.3.7 getSpecularComponent() . . . 53
6.11.3.8 getType() . . . 53
6.12 Face Class Reference . . . 54
6.12.1 Detailed Description . . . 54
6.12.2 Constructor & Destructor Documentation . . . 54
6.12.2.1 Face() . . . 54
6.12.2.2 ~Face() . . . 55
6.12.3 Member Function Documentation . . . 55
6.12.3.1 getColor() . . . 55
6.12.3.2 getIntersection() . . . 55
6.12.3.3 getMaterial() . . . 56
6.12.3.4 getType() . . . 56
6.12.3.5 intersect() . . . 56
6.12.3.6 normal_at() . . . 56
6.13 Gui Class Reference . . . 57
6.13.1 Constructor & Destructor Documentation . . . 57
6.13.1.1 Gui() [1/2] . . . 57
6.13.1.2 Gui() [2/2] . . . 58
6.13.1.3 ~Gui() . . . 58
6.13.2 Member Function Documentation . . . 58
6.13.2.1 createImage() . . . 58
6.13.2.2 handleInput() . . . 59
6.13.2.3 isOpen() . . . 59
6.13.2.4 renderScene() . . . 59
6.13.2.5 updateScene() . . . 59
6.14 HitInfo Class Reference . . . 59
6.14.1 Detailed Description . . . 60
6.14.2 Constructor & Destructor Documentation . . . 60
6.14.2.1 HitInfo() [1/2] . . . 60
6.14.2.2 HitInfo() [2/2] . . . 60
6.14.3 Member Function Documentation . . . 61
6.14.3.1 getColor() . . . 61
6.14.3.2 getDistance() . . . 61
6.14.3.3 getNormal() . . . 61
6.14.3.4 getPosition() . . . 62
6.15 ILight Class Reference . . . 62
6.15.1 Detailed Description . . . 63
6.15.2 Constructor & Destructor Documentation . . . 63

6.15.2.1 ~ILight() . . . 63
6.15.3 Member Function Documentation . . . 63
6.15.3.1 getAmbientComponent() . . . 63
6.15.3.2 getColor() . . . 63
6.15.3.3 getDiffuseComponent() . . . 63
6.15.3.4 getDirection() . . . 64
6.15.3.5 getIntensity() . . . 64
6.15.3.6 getPosition() . . . 64
6.15.3.7 getSpecularComponent() . . . 64
6.15.3.8 getType() . . . 65
6.16 Image Class Reference . . . 65
6.16.1 Detailed Description . . . 65
6.16.2 Constructor & Destructor Documentation . . . 65
6.16.2.1 Image() . . . 65
6.16.2.2 ~Image() . . . 66
6.16.3 Member Function Documentation . . . 66
6.16.3.1 createDirAndFileName() . . . 66
6.16.3.2 save() . . . 66
6.17 ImageException Class Reference . . . 67
6.17.1 Detailed Description . . . 67
6.17.2 Constructor & Destructor Documentation . . . 67
6.17.2.1 ImageException() . . . 67
6.18 InfinitCone Class Reference . . . 68
6.18.1 Constructor & Destructor Documentation . . . 69
6.18.1.1 InfinitCone() [1/2] . . . 69
6.18.1.2 InfinitCone() [2/2] . . . 69
6.18.1.3 ~InfinitCone() . . . 69
6.18.2 Member Function Documentation . . . 70
6.18.2.1 getColor() . . . 70
6.18.2.2 getIntersection() . . . 70
6.18.2.3 getMaterial() . . . 70
6.18.2.4 getType() . . . 70
6.18.2.5 intersect() . . . 70
6.18.2.6 normal_at() . . . 71
6.19 InfiniteCylindre Class Reference . . . 71
6.19.1 Constructor & Destructor Documentation . . . 72
6.19.1.1 InfiniteCylindre() [1/2] . . . 72
6.19.1.2 InfiniteCylindre() [2/2] . . . 72
6.19.1.3 ~InfiniteCylindre() . . . 73
6.19.2 Member Function Documentation . . . 73
6.19.2.1 getColor() . . . 73
6.19.2.2 getIntersection() . . . 73

6.19.2.3 getMaterial() . . . 73
6.19.2.4 getType() . . . 74
6.19.2.5 intersect() . . . 74
6.19.2.6 normal_at() . . . 74
6.20 IPrimitive Class Reference . . . 75
6.20.1 Detailed Description . . . 75
6.20.2 Constructor & Destructor Documentation . . . 75
6.20.2.1 ∼IPrimitive() . . . 75
6.20.3 Member Function Documentation . . . 75
6.20.3.1 getColor() . . . 75
6.20.3.2 getIntersection() . . . 76
6.20.3.3 getMaterial() . . . 76
6.20.3.4 getType() . . . 76
6.20.3.5 intersect() . . . 76
6.20.3.6 normal_at() . . . 77
6.21 ITransformations Class Reference . . . 77
6.21.1 Detailed Description . . . 78
6.21.2 Constructor & Destructor Documentation . . . 78
6.21.2.1 ∼ITransformations() . . . 78
6.21.3 Member Function Documentation . . . 78
6.21.3.1 apply() . . . 78
6.22 Math::Matrix< X, Y > Class Template Reference . . . 79
6.22.1 Detailed Description . . . 79
6.22.2 Constructor & Destructor Documentation . . . 80
6.22.2.1 Matrix() [1/2] . . . 80
6.22.2.2 Matrix() [2/2] . . . 80
6.22.2.3 ∼Matrix() . . . 80
6.22.3 Member Function Documentation . . . 80
6.22.3.1 getCol() . . . 80
6.22.3.2 getRow() . . . 80
6.22.3.3 operator()() [1/2] . . . 80
6.22.3.4 operator()() [2/2] . . . 81
6.22.3.5 operator∗() . . . 81
6.22.4 Member Data Documentation . . . 82
6.22.4.1 _col . . . 82
6.22.4.2 _matrix . . . 82
6.22.4.3 _row . . . 82
6.23 Metal Class Reference . . . 82
6.23.1 Constructor & Destructor Documentation . . . 83
6.23.1.1 Metal() . . . 83
6.24 Mtl Class Reference . . . 83
6.24.1 Constructor & Destructor Documentation . . . 84

6.24.1.1 Mtl() . . . 84
6.24.1.2 ~Mtl() . . . 85
6.25 ObjParser Class Reference . . . 85
6.25.1 Detailed Description . . . 85
6.25.2 Member Function Documentation . . . 85
6.25.2.1 loadMaterials() . . . 85
6.25.2.2 parse() . . . 85
6.25.2.3 parseMaterial() . . . 86
6.26 PerlinNoiseMaterial Class Reference . . . 86
6.26.1 Constructor & Destructor Documentation . . . 87
6.26.1.1 PerlinNoiseMaterial() [1/2] . . . 87
6.26.1.2 PerlinNoiseMaterial() [2/2] . . . 87
6.26.1.3 ~PerlinNoiseMaterial() . . . 88
6.26.2 Member Function Documentation . . . 88
6.26.2.1 getColorAt() . . . 88
6.27 Plane Class Reference . . . 88
6.27.1 Detailed Description . . . 89
6.27.2 Constructor & Destructor Documentation . . . 89
6.27.2.1 Plane() . . . 89
6.27.2.2 ~Plane() . . . 90
6.27.3 Member Function Documentation . . . 90
6.27.3.1 getColor() . . . 90
6.27.3.2 getIntersection() . . . 90
6.27.3.3 getMaterial() . . . 90
6.27.3.4 getType() . . . 91
6.27.3.5 intersect() . . . 91
6.27.3.6 normal_at() . . . 91
6.28 PlugInException Class Reference . . . 92
6.28.1 Detailed Description . . . 92
6.29 PluginsException Class Reference . . . 92
6.29.1 Constructor & Destructor Documentation . . . 92
6.29.1.1 PluginsException() . . . 92
6.30 PluginsHandle Class Reference . . . 93
6.30.1 Detailed Description . . . 93
6.30.2 Constructor & Destructor Documentation . . . 93
6.30.2.1 PluginsHandle() [1/3] . . . 93
6.30.2.2 ~PluginsHandle() . . . 93
6.30.2.3 PluginsHandle() [2/3] . . . 94
6.30.2.4 PluginsHandle() [3/3] . . . 95
6.30.3 Member Function Documentation . . . 95
6.30.3.1 getSymbol() . . . 95
6.30.3.2 operator=() [1/2] . . . 95

6.30.3.3 operator=() [2/2] . . . 96
6.31 PluginsManager Class Reference . . . 96
6.31.1 Detailed Description . . . 97
6.31.2 Member Function Documentation . . . 97
6.31.2.1 createLight() . . . 97
6.31.2.2 createPrimitive() . . . 97
6.31.2.3 getSymbol() . . . 97
6.31.2.4 loadPlugins() . . . 98
6.31.2.5 registerLightFactory() . . . 98
6.31.2.6 registerPrimitiveFactory() . . . 98
6.31.2.7 unregisterAllFactories() . . . 99
6.32 Math::Point Class Reference . . . 99
6.32.1 Detailed Description . . . 100
6.32.2 Constructor & Destructor Documentation . . . 100
6.32.2.1 Point() [1/2] . . . 100
6.32.2.2 Point() [2/2] . . . 100
6.32.3 Member Function Documentation . . . 101
6.32.3.1 dot() . . . 101
6.32.3.2 getX() . . . 101
6.32.3.3 getY() . . . 101
6.32.3.4 getZ() . . . 101
6.32.3.5 operator+() [1/2] . . . 101
6.32.3.6 operator+() [2/2] . . . 102
6.32.3.7 operator+=() . . . 102
6.32.3.8 operator-() [1/2] . . . 102
6.32.3.9 operator-() [2/2] . . . 103
6.32.3.10 operator==() . . . 103
6.32.3.11 pointToMatrix() . . . 103
6.32.3.12 setX() . . . 104
6.32.3.13 setY() . . . 104
6.32.3.14 setZ() . . . 104
6.32.3.15 shearToMatrix() . . . 104
6.33 PointLight Class Reference . . . 104
6.33.1 Detailed Description . . . 105
6.33.2 Constructor & Destructor Documentation . . . 105
6.33.2.1 PointLight() . . . 105
6.33.3 Member Function Documentation . . . 106
6.33.3.1 getAmbientComponent() . . . 106
6.33.3.2 getColor() . . . 106
6.33.3.3 getDiffuseComponent() . . . 106
6.33.3.4 getDirection() . . . 106
6.33.3.5 getIntensity() . . . 107

6.33.3.6 getPosition() . . . 107
6.33.3.7 getSpecularComponent() . . . 107
6.33.3.8 getType() . . . 107
6.34 Ray Class Reference . . . 108
6.34.1 Detailed Description . . . 108
6.34.2 Constructor & Destructor Documentation . . . 108
6.34.2.1 Ray() . . . 108
6.34.3 Member Function Documentation . . . 109
6.34.3.1 getDirection() . . . 109
6.34.3.2 getOrigin() . . . 109
6.34.3.3 setDirection() . . . 109
6.34.3.4 setOrigin() . . . 109
6.35 RaytracerException Class Reference . . . 110
6.35.1 Detailed Description . . . 110
6.35.2 Constructor & Destructor Documentation . . . 110
6.35.2.1 RaytracerException() . . . 110
6.36 Render Class Reference . . . 110
6.36.1 Detailed Description . . . 111
6.36.2 Constructor & Destructor Documentation . . . 111
6.36.2.1 Render() . . . 111
6.36.2.2 ∼Render() . . . 112
6.36.3 Member Function Documentation . . . 112
6.36.3.1 adaptiveSample() . . . 112
6.36.3.2 AmbiantOcclusion() . . . 113
6.36.3.3 ComputeDiffuseLights() . . . 113
6.36.3.4 computeFastRender() . . . 113
6.36.3.5 ComputePixelColor() . . . 113
6.36.3.6 computeRender() . . . 113
6.36.3.7 ComputeSpecularLight() . . . 114
6.36.3.8 isInShadowOfShape() . . . 114
6.36.3.9 randomHemisphereDirection() . . . 114
6.36.3.10 reflect() . . . 114
6.36.3.11 renderPixel() . . . 115
6.36.3.12 renderReflection() . . . 116
6.36.3.13 renderRefraction() . . . 116
6.36.3.14 sendRay() . . . 116
6.36.3.15 setProgressCallback() . . . 117
6.37 RenderException Class Reference . . . 117
6.37.1 Detailed Description . . . 118
6.37.2 Constructor & Destructor Documentation . . . 118
6.37.2.1 RenderException() . . . 118
6.38 Rotation Class Reference . . . 118

6.38.1 Detailed Description . . . 119
6.38.2 Constructor & Destructor Documentation . . . 119
6.38.2.1 Rotation() . . . 119
6.38.2.2 ∼Rotation() . . . 119
6.38.3 Member Function Documentation . . . 120
6.38.3.1 apply() . . . 120
6.38.3.2 RotateX() . . . 120
6.38.3.3 RotateY() . . . 120
6.38.3.4 RotateZ() . . . 120
6.39 Scene Class Reference . . . 121
6.39.1 Detailed Description . . . 121
6.39.2 Constructor & Destructor Documentation . . . 121
6.39.2.1 Scene() . . . 121
6.39.2.2 ∼Scene() . . . 122
6.39.3 Member Function Documentation . . . 122
6.39.3.1 addCamera() . . . 122
6.39.3.2 addLight() . . . 122
6.39.3.3 addPrimitive() . . . 122
6.39.3.4 getCamera() . . . 122
6.39.3.5 getDiffuse() . . . 123
6.39.3.6 getLights() . . . 123
6.39.3.7 getModifableCamera() . . . 123
6.39.3.8 getPrimitives() . . . 123
6.39.3.9 setDiffuse() . . . 123
6.39.3.10 setRotate() . . . 124
6.39.3.11 setShear() . . . 124
6.39.3.12 setTranslateScene() . . . 124
6.40 SceneException Class Reference . . . 124
6.40.1 Detailed Description . . . 125
6.40.2 Constructor & Destructor Documentation . . . 125
6.40.2.1 SceneException() . . . 125
6.41 SceneLoader Class Reference . . . 125
6.41.1 Detailed Description . . . 125
6.41.2 Member Function Documentation . . . 125
6.41.2.1 parseSceneCamera() . . . 125
6.41.2.2 parseSceneFile() . . . 126
6.41.2.3 parseSceneLight() . . . 126
6.41.2.4 parseScenePrimitive() . . . 126
6.42 Screen Class Reference . . . 127
6.42.1 Constructor & Destructor Documentation . . . 127
6.42.1.1 Screen() [1/2] . . . 127
6.42.1.2 Screen() [2/2] . . . 127

6.42.1.3 ~Screen() . . . 127
6.42.2 Member Function Documentation . . . 128
6.42.2.1 getOrigin() . . . 128
6.42.2.2 pointAt() . . . 128
6.42.2.3 rotate() . . . 128
6.42.2.4 shear() . . . 128
6.42.2.5 translate() . . . 128
6.43 Shear Class Reference . . . 128
6.43.1 Detailed Description . . . 129
6.43.2 Constructor & Destructor Documentation . . . 129
6.43.2.1 Shear() . . . 129
6.43.2.2 ~Shear() . . . 129
6.43.3 Member Function Documentation . . . 129
6.43.3.1 apply() . . . 129
6.43.3.2 shear() . . . 130
6.44 Sphere Class Reference . . . 130
6.44.1 Detailed Description . . . 131
6.44.2 Constructor & Destructor Documentation . . . 131
6.44.2.1 Sphere() . . . 131
6.44.2.2 ~Sphere() . . . 131
6.44.3 Member Function Documentation . . . 131
6.44.3.1 getColor() . . . 131
6.44.3.2 getIntersection() . . . 132
6.44.3.3 getMaterial() . . . 133
6.44.3.4 getType() . . . 133
6.44.3.5 intersect() . . . 133
6.44.3.6 normal_at() . . . 134
6.45 Translation Class Reference . . . 134
6.45.1 Detailed Description . . . 135
6.45.2 Constructor & Destructor Documentation . . . 135
6.45.2.1 Translation() . . . 135
6.45.2.2 ~Translation() . . . 136
6.45.3 Member Function Documentation . . . 136
6.45.3.1 apply() . . . 136
6.45.3.2 translateDown() . . . 136
6.45.3.3 translateIn() . . . 137
6.45.3.4 translateLeft() . . . 137
6.45.3.5 translateOut() . . . 137
6.45.3.6 translateRight() . . . 137
6.45.3.7 translateUp() . . . 137
6.46 Transparent Class Reference . . . 137
6.46.1 Constructor & Destructor Documentation . . . 138

6.46.1.1 Transparent() . . . 138
6.47 Triangle Class Reference . . . 139
6.47.1 Detailed Description . . . 139
6.47.2 Constructor & Destructor Documentation . . . 139
6.47.2.1 Triangle() . . . 139
6.47.2.2 ∼Triangle() . . . 140
6.47.3 Member Function Documentation . . . 140
6.47.3.1 getColor() . . . 140
6.47.3.2 getIntersection() . . . 140
6.47.3.3 getMaterial() . . . 141
6.47.3.4 getType() . . . 141
6.47.3.5 intersect() . . . 141
6.47.3.6 normal_at() . . . 141
6.48 Math::Vector Class Reference . . . 142
6.48.1 Detailed Description . . . 143
6.48.2 Constructor & Destructor Documentation . . . 143
6.48.2.1 Vector() . . . 143
6.48.3 Member Function Documentation . . . 144
6.48.3.1 crossProduct() . . . 144
6.48.3.2 dot() . . . 144
6.48.3.3 getX() . . . 144
6.48.3.4 getY() . . . 145
6.48.3.5 getZ() . . . 145
6.48.3.6 invert() . . . 145
6.48.3.7 length() . . . 145
6.48.3.8 near_zero() . . . 146
6.48.3.9 normalize() . . . 146
6.48.3.10 operator∗() [1/2] . . . 146
6.48.3.11 operator∗() [2/2] . . . 146
6.48.3.12 operator∗=() [1/2] . . . 146
6.48.3.13 operator∗=() [2/2] . . . 147
6.48.3.14 operator+() . . . 147
6.48.3.15 operator+=() . . . 147
6.48.3.16 operator-() [1/2] . . . 148
6.48.3.17 operator-() [2/2] . . . 148
6.48.3.18 operator-=() . . . 148
6.48.3.19 operator/() [1/2] . . . 148
6.48.3.20 operator/() [2/2] . . . 149
6.48.3.21 operator/=() [1/2] . . . 149
6.48.3.22 operator/=() [2/2] . . . 149
6.48.3.23 operator==() [1/2] . . . 150
6.48.3.24 operator==() [2/2] . . . 150

6.48.3.25 random() [1/2] . . . 150
6.48.3.26 random() [2/2] . . . 150
6.48.3.27 random_double() [1/2] . . . 151
6.48.3.28 random_double() [2/2] . . . 151
6.48.3.29 reflect() . . . 151
6.48.3.30 shearToMatrix() . . . 151
6.48.3.31 vectorToMatrix() . . . 151

7 File Documentation **153**
7.1 include/core/Camera.hpp File Reference . . . 153
7.2 Camera.hpp . . . 153
7.3 include/core/Core.hpp File Reference . . . 154
7.3.1 Detailed Description . . . 154
7.4 Core.hpp . . . 155
7.5 include/core/Gui.hpp File Reference . . . 156
7.5.1 Detailed Description . . . 156
7.6 Gui.hpp . . . 156
7.7 include/core/Render.hpp File Reference . . . 157
7.7.1 Detailed Description . . . 157
7.8 Render.hpp . . . 158
7.9 include/core/Scene.hpp File Reference . . . 159
7.9.1 Detailed Description . . . 159
7.10 Scene.hpp . . . 159
7.11 include/core/Screen.hpp File Reference . . . 160
7.12 Screen.hpp . . . 160
7.13 include/Define.hpp File Reference . . . 161
7.13.1 Macro Definition Documentation . . . 161
7.13.1.1 BACKGROUNDCOLOR . . . 161
7.13.1.2 COLOR_THRESHOLD . . . 162
7.13.1.3 FAST_RENDER_RATIO . . . 162
7.13.1.4 FONT . . . 162
7.13.1.5 MAX_DEPTH . . . 162
7.13.1.6 OCCLUSION_SAMPLE_PX . . . 162
7.13.1.7 RENDER_COLOMNS_DIVISION_PIXEL . . . 162
7.13.1.8 ROTATE_X . . . 162
7.13.1.9 ROTATE_Y . . . 162
7.13.1.10 ROTATE_Z . . . 162
7.13.1.11 THREADS_NUMBER . . . 162
7.13.1.12 TRANSLATE_X . . . 163
7.13.1.13 TRANSLATE_Y . . . 163
7.13.1.14 TRANSLATE_Z . . . 163
7.14 Define.hpp . . . 163

7.15 include/exceptions/Exception.hpp File Reference . . . 164
7.15.1 Detailed Description . . . 164
7.16 Exception.hpp . . . 164
7.17 include/images/Image.hpp File Reference . . . 165
7.17.1 Detailed Description . . . 165
7.18 Image.hpp . . . 165
7.19 include/interfaces/ILight.hpp File Reference . . . 166
7.19.1 Detailed Description . . . 166
7.20 ILight.hpp . . . 166
7.21 include/interfaces/IPrimitive.hpp File Reference . . . 167
7.21.1 Detailed Description . . . 167
7.22 IPrimitive.hpp . . . 167
7.23 include/interfaces/ITransformations.hpp File Reference . . . 168
7.23.1 Detailed Description . . . 168
7.24 ITransformations.hpp . . . 168
7.25 include/loader/ObjParser.hpp File Reference . . . 168
7.26 ObjParser.hpp . . . 169
7.27 include/loader/SceneLoader.hpp File Reference . . . 169
7.27.1 Detailed Description . . . 170
7.28 SceneLoader.hpp . . . 170
7.29 include/materials/AMaterial.hpp File Reference . . . 170
7.30 AMaterial.hpp . . . 171
7.31 include/materials/Chessboard.hpp File Reference . . . 171
7.32 Chessboard.hpp . . . 172
7.33 include/materials/Default.hpp File Reference . . . 172
7.34 Default.hpp . . . 172
7.35 include/materials/Metal.hpp File Reference . . . 173
7.36 Metal.hpp . . . 173
7.37 include/materials/Mtl.hpp File Reference . . . 173
7.38 Mtl.hpp . . . 173
7.39 include/materials/PerlinNoise.hpp File Reference . . . 174
7.40 PerlinNoise.hpp . . . 174
7.41 include/materials/Transparent.hpp File Reference . . . 174
7.42 Transparent.hpp . . . 175
7.43 include/plugins/lights/AmbientLight.hpp File Reference . . . 175
7.43.1 Function Documentation . . . 175
7.43.1.1 createLight() . . . 175
7.43.1.2 registerPlugin() . . . 176
7.44 AmbientLight.hpp . . . 176
7.45 include/plugins/lights/DirectionalLight.hpp File Reference . . . 177
7.45.1 Function Documentation . . . 177
7.45.1.1 createLight() . . . 177

7.45.1.2 registerPlugin() . . . 178
7.46 DirectionalLight.hpp . . . 178
7.47 include/plugins/lights/PointLight.hpp File Reference . . . 179
7.47.1 Function Documentation . . . 179
7.47.1.1 createLight() . . . 179
7.47.1.2 registerPlugin() . . . 180
7.48 PointLight.hpp . . . 180
7.49 include/plugins/PluginsFactory.hpp File Reference . . . 181
7.49.1 Detailed Description . . . 181
7.49.2 Typedef Documentation . . . 182
7.49.2.1 LightsFactory . . . 182
7.49.2.2 PrimitivesFactory . . . 182
7.50 PluginsFactory.hpp . . . 182
7.51 include/plugins/PluginsHandle.hpp File Reference . . . 182
7.51.1 Detailed Description . . . 183
7.52 PluginsHandle.hpp . . . 183
7.53 include/plugins/PluginsManager.hpp File Reference . . . 183
7.53.1 Detailed Description . . . 184
7.54 PluginsManager.hpp . . . 184
7.55 include/plugins/primitives/Cone.hpp File Reference . . . 185
7.55.1 Function Documentation . . . 185
7.55.1.1 createPrimitive() . . . 185
7.55.1.2 registerPlugin() . . . 185
7.56 Cone.hpp . . . 186
7.57 include/plugins/primitives/Cylindre.hpp File Reference . . . 187
7.57.1 Function Documentation . . . 188
7.57.1.1 createPrimitive() . . . 188
7.57.1.2 registerPlugin() . . . 188
7.58 Cylindre.hpp . . . 188
7.59 include/plugins/primitives/InfinitCone.hpp File Reference . . . 190
7.59.1 Function Documentation . . . 190
7.59.1.1 createPrimitive() . . . 190
7.59.1.2 registerPlugin() . . . 191
7.60 InfinitCone.hpp . . . 191
7.61 include/plugins/primitives/InfiniteCylindre.hpp File Reference . . . 192
7.61.1 Function Documentation . . . 193
7.61.1.1 createPrimitive() . . . 193
7.61.1.2 registerPlugin() . . . 193
7.62 InfiniteCylindre.hpp . . . 194
7.63 include/plugins/primitives/Plane.hpp File Reference . . . 195
7.63.1 Function Documentation . . . 196
7.63.1.1 createPrimitive() . . . 196

7.63.1.2 registerPlugin() . . . 196
7.64 Plane.hpp . . . 196
7.65 include/plugins/primitives/Sphere.hpp File Reference . . . 198
7.65.1 Function Documentation . . . 198
7.65.1.1 createPrimitive() . . . 198
7.65.1.2 registerPlugin() . . . 199
7.66 Sphere.hpp . . . 199
7.67 include/plugins/primitives/Triangle.hpp File Reference . . . 200
7.67.1 Function Documentation . . . 201
7.67.1.1 createPrimitive() . . . 201
7.67.1.2 registerPlugin() . . . 201
7.68 Triangle.hpp . . . 202
7.69 include/transformations/Rotation.hpp File Reference . . . 203
7.69.1 Detailed Description . . . 204
7.70 Rotation.hpp . . . 204
7.71 include/transformations/Shear.hpp File Reference . . . 204
7.71.1 Detailed Description . . . 204
7.72 Shear.hpp . . . 205
7.73 include/transformations/Translation.hpp File Reference . . . 205
7.73.1 Detailed Description . . . 205
7.74 Translation.hpp . . . 206
7.75 include/utilitaries/Color.hpp File Reference . . . 206
7.75.1 Detailed Description . . . 206
7.75.2 Function Documentation . . . 207
7.75.2.1 operator<<() . . . 207
7.76 Color.hpp . . . 207
7.77 include/utilitaries/Cursor.hpp File Reference . . . 207
7.77.1 Enumeration Type Documentation . . . 208
7.77.1.1 CursorType . . . 208
7.78 Cursor.hpp . . . 208
7.79 include/utilitaries/Face.hpp File Reference . . . 209
7.80 Face.hpp . . . 209
7.81 include/utilitaries/HitInfo.hpp File Reference . . . 210
7.81.1 Detailed Description . . . 210
7.82 HitInfo.hpp . . . 210
7.83 include/utilitaries/Matrix.hpp File Reference . . . 211
7.84 Matrix.hpp . . . 211
7.85 include/utilitaries/Point.hpp File Reference . . . 212
7.86 Point.hpp . . . 212
7.87 include/utilitaries/Ray.hpp File Reference . . . 213
7.87.1 Detailed Description . . . 214
7.88 Ray.hpp . . . 214

7.89 include/utilitaries/Vector.hpp File Reference . . . 214
7.89.1 Detailed Description . . . 215
7.90 Vector.hpp . . . 215
7.91 obj/core/CalculateRender.d File Reference . . . 218
7.92 obj/core/Camera.d File Reference . . . 218
7.93 obj/core/Core.d File Reference . . . 218
7.94 obj/core/Gui.d File Reference . . . 218
7.95 obj/core/hotreload.d File Reference . . . 218
7.96 obj/core/Main.d File Reference . . . 218
7.97 obj/core/PluginsHandle.d File Reference . . . 218
7.98 obj/core/PluginsManager.d File Reference . . . 218
7.99 obj/core/Render.d File Reference . . . 218
7.100 obj/core/Scene.d File Reference . . . 218
7.101 obj/core/Screen.d File Reference . . . 218
7.102 obj/exceptions/Exception.d File Reference . . . 218
7.103 obj/images/Image.d File Reference . . . 218
7.104 obj/loader/ObjParser.d File Reference . . . 218
7.105 obj/loader/SceneLoader.d File Reference . . . 218
7.106 obj/materials/AMaterial.d File Reference . . . 218
7.107 obj/materials/Chessboard.d File Reference . . . 218
7.108 obj/materials/Default.d File Reference . . . 218
7.109 obj/materials/Metal.d File Reference . . . 218
7.110 obj/materials/PerlinNoise.d File Reference . . . 218
7.111 obj/materials/Transparent.d File Reference . . . 218
7.112 obj/transformation/Rotation.d File Reference . . . 218
7.113 obj/transformation/Shear.d File Reference . . . 218
7.114 obj/transformation/Translation.d File Reference . . . 218
7.115 obj/utilitaries/Color.d File Reference . . . 218
7.116 obj/utilitaries/Cursor.d File Reference . . . 218
7.117 obj/utilitaries/Face.d File Reference . . . 218
7.118 obj/utilitaries/HitInfo.d File Reference . . . 218
7.119 obj/utilitaries/Point.d File Reference . . . 218
7.120 obj/utilitaries/Ray.d File Reference . . . 218
7.121 obj/utilitaries/Vector.d File Reference . . . 218
7.122 src/core/CalculateRender.cpp File Reference . . . 218
7.122.1 Detailed Description . . . 219
7.122.2 Function Documentation . . . 219
7.122.2.1 orthogonalVector() . . . 219
7.122.2.2 random_double() . . . 219
7.123 src/core/Camera.cpp File Reference . . . 219
7.123.1 Detailed Description . . . 219
7.124 src/core/Core.cpp File Reference . . . 220

7.125 src/core/Gui.cpp File Reference . . . 220
7.125.1 Detailed Description . . . 220
7.126 src/core/hotreload.cpp File Reference . . . 220
7.126.1 Detailed Description . . . 220
7.127 src/core/Main.cpp File Reference . . . 220
7.127.1 Detailed Description . . . 221
7.127.2 Function Documentation . . . 221
7.127.2.1 help() . . . 221
7.127.2.2 main() . . . 221
7.128 src/core/PluginsHandle.cpp File Reference . . . 222
7.129 src/core/PluginsManager.cpp File Reference . . . 222
7.129.1 Detailed Description . . . 222
7.130 src/core/Render.cpp File Reference . . . 222
7.130.1 Detailed Description . . . 222
7.131 src/core/Scene.cpp File Reference . . . 222
7.131.1 Detailed Description . . . 222
7.132 src/core/Screen.cpp File Reference . . . 223
7.133 src/exceptions/Exception.cpp File Reference . . . 223
7.134 src/images/Image.cpp File Reference . . . 223
7.134.1 Detailed Description . . . 223
7.135 src/loader/ObjParser.cpp File Reference . . . 223
7.136 src/loader/SceneLoader.cpp File Reference . . . 223
7.136.1 Detailed Description . . . 224
7.137 src/materials/AMaterial.cpp File Reference . . . 224
7.138 src/materials/Chessboard.cpp File Reference . . . 224
7.139 src/materials/Default.cpp File Reference . . . 224
7.140 src/materials/Metal.cpp File Reference . . . 224
7.141 src/materials/PerlinNoise.cpp File Reference . . . 224
7.142 src/materials/Transparent.cpp File Reference . . . 224
7.143 src/plugins/lights/AmbientLight.cpp File Reference . . . 225
7.143.1 Detailed Description . . . 225
7.144 src/plugins/lights/DirectionalLight.cpp File Reference . . . 225
7.144.1 Detailed Description . . . 225
7.145 src/plugins/lights/PointLight.cpp File Reference . . . 225
7.145.1 Detailed Description . . . 225
7.146 src/plugins/primitives/Cone.cpp File Reference . . . 226
7.146.1 Detailed Description . . . 226
7.147 src/plugins/primitives/Cylindre.cpp File Reference . . . 226
7.147.1 Detailed Description . . . 226
7.148 src/plugins/primitives/InfinitCone.cpp File Reference . . . 226
7.148.1 Detailed Description . . . 227
7.149 src/plugins/primitives/InfiniteCylindre.cpp File Reference . . . 227

7.149.1 Detailed Description . . . 227
7.150 src/plugins/primitives/Plane.cpp File Reference . . . 227
7.150.1 Detailed Description . . . 227
7.151 src/plugins/primitives/Sphere.cpp File Reference . . . 228
7.151.1 Detailed Description . . . 228
7.152 src/plugins/primitives/Triangle.cpp File Reference . . . 228
7.152.1 Detailed Description . . . 228
7.153 src/transformation/Rotation.cpp File Reference . . . 228
7.154 src/transformation/Shear.cpp File Reference . . . 228
7.155 src/transformation/Translation.cpp File Reference . . . 229
7.155.1 Detailed Description . . . 229
7.156 src/utilitaries/Color.cpp File Reference . . . 229
7.156.1 Detailed Description . . . 229
7.156.2 Function Documentation . . . 229
7.156.2.1 operator<<() . . . 229
7.157 src/utilitaries/Cursor.cpp File Reference . . . 230
7.158 src/utilitaries/Face.cpp File Reference . . . 230
7.158.1 Detailed Description . . . 230
7.159 src/utilitaries/HitInfo.cpp File Reference . . . 230
7.159.1 Detailed Description . . . 230
7.160 src/utilitaries/Point.cpp File Reference . . . 230
7.161 src/utilitaries/Ray.cpp File Reference . . . 231
7.162 src/utilitaries/Vector.cpp File Reference . . . 231
7.162.1 Detailed Description . . . 231

Index **233**

Chapter 1

Namespace Index

1.1 Namespace List

Here is a list of all namespaces with brief descriptions:

Math 11

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

AMaterial . . . 13
- ChessboardMaterial . . . 24
- Default . . . 49
- Metal . . . 82
- Mtl . . . 83
- PerlinNoiseMaterial . . . 86
- Transparent . . . 137

Camera . . . 20
Color . . . 26
Core . . . 38
Cursor . . . 41
Gui . . . 57
HitInfo . . . 59
ILight . . . 62
- AmbientLight . . . 17
- DirectionalLight . . . 50
- PointLight . . . 104

Image . . . 65
IPrimitive . . . 75
- Cone . . . 33
- Cylindre . . . 46
- Face . . . 54
- InfinitCone . . . 68
- InfiniteCylindre . . . 71
- Plane . . . 88
- Sphere . . . 130
- Triangle . . . 139

ITransformations . . . 77
- Rotation . . . 118
- Shear . . . 128
- Translation . . . 134

Math::Matrix< X, Y > . . . 79
ObjParser . . . 85
PlugInException . . . 92

PluginsHandle 93
PluginsManager 96
Math::Point 99
Ray 108
Render 110
std::runtime_error
    ImageException 67
    PluginsException 92
    RaytracerException 110
    RenderException 117
    SceneException 124
Scene 121
SceneLoader 125
Screen 127
Math::Vector 142

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

AMaterial 13

AmbientLight
A class to represent an ambient light source in the raytracer project 17

Camera
A class to represent a camera in the raytracer project 20

ChessboardMaterial
Une classe pour représenter un matériau avec un pattern d'échiquier procédural 24

Color
A class to represent a color in RGB format 26

Cone 33

Core
A class to manage the raytracer project 38

Cursor 41

Cylindre 46

Default 49

DirectionalLight
A class to represent a directional light source in the raytracer project 50

Face
A class to represent a Face in 3D space 54

Gui 57

HitInfo
A class to represent hit information in the raytracer project 59

ILight
An interface for light sources in the raytracer project 62

Image
A class to represent an image in the raytracer project 65

ImageException
Exception class for the raytracer project 67

InfinitCone 68

InfiniteCylindre 71

IPrimitive
An interface for primitive shapes in the raytracer project 75

ITransformations
An interface for transformations in the raytracer project 77

Math::Matrix< X, Y >
A class to use Matrix for calculus 79

Metal . . . 82
Mtl . . . 83
ObjParser
A class to parse .obj files and create shapes for the raytracer . . . 85
PerlinNoiseMaterial . . . 86
Plane
A class to represent a plane in 3D space . . . 88
PlugInException
Exception class for the raytracer project . . . 92
PluginsException . . . 92
PluginsHandle
A class to manage plugins in the raytracer project . . . 93
PluginsManager
A class to manage plugins in the raytracer project . . . 96
Math::Point
A class to represent a 3D point with basic operations . . . 99
PointLight
A class to represent a point light source in the raytracer project . . . 104
Ray
A class to represent a ray in 3D space . . . 108
RaytracerException
Exception class for the raytracer project . . . 110
Render
A class to render the scene in the raytracer project . . . 110
RenderException
Exception class for the raytracer project . . . 117
Rotation
A class to represent a Rotation transformation . . . 118
Scene
A class to represent a scene in the raytracer project . . . 121
SceneException
Exception class for the raytracer project . . . 124
SceneLoader
A class to load scenes from files in the raytracer project . . . 125
Screen . . . 127
Shear
A class to represent a Shear transformation . . . 128
Sphere
A class to represent a Sphere in 3D space . . . 130
Translation
A class to represent a translation transformation . . . 134
Transparent . . . 137
Triangle
A class to represent a Triangle in 3D space . . . 139
Math::Vector
A class to represent a 3D vector with basic operations . . . 142

Chapter 4

File Index

4.1 File List

Here is a list of all files with brief descriptions:

include/Define.hpp . . . 161
include/core/Camera.hpp . . . 153
include/core/Core.hpp
Core class header file . . . 154
include/core/Gui.hpp
GUI class header file . . . 156
include/core/Render.hpp
Render class header file . . . 157
include/core/Scene.hpp
Scene class header file . . . 159
include/core/Screen.hpp . . . 160
include/exceptions/Exception.hpp
Exception class for the raytracer project . . . 164
include/images/Image.hpp
Image class header file . . . 165
include/interfaces/ILight.hpp
ILight class header file . . . 166
include/interfaces/IPrimitive.hpp
IPrimitive class header file . . . 167
include/interfaces/ITransformations.hpp
ITransformations interface header file . . . 168
include/loader/ObjParser.hpp . . . 168
include/loader/SceneLoader.hpp
SceneLoader class header file . . . 169
include/materials/AMaterial.hpp . . . 170
include/materials/Chessboard.hpp . . . 171
include/materials/Default.hpp . . . 172
include/materials/Metal.hpp . . . 173
include/materials/Mtl.hpp . . . 173
include/materials/PerlinNoise.hpp . . . 174
include/materials/Transparent.hpp . . . 174
include/plugins/PluginsFactory.hpp
File that declares the factory type for the plugins . . . 181
include/plugins/PluginsHandle.hpp
Header file for the PluginsHandle class . . . 182

include/plugins/PluginsManager.hpp
PluginsManager class header file . . . 183
include/plugins/lights/AmbientLight.hpp . . . 175
include/plugins/lights/DirectionalLight.hpp . . . 177
include/plugins/lights/PointLight.hpp . . . 179
include/plugins/primitives/Cone.hpp . . . 185
include/plugins/primitives/Cylindre.hpp . . . 187
include/plugins/primitives/InfinitCone.hpp . . . 190
include/plugins/primitives/InfiniteCylindre.hpp . . . 192
include/plugins/primitives/Plane.hpp . . . 195
include/plugins/primitives/Sphere.hpp . . . 198
include/plugins/primitives/Triangle.hpp . . . 200
include/transformations/Rotation.hpp
Rotation class header file . . . 203
include/transformations/Shear.hpp
Shear class header file . . . 204
include/transformations/Translation.hpp
Translation class header file . . . 205
include/utilitaries/Color.hpp
Color class header file . . . 206
include/utilitaries/Cursor.hpp . . . 207
include/utilitaries/Face.hpp . . . 209
include/utilitaries/HitInfo.hpp
HitInfo class header file . . . 210
include/utilitaries/Matrix.hpp . . . 211
include/utilitaries/Point.hpp . . . 212
include/utilitaries/Ray.hpp
Ray class header file . . . 213
include/utilitaries/Vector.hpp
Vector class header file . . . 214
obj/core/CalculateRender.d . . . 218
obj/core/Camera.d . . . 218
obj/core/Core.d . . . 218
obj/core/Gui.d . . . 218
obj/core/hotreload.d . . . 218
obj/core/Main.d . . . 218
obj/core/PluginsHandle.d . . . 218
obj/core/PluginsManager.d . . . 218
obj/core/Render.d . . . 218
obj/core/Scene.d . . . 218
obj/core/Screen.d . . . 218
obj/exceptions/Exception.d . . . 218
obj/images/Image.d . . . 218
obj/loader/ObjParser.d . . . 218
obj/loader/SceneLoader.d . . . 218
obj/materials/AMaterial.d . . . 218
obj/materials/Chessboard.d . . . 218
obj/materials/Default.d . . . 218
obj/materials/Metal.d . . . 218
obj/materials/PerlinNoise.d . . . 218
obj/materials/Transparent.d . . . 218
obj/transformation/Rotation.d . . . 218
obj/transformation/Shear.d . . . 218
obj/transformation/Translation.d . . . 218
obj/utilitaries/Color.d . . . 218
obj/utilitaries/Cursor.d . . . 218
obj/utilitaries/Face.d . . . 218
obj/utilitaries/HitInfo.d . . . 218

obj/utilitaries/Point.d . . . 218
obj/utilitaries/Ray.d . . . 218
obj/utilitaries/Vector.d . . . 218
src/core/CalculateRender.cpp
Render class implementation file . . . 218
src/core/Camera.cpp
Camera class implementation . . . 219
src/core/Core.cpp . . . 220
src/core/Gui.cpp
GUI class implementation file . . . 220
src/core/hotreload.cpp
Hot reload implementation for the raytracer project . . . 220
src/core/Main.cpp
Main file for the raytracer project . . . 220
src/core/PluginsHandle.cpp . . . 222
src/core/PluginsManager.cpp
Implementation of the PluginsManager class . . . 222
src/core/Render.cpp
Render class implementation file . . . 222
src/core/Scene.cpp
Scene class implementation . . . 222
src/core/Screen.cpp . . . 223
src/exceptions/Exception.cpp . . . 223
src/images/Image.cpp
Image class implementation . . . 223
src/loader/ObjParser.cpp . . . 223
src/loader/SceneLoader.cpp
SceneLoader class implementation . . . 223
src/materials/AMaterial.cpp . . . 224
src/materials/Chessboard.cpp . . . 224
src/materials/Default.cpp . . . 224
src/materials/Metal.cpp . . . 224
src/materials/PerlinNoise.cpp . . . 224
src/materials/Transparent.cpp . . . 224
src/plugins/lights/AmbientLight.cpp
AmbientLight class implementation . . . 225
src/plugins/lights/DirectionalLight.cpp
DirectionalLight class implementation . . . 225
src/plugins/lights/PointLight.cpp
PointLight class implementation . . . 225
src/plugins/primitives/Cone.cpp
Cone class implementation . . . 226
src/plugins/primitives/Cylindre.cpp
Cylindre class implementation . . . 226
src/plugins/primitives/InfinitCone.cpp
InfinitCone class implementation . . . 226
src/plugins/primitives/InfiniteCylindre.cpp
InfiniteCylindre class implementation . . . 227
src/plugins/primitives/Plane.cpp
Plane class implementation . . . 227
src/plugins/primitives/Sphere.cpp
Sphere class implementation . . . 228
src/plugins/primitives/Triangle.cpp
Triangle class implementation . . . 228
src/transformation/Rotation.cpp . . . 228
src/transformation/Shear.cpp . . . 228
src/transformation/Translation.cpp
Translation class implementation file . . . 229

src/utilitaries/Color.cpp
Color class implementation 229
src/utilitaries/Cursor.cpp 230
src/utilitaries/Face.cpp
Face class implementation 230
src/utilitaries/HitInfo.cpp
HitInfo class implementation 230
src/utilitaries/Point.cpp 230
src/utilitaries/Ray.cpp 231
src/utilitaries/Vector.cpp
Vector class implementation 231

Chapter 5

Namespace Documentation

5.1 Math Namespace Reference

Classes

- class Matrix

  A class to use Matrix for calculus.
- class Point

  A class to represent a 3D point with basic operations.
- class Vector

  A class to represent a 3D vector with basic operations.

Functions

- Vector operator∗ (double scalar, const Vector &vector)

  Scalar multiplication operator overload (commutative).

5.1.1 Function Documentation

5.1.1.1 operator∗()

```
Vector Math::operator* (
            double scalar,
            const Vector & vector)
```

Scalar multiplication operator overload (commutative).

Parameters

scalar	The scalar to multiply with.
vector	The vector to multiply with the scalar.

Returns

A new vector that is the product of the scalar and the vector.

Chapter 6

Class Documentation

6.1 AMaterial Class Reference

```
#include <AMaterial.hpp>
```

Inheritance diagram for AMaterial:

Public Member Functions

- AMaterial (const std::string &name, float ambient, float diffuse, float specular, float shininess)

  Construct a new AMaterial object with given coefficients.
- virtual ~AMaterial ()=default
- const std::string & getName () const
- float getAmbient () const
- float getDiffuse () const
- float getSpecular () const
- float getShininess () const
- float getTransparency () const
- float getRefractionIndex () const
- void setDiffuseColor (const Color &color)

  Set the diffuse color of the material.
- const Color & getDiffuseColor () const

  Get the diffuse color of the material.
- void setSpecularColor (const Color &color)

  Set the specular color of the material.
- const Color & getSpecularColor () const

  Get the specular color of the material.
- void setShininess (float shininess)

  Set the shininess of the material.
- virtual Color getColorAt (const Math::Point &point) const

  Get the color at a specific point on the surface (for procedural textures)
- void setTransparency (float transparency)

  Set the transparency of the material.
- void setRefractionIndex (float refractionIndex)

  Set the refraction index of the material.
- void loadFromMtlLine (const std::string &line)

  Load material properties from a .mtl file line.

6.1.1 Constructor & Destructor Documentation

6.1.1.1 AMaterial()

```
AMaterial::AMaterial (
            const std::string & name,
            float ambient,
            float diffuse,
            float specular,
            float shininess)
```

Construct a new AMaterial object with given coefficients.

Parameters

name	Material name identifier.
ambient	Ambient coefficient (ka).
diffuse	Diffuse coefficient (kd).
specular	Specular coefficient (ks).
shininess	Shininess exponent (alpha).

6.1.1.2 ~AMaterial()

```
virtual AMaterial::~AMaterial () [virtual], [default]
```

6.1.2 Member Function Documentation

6.1.2.1 getAmbient()

```
float AMaterial::getAmbient () const
```

6.1.2.2 getColorAt()

```
Color AMaterial::getColorAt (
            const Math::Point & point) const [virtual]
```

Get the color at a specific point on the surface (for procedural textures)

Parameters

point	The point on the surface

Returns

The color at that point

By default, returns the diffuse color. Override this in derived classes to implement procedural textures like the chessboard pattern.

Reimplemented in ChessboardMaterial, and PerlinNoiseMaterial.

6.1.2.3 getDiffuse()

```
float AMaterial::getDiffuse () const
```

6.1.2.4 getDiffuseColor()

```
const Color & AMaterial::getDiffuseColor () const
```

Get the diffuse color of the material.

Returns

The diffuse color.

6.1.2.5 getName()

```
const std::string & AMaterial::getName () const
```

6.1.2.6 getRefractionIndex()

```
float AMaterial::getRefractionIndex () const
```

6.1.2.7 getShininess()

```
float AMaterial::getShininess () const
```

6.1.2.8 getSpecular()

```
float AMaterial::getSpecular () const
```

6.1.2.9 getSpecularColor()

```
const Color & AMaterial::getSpecularColor () const
```

Get the specular color of the material.

Returns

The specular color.

6.1.2.10 getTransparency()

```
float AMaterial::getTransparency () const
```

6.1.2.11 loadFromMtlLine()

```
void AMaterial::loadFromMtlLine (
            const std::string & line)
```

Load material properties from a .mtl file line.

Parameters

line	The line from the .mtl file.

6.1.2.12 setDiffuseColor()

```
void AMaterial::setDiffuseColor (
            const Color & color)
```

Set the diffuse color of the material.

Parameters

color	The diffuse color.

6.1.2.13 setRefractionIndex()

```
void AMaterial::setRefractionIndex (
            float refractionIndex)
```

Set the refraction index of the material.

Parameters

refractionIndex	The refraction index value.

6.1.2.14 setShininess()

```
void AMaterial::setShininess (
            float shininess)
```

Set the shininess of the material.

Parameters

shininess	The shininess value.

6.1.2.15 setSpecularColor()

```
void AMaterial::setSpecularColor (
            const Color & color)
```

Set the specular color of the material.

Parameters

color	The specular color.

6.1.2.16 setTransparency()

```
void AMaterial::setTransparency (
            float transparency)
```

Set the transparency of the material.

Parameters

transparency	The transparency value (0 = opaque, 1 = fully transparent).

The documentation for this class was generated from the following files:

- include/materials/AMaterial.hpp
- src/materials/AMaterial.cpp

6.2 AmbientLight Class Reference

A class to represent an ambient light source in the raytracer project.

```
#include <AmbientLight.hpp>
```

Inheritance diagram for AmbientLight:

Collaboration diagram for AmbientLight:

Public Member Functions

- AmbientLight (float intensity, const Color &color)

  Constructor for the AmbientLight class.
- const std::string & getType () const override

  Get the type of the light source.
- const Math::Point & getPosition () const override

  Get the position of the light source.
- const Math::Vector & getDirection () const override

  Get the direction of the light source.
- float getIntensity () const override

  Get the intensity of the light source.
- const Color & getColor () const override

  Get the color of the light source.
- const Color & getAmbientComponent () const override

  Get the ambient component of the light source.
- const Color & getDiffuseComponent () const override

  Get the diffuse component of the light source.
- const Color & getSpecularComponent () const override

  Get the specular component of the light source.

Public Member Functions inherited from ILight

- virtual ~ILight ()=default

  Virtual destructor for ILight class.

6.2.1 Detailed Description

A class to represent an ambient light source in the raytracer project.

6.2.2 Constructor & Destructor Documentation

6.2.2.1 AmbientLight()

```
AmbientLight::AmbientLight (
            float intensity,
            const Color & color)
```

Constructor for the AmbientLight class.

Parameters

intensity	The intensity of the light source.

6.2.3 Member Function Documentation

6.2.3.1 getAmbientComponent()

```
const Color & AmbientLight::getAmbientComponent () const  [override], [virtual]
```

Get the ambient component of the light source.

Returns

The ambient component of the light source.

Implements ILight.

6.2.3.2 getColor()

```
const Color & AmbientLight::getColor () const  [override], [virtual]
```

Get the color of the light source.

Returns

The color of the light source.

Implements ILight.

6.2.3.3 getDiffuseComponent()

```
const Color & AmbientLight::getDiffuseComponent () const  [override], [virtual]
```

Get the diffuse component of the light source.

Returns

The diffuse component of the light source.

Implements ILight.

6.2.3.4 getDirection()

```
const Math::Vector & AmbientLight::getDirection () const  [override], [virtual]
```

Get the direction of the light source.

Returns

The direction of the light source.

Implements ILight.

6.2.3.5 getIntensity()

```
float AmbientLight::getIntensity () const  [override], [virtual]
```

Get the intensity of the light source.

Returns

The intensity of the light source.

Implements ILight.

6.2.3.6 getPosition()

```
const Math::Point & AmbientLight::getPosition () const  [override], [virtual]
```

Get the position of the light source.

Returns

The position of the light source.

Implements ILight.

6.2.3.7 getSpecularComponent()

```
const Color & AmbientLight::getSpecularComponent () const [override], [virtual]
```

Get the specular component of the light source.

Returns

The specular component of the light source.

Implements ILight.

6.2.3.8 getType()

```
const std::string & AmbientLight::getType () const [override], [virtual]
```

Get the type of the light source.

Returns

The type of the light source.

Implements ILight.

The documentation for this class was generated from the following files:

- include/plugins/lights/AmbientLight.hpp
- src/plugins/lights/AmbientLight.cpp

6.3 Camera Class Reference

A class to represent a camera in the raytracer project.

```
#include <Camera.hpp>
```

Public Member Functions

- Camera (const Math::Vector &position=Math::Vector(), const Math::Vector &rotation=Math::Vector(), double fov=90.0f, int width=800, int height=600)

  Default constructor for Camera class. Initializes the camera with a position and rotation.
- const Math::Point & getPosition () const

  Get the position of the camera.
- double getFov () const

  Get the field of view of the camera.
- int getWidth () const

  Get the width of the camera.
- int getHeight () const

  Get the height of the camera.
- void setRay (double x, double y)

  Set the attribute of a ray.
- Ray getRay () const

  Get the current ray send by the camera.
- void setPosition (const Math::Point &position)

  Set the position of the camera.
- void translate (Math::Vector translation)

  Translate the camera base on a vector.
- void rotate (double angle, std::string axis)
- void shear (double strength)

6.3.1 Detailed Description

A class to represent a camera in the raytracer project.

The Camera class contains a position and a direction vector.

6.3.2 Constructor & Destructor Documentation

6.3.2.1 Camera()

```
Camera::Camera (
            const Math::Vector & position = Math::Vector(),
            const Math::Vector & rotation = Math::Vector(),
            double fov = 90.0f,
            int width = 800,
            int height = 600)
```

Default constructor for Camera class. Initializes the camera with a position and rotation.

Constructor for the Camera class.

Parameters

position	The position of the camera.
rotation	The rotation of the camera.

Note

If no parameters are provided, the camera is initialized to (0, 0, 0).

This constructor can be used to create a camera with specific position and rotation.

Parameters

position	The position of the camera in 3D space.
rotation	The rotation the camera is facing.
fov	The field of view of the camera.
width	The width of the camera's viewport.
height	The height of the camera's viewport.

6.3.3 Member Function Documentation

6.3.3.1 getFov()

```
double Camera::getFov () const
```

Get the field of view of the camera.

Returns

The field of view of the camera.

Note

This method returns the field of view of the camera in degrees.

Returns

The field of view of the camera in degrees.

6.3.3.2 getHeight()

```
int Camera::getHeight () const
```

Get the height of the camera.

Get the height of the camera's viewport.

Returns

The height of the camera.

Note

This method returns the height of the camera in pixels.

Returns

The height of the camera's viewport.

6.3.3.3 getPosition()

```
const Math::Point & Camera::getPosition () const
```

Get the position of the camera.

Returns

The position of the camera.

Note

This method returns the position point of the camera.

Returns

The position of the camera in 3D space.

6.3.3.4 getRay()

```
Ray Camera::getRay () const
```

Get the current ray send by the camera.

Returns

a copy of the pointer to the ray

6.3.3.5 getWidth()

```
int Camera::getWidth () const
```

Get the width of the camera.

Get the width of the camera's viewport.

Returns

The width of the camera.

Note

This method returns the width of the camera in pixels.

Returns

The width of the camera's viewport.

6.3.3.6 rotate()

```
void Camera::rotate (
            double angle,
            std::string axis)
```

6.3.3.7 setPosition()

```
void Camera::setPosition (
            const Math::Point & position)
```

Set the position of the camera.

Parameters

position	The new position of the camera.

Note

This method sets the position vector of the camera.

Parameters

position	The new position of the camera in 3D space.

6.3.3.8 setRay()

```
void Camera::setRay (
            double x,
            double y)
```

Set the attribute of a ray.

Parameters

origin	the origin of the ray
direction	the direction of the ray

Returns

The height of the camera

6.3.3.9 shear()

```
void Camera::shear (
            double strength)
```

6.3.3.10 translate()

```
void Camera::translate (
            Math::Vector translation)
```

Translate the camera base on a vector.

Parameters

translation	the vector that define the translation

The documentation for this class was generated from the following files:

- include/core/Camera.hpp
- src/core/Camera.cpp

6.4 ChessboardMaterial Class Reference

Une classe pour représenter un matériau avec un pattern d'échiquier procédural.

`#include <Chessboard.hpp>`

Inheritance diagram for ChessboardMaterial:

Collaboration diagram for ChessboardMaterial:

Public Member Functions

- ChessboardMaterial ()

  Constructeur par défaut de la classe ChessboardMaterial.
- ChessboardMaterial (const Color &color1, const Color &color2, double scale=1.0)

  Constructeur de la classe ChessboardMaterial.
- ~ChessboardMaterial () override=default

  Déstructeur par défaut.
- Color getColorAt (const Math::Point &point) const

  Calcule la couleur du matériau à un point donné selon un pattern d'échiquier.

Public Member Functions inherited from AMaterial

- AMaterial (const std::string &name, float ambient, float diffuse, float specular, float shininess)

  Construct a new AMaterial object with given coefficients.
- virtual ~AMaterial ()=default
- const std::string & getName () const
- float getAmbient () const
- float getDiffuse () const
- float getSpecular () const
- float getShininess () const
- float getTransparency () const
- float getRefractionIndex () const
- void setDiffuseColor (const Color &color)

  Set the diffuse color of the material.
- const Color & getDiffuseColor () const

  Get the diffuse color of the material.
- void setSpecularColor (const Color &color)

  Set the specular color of the material.
- const Color & getSpecularColor () const

  Get the specular color of the material.
- void setShininess (float shininess)

  Set the shininess of the material.
- void setTransparency (float transparency)

  Set the transparency of the material.
- void setRefractionIndex (float refractionIndex)

  Set the refraction index of the material.
- void loadFromMtlLine (const std::string &line)

  Load material properties from a .mtl file line.

6.4.1 Detailed Description

Une classe pour représenter un matériau avec un pattern d'échiquier procédural.

6.4.2 Constructor & Destructor Documentation

6.4.2.1 ChessboardMaterial() [1/2]

```
ChessboardMaterial::ChessboardMaterial ()
```

Constructeur par défaut de la classe ChessboardMaterial.

6.4.2.2 ChessboardMaterial() [2/2]

```
ChessboardMaterial::ChessboardMaterial (
            const Color & color1,
            const Color & color2,
            double scale = 1.0)
```

Constructeur de la classe ChessboardMaterial.

Parameters

color1	Première couleur de l'échiquier
color2	Deuxième couleur de l'échiquier
scale	Taille des cases (scale de la texture)

6.4.2.3 ~ChessboardMaterial()

```
ChessboardMaterial::~ChessboardMaterial () [override], [default]
```

Déstructeur par défaut.

6.4.3 Member Function Documentation

6.4.3.1 getColorAt()

```
Color ChessboardMaterial::getColorAt (
            const Math::Point & point) const [virtual]
```

Calcule la couleur du matériau à un point donné selon un pattern d'échiquier.

Parameters

point	Point d'intersection où calculer la couleur

Returns

Couleur à ce point

Reimplemented from AMaterial.

The documentation for this class was generated from the following files:

- include/materials/Chessboard.hpp
- src/materials/Chessboard.cpp

6.5 Color Class Reference

A class to represent a color in RGB format.

```
#include <Color.hpp>
```

Public Member Functions

- Color (double r=0, double g=0, double b=0)

  Default constructor for Color class. Initializes the color to (0, 0, 0).
- Color operator+ (const Color &other) const

  Addition operator overload.
- Color & operator+= (const Color &other)

  Addition operator overload.
- Color operator- (const Color &other) const

  Subtraction operator overload.
- Color operator∗ (double scalar) const

  Scalar multiplication operator overload.
- Color & operator∗= (double scalar)

  Scalar multiplication operator overload.
- Color & operator/= (int divide)
- Color operator∗ (const Color &other) const

  Multiplication operator overload.
- Color & operator∗= (const Color &other)

  Multiplication operator overload.
- Color operator/ (double scalar) const

  Scalar division operator overload.
- Color applyIntensity (double intensity) const

  Apply the color intensity.
- Color clamp () const

  Clamp the color values to be within the range [0, 255].
- double getR () const

  Get the red component of the color.
- double getG () const

  Get the green component of the color.
- double getB () const

  Get the blue component of the color.
- void setColor (double r, double g, double b)

  Set the blue component of the color.

6.5.1 Detailed Description

A class to represent a color in RGB format.

6.5.2 Constructor & Destructor Documentation

6.5.2.1 Color()

```
Color::Color (
            double r = 0,
            double g = 0,
            double b = 0)
```

Default constructor for Color class. Initializes the color to (0, 0, 0).

Parameters

r	The red component of the color.
g	The green component of the color.
b	The blue component of the color.

Note

If no parameters are provided, the color is initialized to (0, 0, 0).

This constructor can be used to create a color with specific RGB values.

Parameters

r	The red component of the color.
g	The green component of the color.
b	The blue component of the color.

Note

If no parameters are provided, the color is initialized to (0, 0, 0).

6.5.3 Member Function Documentation

6.5.3.1 applyIntensity()

```
Color Color::applyIntensity (
            double intensity) const
```

Apply the color intensity.

Parameters

intensity	The intensity to apply.

Returns

A new color that is the product of this color and the intensity.

6.5.3.2 clamp()

```
Color Color::clamp () const
```

Clamp the color values to be within the range [0, 255].

Clamp the color components to be in the range [0, 255].

Returns

A new color with clamped values.

A new color with clamped components.

6.5.3.3 getB()

```
double Color::getB () const
```

Get the blue component of the color.

Returns

The blue component of the color.

6.5.3.4 getG()

```
double Color::getG () const
```

Get the green component of the color.

Returns

The green component of the color.

6.5.3.5 getR()

```
double Color::getR () const
```

Get the red component of the color.

Returns

The red component of the color.

6.5.3.6 operator*() [1/2]

```
Color Color::operator* (
            const Color & other) const
```

Multiplication operator overload.

Parameters

other	The color to multiply with.

Returns

A new color that is the product of this color and the scalar.

Parameters

other	The color to multiply with.

Returns

A new color that is the product of this color and the other color.

6.5.3.7 operator∗() [2/2]

```
Color Color::operator* (
            double scalar) const
```

Scalar multiplication operator overload.

Parameters

scalar	The scalar to multiply with.

Returns

A new color that is the product of this color and the scalar.

6.5.3.8 operator∗=() [1/2]

```
Color & Color::operator*= (
            const Color & other)
```

Multiplication operator overload.

Parameters

other	The color to multiply with.

Returns

A reference to this color after multiplying by the other color.

6.5.3.9 operator∗=() [2/2]

```
Color & Color::operator*= (
            double scalar)
```

Scalar multiplication operator overload.

Parameters

scalar	The scalar to multiply with.

Returns

A reference to this color after multiplying by the scalar.

6.5.3.10 operator+()

```
Color Color::operator+ (
            const Color & other) const
```

Addition operator overload.

Get the red component of the color.

Parameters

other	The color to add.

Returns

A new color that is the sum of this color and the other color.

The red component of the color.

6.5.3.11 operator+=()

```
Color & Color::operator+= (
            const Color & other)
```

Addition operator overload.

Parameters

other	The color to add.

Returns

A reference to this color after adding the other color.

6.5.3.12 operator-()

```
Color Color::operator- (
            const Color & other) const
```

Subtraction operator overload.

Parameters

other	The color to subtract.

Returns

A new color that is the difference of this color and the other color.

6.5.3.13 operator/()

```
Color Color::operator/ (
            double scalar) const
```

Scalar division operator overload.

Parameters

scalar	The scalar to divide by.

Returns

A new color that is the quotient of this color and the scalar.

6.5.3.14 operator/=()

```
Color & Color::operator/= (
            int divide)
```

6.5.3.15 setColor()

```
void Color::setColor (
            double r,
            double g,
            double b)
```

Set the blue component of the color.

Returns

The blue component of the color.

The documentation for this class was generated from the following files:

- include/utilitaries/Color.hpp
- src/utilitaries/Color.cpp

6.6 Cone Class Reference

```
#include <Cone.hpp>
```

Inheritance diagram for Cone:

Collaboration diagram for Cone:

Public Member Functions

- Cone ()

  Default constructor for the cone shape. By default tip is at (0, 0, 0), height at (0, 0, 0) and angle at 0.
- Cone (Math::Point tip, Math::Vector height, double angle, Color &color, const std::shared_ptr< AMaterial > &mat)

  Constructor of the cone shape.
- ~Cone ()

  basic destructor of the Cone class
- const std::string & getType () const override

  Get the type of the primitive shape.
- bool intersect_base (const Ray &ray) const

  Check if a ray intersect with the base of the cone (represented by a circle)
- bool intersectCone (const Ray &ray) const

  Check if a ray intersect with the cone.
- bool intersect (const Ray &ray) const override

  Check if a ray intersects with the primitive shape.
- Math::Point getIntersectBase (const Ray &ray) const

  Get the intersection point with the base of the cone (represented by a circle)
- Math::Point getIntersectCone (const Ray &ray) const

  Get the intersection point with the cone.
- Math::Point getIntersection (const Ray &ray) const override

  This function does exactly what you think it does.
- Math::Vector normal_at (const Math::Point &point) const override

  Create a normal vector to the shape.
- Color getColor () const override

  Get the color of the primitive shape.
- std::shared_ptr< AMaterial > getMaterial () const override

  Get the material of the primitive shape.

Public Member Functions inherited from IPrimitive

- virtual ~IPrimitive ()=default

  Virtual destructor for IPrimitive class.

6.6.1 Constructor & Destructor Documentation

6.6.1.1 Cone() [1/2]

```
Cone::Cone ()
```

Default constructor for the cone shape. By default tip is at (0, 0, 0), height at (0, 0, 0) and angle at 0.

6.6.1.2 Cone() [2/2]

```
Cone::Cone (
            Math::Point tip,
            Math::Vector height,
            double angle,
            Color & color,
            const std::shared_ptr< AMaterial > & mat)
```

Constructor of the cone shape.

Parameters

tip	Point that represent the tip of the cone
height	height vector, the vector's origin is the tip of the cone
angle	the angle in degree represented between the height vector and the surface of the shape
color	The color of the cone.

Note

If you want a straight, x and z align axis cone a basic height vector
would looks like height = (0, -1, 0)

Parameters

tip	Point that represent the tip of the cone
height	height vector, the vector's origin is the tip of the cone
angle	the angle in degree represented between the height vector and the surface of the shape

Note

If you want a straight, x and z align axis cone a basic height vector
would looks like height = (0, -1, 0)

6.6.1.3 ~Cone()

```
Cone::~Cone ()
```

basic destructor of the Cone class

6.6.2 Member Function Documentation

6.6.2.1 getColor()

```
Color Cone::getColor () const  [override], [virtual]
```

Get the color of the primitive shape.

Returns

The color of the shape.

Implements IPrimitive.

6.6.2.2 getIntersectBase()

```
Math::Point Cone::getIntersectBase (
            const Ray & ray) const
```

Get the intersection point with the base of the cone (represented by a circle)

Parameters

ray	The ray to check for intersection.

Returns

the intersection point between the ray and the base

6.6.2.3 getIntersectCone()

```
Math::Point Cone::getIntersectCone (
            const Ray & ray) const
```

Get the intersection point with the cone.

Parameters

ray	The ray to check for intersection.

Returns

the intersection point between the ray and the cone

6.6.2.4 getIntersection()

```
Math::Point Cone::getIntersection (
            const Ray & ray) const  [override], [virtual]
```

This function does exactly what you think it does.

Parameters

ray	The ray to check for intersection.

Returns

the intersection point between a Ray and a shape

Implements IPrimitive.

6.6.2.5 getMaterial()

```
std::shared_ptr< AMaterial > Cone::getMaterial () const  [override], [virtual]
```

Get the material of the primitive shape.

Returns

The material of the shape.

Implements IPrimitive.

6.6.2.6 getType()

```
const std::string & Cone::getType () const  [override], [virtual]
```

Get the type of the primitive shape.

Returns

The type of the shape as a string.

Implements IPrimitive.

6.6.2.7 intersect()

```
bool Cone::intersect (
            const Ray & ray) const  [override], [virtual]
```

Check if a ray intersects with the primitive shape.

Parameters

ray	The ray to check for intersection.

Returns

True if the ray intersects with the shape, false otherwise.

Implements IPrimitive.

6.6.2.8 intersect_base()

```
bool Cone::intersect_base (
            const Ray & ray) const
```

Check if a ray intersect with the base of the cone (represented by a circle)

Parameters

ray	The ray to check for intersection.

Returns

true if it does, false if not

6.6.2.9 intersectCone()

```
bool Cone::intersectCone (
            const Ray & ray) const
```

Check if a ray intersect with the cone.

Parameters

ray	The ray to check for intersection.

Returns

true if it does, false if not

6.6.2.10 normal_at()

```
Math::Vector Cone::normal_at (
            const Math::Point & point) const  [override], [virtual]
```

Create a normal vector to the shape.

Parameters

point	origin of the normal vector

Returns

normal vector to the shape

Implements IPrimitive.

The documentation for this class was generated from the following files:

- include/plugins/primitives/Cone.hpp
- src/plugins/primitives/Cone.cpp

6.7 Core Class Reference

A class to manage the raytracer project.

```
#include <Core.hpp>
```

Public Member Functions

- Core ()

  Default constructor for Core class. Initializes the core of the raytracer project.
- ~Core ()

  Destructor for Core class. Cleans up the resources used by the core.
- void init (const std::string &filename)

  Initialize the core of the raytracer project.
- void loop ()

  Loop the core of the raytracer project.
- void createGuiThread ()

  Create the GUI thread.
- void createRenderThread ()

  Create the render thread.
- void setCtrl ()
- void setHandleTranslation ()

  set the pointer to function for the translation
- void setHandleRotation ()

  set the pointer to function for the translation
- void notifyRenderThread ()

  notify the render thread that a render update is needed
- void checkSceneFile ()

  check changes in the scene file
- void reloadScene ()

  reload the scene if needed

6.7.1 Detailed Description

A class to manage the raytracer project.

6.7.2 Constructor & Destructor Documentation

6.7.2.1 Core()

```
Core::Core ()
```

Default constructor for Core class. Initializes the core of the raytracer project.

Note

This constructor can be used to create a core with default values.

6.7.2.2 ~Core()

```
Core::~Core ()
```

Destructor for Core class. Cleans up the resources used by the core.

6.7.3 Member Function Documentation

6.7.3.1 checkSceneFile()

```
void Core::checkSceneFile ()
```

check changes in the scene file

Check for changes in the scene file and reload it if modified.

Note

This function is used to check changes in the scene file and reload the scene if needed.

Avoids accessing the scene file too soon after changes are detected.

6.7.3.2 createGuiThread()

```
void Core::createGuiThread ()
```

Create the GUI thread.

Note

This function is used to create the GUI thread.

6.7.3.3 createRenderThread()

```
void Core::createRenderThread ()
```

Create the render thread.

Note

This function is used to create the render thread.

6.7.3.4 init()

```
void Core::init (
            const std::string & filename)
```

Initialize the core of the raytracer project.

Parameters

filename	The name of the file to load the scene from.

Note

This function is used to initialize the core of the raytracer project.

6.7.3.5 loop()

```
void Core::loop ()
```

Loop the core of the raytracer project.

Note

This function is used to loop the core of the raytracer project.

6.7.3.6 notifyRenderThread()

```
void Core::notifyRenderThread ()
```

notify the render thread that a render update is needed

Notify the render thread that a render update is needed.

Note

This function is used to notify the render thread that a render update is needed.

6.7.3.7 reloadScene()

```
void Core::reloadScene ()
```

reload the scene if needed

Reload the scene if needed.

Note

This function is used to reload the scene if needed.

6.7.3.8 setCtrl()

```
void Core::setCtrl ()
```

6.7.3.9 setHandleRotation()

```
void Core::setHandleRotation ()
```

set the pointer to function for the translation

6.7.3.10 setHandleTranslation()

```
void Core::setHandleTranslation ()
```

set the pointer to function for the translation

The documentation for this class was generated from the following files:

- include/core/Core.hpp
- src/core/Core.cpp
- src/core/hotreload.cpp

6.8 Cursor Class Reference

```
#include <Cursor.hpp>
```

Public Member Functions

- Cursor (std::pair< double, double > percentageStartLine, std::pair< double, double > percentageFinishLine, int width, int height, const sf::Font &font, std::string cursorTitleText)
- ~Cursor ()
- void initText (const sf::Font &font, sf::Vector2f _startLine, sf::Vector2f _finishLine, std::string cursorTitleText)

  Initialise all the text of the cursor.
- void initShape (sf::Vector2f _startLine, sf::Vector2f _finishLine)

  Initialise the shape of the cursor.
- sf::Text getText () const

  get the Text of the cursor
- sf::Text getCursorTitleText () const

  get the title text of the cursor
- sf::RectangleShape getRectangle () const

  get the rectangle shape of the cursor
- std::array< sf::Vertex, 2 > getCursorLine () const

  get the cursor line of the cursor
- int getPosCursor () const

  get the position of the cursor
- bool isCursorSelected () const

  check if the cursor is selected
- void setCursorSelected (const bool cursorSelect)

  set if the cursors is selected of not
- void moveCursor (const int offset)

  move the cursor position by an offset
- void setCursorText (const std::string cursorsText)

set the text of the cursors

- void moveRectangleShape (const sf::Vector2f offset)

  move the rectangle shape along the cursor axis
- sf::Vector2i updateCursor (sf::Vector2i mousePos, sf::Vector2i prevMousePos, int &eventKey)

  if the cursor is selected, update all the value to it
- void mouseClickedOnCursor (sf::Vector2i &prevMousPos)

  Check if the cursor as been selected by the user using the mouse button.
- CursorType getType () const

  getter for the type of the cursor

6.8.1 Constructor & Destructor Documentation

6.8.1.1 Cursor()

```
Cursor::Cursor (
            std::pair< double, double > percentageStartLine,
            std::pair< double, double > percentageFinishLine,
            int width,
            int height,
            const sf::Font & font,
            std::string cursorTitleText)
```

6.8.1.2 ~Cursor()

```
Cursor::~Cursor ()
```

6.8.2 Member Function Documentation

6.8.2.1 getCursorLine()

```
std::array< sf::Vertex, 2 > Cursor::getCursorLine () const
```

get the cursor line of the cursor

Returns

the array of vertex reprensenting the cursor line of the cursor

6.8.2.2 getCursorTitleText()

```
sf::Text Cursor::getCursorTitleText () const
```

get the title text of the cursor

Returns

the title text of the cursor

6.8.2.3 getPosCursor()

```
int Cursor::getPosCursor () const
```

get the position of the cursor

Returns

the position of the cursor

6.8.2.4 getRectangle()

```
sf::RectangleShape Cursor::getRectangle () const
```

get the rectangle shape of the cursor

Returns

the rectangle shape of the cursor

6.8.2.5 getText()

```
sf::Text Cursor::getText () const
```

get the Text of the cursor

Returns

the Text of the cursor

6.8.2.6 getType()

```
CursorType Cursor::getType () const
```

getter for the type of the cursor

6.8.2.7 initShape()

```
void Cursor::initShape (
            sf::Vector2f _startLine,
            sf::Vector2f _finishLine)
```

Initialise the shape of the cursor.

Parameters

_startLine	position of the starting line of the cursor
_finishLine	position of the finish line of the cursor

6.8.2.8 initText()

```
void Cursor::initText (
            const sf::Font & font,
            sf::Vector2f _startLine,
            sf::Vector2f _finishLine,
            std::string cursorTitleText)
```

Initialise all the text of the cursor.

Parameters

font	The font for all the text of the cursor
_startLine	position of the starting line of the cursor
_finishLine	position of the finish line of the cursor
cursorTitleText	The title of the cursor

6.8.2.9 isCursorSelected()

```
bool Cursor::isCursorSelected () const
```

check if the cursor is selected

Returns

true or false :)

6.8.2.10 mouseClickedOnCursor()

```
void Cursor::mouseClickedOnCursor (
            sf::Vector2i & prevMousPos)
```

Check if the cursor as been selected by the user using the mouse button.

Parameters

prevMousPos	Previous position of the mouse

6.8.2.11 moveCursor()

```
void Cursor::moveCursor (
            const int offset)
```

move the cursor position by an offset

Parameters

offset	value to move the cursor

6.8.2.12 moveRectangleShape()

```
void Cursor::moveRectangleShape (
            const sf::Vector2f offset)
```

move the rectangle shape along the cursor axis

Parameters

offset	vector to move the rectangle
offset	pair to move the rectangle to

6.8.2.13 setCursorSelected()

```
void Cursor::setCursorSelected (
            const bool cursorSelect)
```

set if the cursors is selected of not

Parameters

cursorSelected	boolean to switch mode

6.8.2.14 setCursorText()

```
void Cursor::setCursorText (
            const std::string cursorsText)
```

set the text of the cursors

Parameters

cursorsText	text to set

Note

it's mainly remind the user by how many pixel he moves the scene

it use the position of the cursor

6.8.2.15 updateCursor()

```
sf::Vector2i Cursor::updateCursor (
            sf::Vector2i mousePos,
            sf::Vector2i prevMousePos,
            int & eventKey)
```

if the cursor is selected, update all the value to it

Parameters

mousePos	current position of the mouse to compare with the previous
prevMoussPos	previous position of the mouse
eventKey	tell the program to move to a direction

Returns

the update position of the mouse

The documentation for this class was generated from the following files:

- include/utilitaries/Cursor.hpp
- src/utilitaries/Cursor.cpp

6.9 Cylindre Class Reference

`#include <Cylindre.hpp>`

Inheritance diagram for Cylindre:

Collaboration diagram for Cylindre:

Public Member Functions

- Cylindre ()

  Default constructor for the Cylindre shape. By default tip is at (0, 0, 0), height at (0, 0, 0) and angle at 0.
- Cylindre (Math::Point center, double radius, double height, Color &color, const std::shared_ptr< AMaterial > &mat)

  Constructor of the Cylindre shape.
- ~Cylindre ()

  basic destructor of the Cylindre class
- const std::string & getType () const override

  Get the type of the primitive shape.
- bool intersect (const Ray &ray) const override

  Check if a ray intersects with the primitive shape.
- Math::Point intersection_cap (const Ray &ray, const Math::Point &cap_center, const Math::Vector &normal) const

  Get the intersection point with the base of the Cylindre (represented by a circle)
- Math::Point getIntersection (const Ray &ray) const override

  This function does exactly what you think it does.
- Math::Vector normal_at (const Math::Point &point) const override

  Create a normal vector to the shape.
- Color getColor () const override

  Get the color of the primitive shape.
- std::shared_ptr< AMaterial > getMaterial () const override

  Get the material of the primitive shape.

Public Member Functions inherited from IPrimitive

- virtual ~IPrimitive ()=default

  Virtual destructor for IPrimitive class.

6.9.1 Constructor & Destructor Documentation

6.9.1.1 Cylindre() [1/2]

```
Cylindre::Cylindre ()
```

Default constructor for the Cylindre shape. By default tip is at (0, 0, 0), height at (0, 0, 0) and angle at 0.

6.9.1.2 Cylindre() [2/2]

```
Cylindre::Cylindre (
            Math::Point center,
            double radius,
            double height,
            Color & color,
            const std::shared_ptr< AMaterial > & mat)
```

Constructor of the Cylindre shape.

Parameters

center	Point that represents the center of the bottom cap of the Cylindre
radius	The radius of both cap of the Cylindre
height	height of the Cylindre

6.9.1.3 ~Cylindre()

```
Cylindre::~Cylindre ()
```

basic destructor of the Cylindre class

6.9.2 Member Function Documentation

6.9.2.1 getColor()

```
Color Cylindre::getColor () const  [override], [virtual]
```

Get the color of the primitive shape.

Returns

The color of the shape.

Implements IPrimitive.

6.9.2.2 getIntersection()

```
Math::Point Cylindre::getIntersection (
            const Ray & ray) const  [override], [virtual]
```

This function does exactly what you think it does.

Parameters

ray	The ray to check for intersection.

Returns

the intersection point between a Ray and a shape

Implements IPrimitive.

6.9.2.3 getMaterial()

```
std::shared_ptr< AMaterial > Cylindre::getMaterial () const  [override], [virtual]
```

Get the material of the primitive shape.

Returns

The material of the shape.

Implements IPrimitive.

6.9.2.4 getType()

```
const std::string & Cylindre::getType () const  [override], [virtual]
```

Get the type of the primitive shape.

Returns

The type of the shape as a string.

Implements IPrimitive.

6.9.2.5 intersect()

```
bool Cylindre::intersect (
            const Ray & ray) const  [override], [virtual]
```

Check if a ray intersects with the primitive shape.

Parameters

ray	The ray to check for intersection.

Returns

True if the ray intersects with the shape, false otherwise.

Implements IPrimitive.

6.9.2.6 intersection_cap()

```
Math::Point Cylindre::intersection_cap (
            const Ray & ray,
            const Math::Point & cap_center,
            const Math::Vector & normal) const
```

Get the intersection point with the base of the Cylindre (represented by a circle)

Parameters

ray	The ray to check for intersection.
cap_center	Center of the bottom cap of the Cylindre
normal	vector normal at the cylindre

Returns

the intersection point between one of the cap of the Cylindre and the ray

6.9.2.7 normal_at()

```
Math::Vector Cylindre::normal_at (
            const Math::Point & point) const  [override], [virtual]
```

Create a normal vector to the shape.

Parameters

point	origin of the normal vector

Returns

normal vector to the shape

Implements IPrimitive.

The documentation for this class was generated from the following files:

- include/plugins/primitives/Cylindre.hpp
- src/plugins/primitives/Cylindre.cpp

6.10 Default Class Reference

```
#include <Default.hpp>
```

Inheritance diagram for Default:

Collaboration diagram for Default:

Public Member Functions

- Default ()

Public Member Functions inherited from AMaterial

- AMaterial (const std::string &name, float ambient, float diffuse, float specular, float shininess)

  Construct a new AMaterial object with given coefficients.
- virtual ~AMaterial ()=default
- const std::string & getName () const
- float getAmbient () const
- float getDiffuse () const
- float getSpecular () const
- float getShininess () const
- float getTransparency () const
- float getRefractionIndex () const
- void setDiffuseColor (const Color &color)

  Set the diffuse color of the material.
- const Color & getDiffuseColor () const

  Get the diffuse color of the material.
- void setSpecularColor (const Color &color)

  Set the specular color of the material.
- const Color & getSpecularColor () const

  Get the specular color of the material.
- void setShininess (float shininess)

  Set the shininess of the material.
- virtual Color getColorAt (const Math::Point &point) const

  Get the color at a specific point on the surface (for procedural textures)
- void setTransparency (float transparency)

  Set the transparency of the material.
- void setRefractionIndex (float refractionIndex)

  Set the refraction index of the material.
- void loadFromMtlLine (const std::string &line)

  Load material properties from a .mtl file line.

6.10.1 Constructor & Destructor Documentation

6.10.1.1 Default()

```
Default::Default ()
```

The documentation for this class was generated from the following files:

- include/materials/Default.hpp
- src/materials/Default.cpp

6.11 DirectionalLight Class Reference

A class to represent a directional light source in the raytracer project.

```
#include <DirectionalLight.hpp>
```

Inheritance diagram for DirectionalLight:

Collaboration diagram for DirectionalLight:

Public Member Functions

- DirectionalLight (const Math::Vector &direction, const Color &color, float intensity)

  Constructor for the DirectionalLight class.
- const std::string & getType () const override

  Get the type of the light source.
- const Math::Point & getPosition () const override

  Get the position of the light source.
- const Math::Vector & getDirection () const override

  Get the direction of the light source.
- float getIntensity () const override

  Get the intensity of the light source.
- const Color & getColor () const override

  Get the color of the light source.
- const Color & getAmbientComponent () const override

  Get the ambient component of the light source.
- const Color & getDiffuseComponent () const override

  Get the diffuse component of the light source.
- const Color & getSpecularComponent () const override

  Get the specular component of the light source.

Public Member Functions inherited from ILight

- virtual ~ILight ()=default

  Virtual destructor for ILight class.

6.11.1 Detailed Description

A class to represent a directional light source in the raytracer project.

6.11.2 Constructor & Destructor Documentation

6.11.2.1 DirectionalLight()

```
DirectionalLight::DirectionalLight (
            const Math::Vector & direction,
            const Color & color,
            float intensity)
```

Constructor for the DirectionalLight class.

Parameters

position	The position of the light source.
intensity	The intensity of the light source.
direction	The direction of the light source.

6.11.3 Member Function Documentation

6.11.3.1 getAmbientComponent()

```
const Color & DirectionalLight::getAmbientComponent () const [override], [virtual]
```

Get the ambient component of the light source.

Returns

The ambient component of the light source.

Implements ILight.

6.11.3.2 getColor()

```
const Color & DirectionalLight::getColor () const [override], [virtual]
```

Get the color of the light source.

Returns

The color of the light source.

Implements ILight.

6.11.3.3 getDiffuseComponent()

```
const Color & DirectionalLight::getDiffuseComponent () const [override], [virtual]
```

Get the diffuse component of the light source.

Returns

The diffuse component of the light source.

Implements ILight.

6.11.3.4 getDirection()

```
const Math::Vector & DirectionalLight::getDirection () const [override], [virtual]
```

Get the direction of the light source.

Returns

The direction of the light source.

Implements ILight.

6.11.3.5 getIntensity()

```
float DirectionalLight::getIntensity () const  [override], [virtual]
```

Get the intensity of the light source.

Returns

The intensity of the light source.

Implements ILight.

6.11.3.6 getPosition()

```
const Math::Point & DirectionalLight::getPosition () const  [override], [virtual]
```

Get the position of the light source.

Returns

The position of the light source.

Implements ILight.

6.11.3.7 getSpecularComponent()

```
const Color & DirectionalLight::getSpecularComponent () const  [override], [virtual]
```

Get the specular component of the light source.

Returns

The specular component of the light source.

Implements ILight.

6.11.3.8 getType()

```
const std::string & DirectionalLight::getType () const  [override], [virtual]
```

Get the type of the light source.

Returns

The type of the light source.

Implements ILight.

The documentation for this class was generated from the following files:

- include/plugins/lights/DirectionalLight.hpp
- src/plugins/lights/DirectionalLight.cpp

6.12 Face Class Reference

A class to represent a Face in 3D space.

```
#include <Face.hpp>
```

Inheritance diagram for Face:

Collaboration diagram for Face:

Public Member Functions

- Face (Math::Point v0, Math::Point v1, Math::Point v2, const Color &color)

  Constructor for the Face class.
- ~Face () override=default

  Destructor for the Face class.
- const std::string & getType () const override

  Get the type of the Face.
- bool intersect (const Ray &ray) const override

  Check if a ray intersects with the Face.
- Math::Point getIntersection (const Ray &ray) const override

  This function does exactly what you think it does.
- Math::Vector normal_at (const Math::Point &point) const override

  Create a normal vector to the shape.
- Color getColor () const override

  Get the color of the Face.
- std::shared_ptr< AMaterial > getMaterial () const override

  Get the material of the primitive shape.

Public Member Functions inherited from IPrimitive

- virtual ~IPrimitive ()=default

  Virtual destructor for IPrimitive class.

6.12.1 Detailed Description

A class to represent a Face in 3D space.

6.12.2 Constructor & Destructor Documentation

6.12.2.1 Face()

```
Face::Face (
            Math::Point v0,
            Math::Point v1,
            Math::Point v2,
            const Color & color)
```

Constructor for the Face class.

Parameters

axis	The axis of the Face (x, y, or z).
v0	The position of the first vertex
v1	The position of the second vertex
v2	The position of the third vertex
color	The color of the Face.

6.12.2.2 ~Face()

```
Face::~Face () [override], [default]
```

Destructor for the Face class.

Note

This destructor is virtual to ensure proper cleanup of derived classes.

6.12.3 Member Function Documentation

6.12.3.1 getColor()

```
Color Face::getColor () const [override], [virtual]
```

Get the color of the Face.

Returns

The color of the Face.

Implements IPrimitive.

6.12.3.2 getIntersection()

```
Math::Point Face::getIntersection (
            const Ray & ray) const [override], [virtual]
```

This function does exactly what you think it does.

Parameters

ray	The ray to check for intersection.

Returns

the intersection point between a Ray and a shape

Implements IPrimitive.

6.12.3.3 getMaterial()

```
std::shared_ptr< AMaterial > Face::getMaterial () const  [override], [virtual]
```

Get the material of the primitive shape.

Returns

The material of the shape.

Implements IPrimitive.

6.12.3.4 getType()

```
const std::string & Face::getType () const  [override], [virtual]
```

Get the type of the Face.

Returns

The type of the Face as a string.

Implements IPrimitive.

6.12.3.5 intersect()

```
bool Face::intersect (
            const Ray & ray) const  [override], [virtual]
```

Check if a ray intersects with the Face.

Parameters

ray	The ray to check for intersection.

Returns

True if the ray intersects with the Face, false otherwise.

Implements IPrimitive.

6.12.3.6 normal_at()

```
Math::Vector Face::normal_at (
            const Math::Point & point) const  [override], [virtual]
```

Create a normal vector to the shape.

Parameters

point	origin of the normal vector

Returns

normal vector to the shape

Implements IPrimitive.

The documentation for this class was generated from the following files:

- include/utilitaries/Face.hpp
- src/utilitaries/Face.cpp

6.13 Gui Class Reference

```
#include <Gui.hpp>
```

Public Member Functions

- Gui ()=default

  Default constructor for Gui class. Initializes the GUI with default settings.
- Gui (int width, int height)

  Constructor for Gui class.
- ~Gui ()

  Destructor for Gui class. Cleans up resources used by the GUI.
- void handleInput (int &eventKey)

  Handles user input in the GUI. Processes events such as keyboard and mouse input.
- void updateScene (std::vector< Color > &pixelsRender, std::vector< Color > &pixelsFastRender)

  Updates the scene in the GUI. Updates the scene based on the rendered pixels.
- void createImage (sf::Image &image, const std::vector< Color > &pixels, int width, int height)

  Creates the image from the pixels.
- void renderScene (int &eventKey)

  Renders the scene in the GUI. Draws the scene to the window.
- bool isOpen () const

  Checks if the window is open.

6.13.1 Constructor & Destructor Documentation

6.13.1.1 Gui() [1/2]

```
Gui::Gui () [default]
```

Default constructor for Gui class. Initializes the GUI with default settings.

Note

This constructor can be used to create a GUI with default values.

6.13.1.2 Gui() [2/2]

```
Gui::Gui (
            int width,
            int height)
```

Constructor for Gui class.

Default constructor for Gui class. Initializes the GUI with default settings.

Parameters

width	The width of the window.
height	The height of the window. Initializes the GUI with default settings.

Note

This constructor can be used to create a GUI with default values.

6.13.1.3 ~Gui()

```
Gui::~Gui ()
```

Destructor for Gui class. Cleans up resources used by the GUI.

Note

This destructor is called when the GUI object is destroyed.

6.13.2 Member Function Documentation

6.13.2.1 createImage()

```
void Gui::createImage (
            sf::Image & image,
            const std::vector< Color > & pixels,
            int width,
            int height)
```

Creates the image from the pixels.

Parameters

image	The image to create.
pixels	The pixels to use for the image.
width	The width of the image.
height	The height of the image.

Note

This function is used to create the image from the pixels.

6.13.2.2 handleInput()

```
void Gui::handleInput (
            int & eventKey)
```

Handles user input in the GUI. Processes events such as keyboard and mouse input.

6.13.2.3 isOpen()

```
bool Gui::isOpen () const
```

Checks if the window is open.

Returns

True if the window is open, false otherwise.

6.13.2.4 renderScene()

```
void Gui::renderScene (
            int & eventKey)
```

Renders the scene in the GUI. Draws the scene to the window.

6.13.2.5 updateScene()

```
void Gui::updateScene (
            std::vector< Color > & pixelsRender,
            std::vector< Color > & pixelsFastRender)
```

Updates the scene in the GUI. Updates the scene based on the rendered pixels.

Updates the scene in the GUI. Updates the scene based on user input and other factors.

The documentation for this class was generated from the following files:

- include/core/Gui.hpp
- src/core/Gui.cpp

6.14 HitInfo Class Reference

A class to represent hit information in the raytracer project.

```
#include <HitInfo.hpp>
```

Public Member Functions

- HitInfo ()

  Default constructor for HitInfo class. Initializes the hit information with default values.
- HitInfo (const Math::Vector &position, const Math::Vector &normal, const Color &color, float distance)

  Constructor for HitInfo class with parameters. Initializes the hit information with specific values.
- const Math::Vector & getPosition () const

  Get the position of the hit.
- const Math::Vector & getNormal () const

  Get the normal vector at the hit point.
- const Color & getColor () const

  Get the color of the hit.
- float getDistance () const

  Get the distance from the ray origin to the hit point.

6.14.1 Detailed Description

A class to represent hit information in the raytracer project.

6.14.2 Constructor & Destructor Documentation

6.14.2.1 HitInfo() [1/2]

```
HitInfo::HitInfo ()
```

Default constructor for HitInfo class. Initializes the hit information with default values.

Note

This constructor can be used to create a HitInfo object with default values.

6.14.2.2 HitInfo() [2/2]

```
HitInfo::HitInfo (
            const Math::Vector & position,
            const Math::Vector & normal,
            const Color & color,
            float distance)
```

Constructor for HitInfo class with parameters. Initializes the hit information with specific values.

Parameters

position	The position of the hit.
normal	The normal vector at the hit point.
color	The color of the hit.
distance	The distance from the ray origin to the hit point.

6.14.3 Member Function Documentation

6.14.3.1 getColor()

```
const Color & HitInfo::getColor () const
```

Get the color of the hit.

Returns

The color of the hit.

Note

This method returns the color of the hit.

6.14.3.2 getDistance()

```
float HitInfo::getDistance () const
```

Get the distance from the ray origin to the hit point.

Returns

The distance from the ray origin to the hit point.

Note

This method returns the distance of the hit.

The distance is a float value representing the distance from the ray origin to the hit point.

This method can be used to determine how far the hit point is from the ray origin.

Example usage: HitInfo hitInfo;

6.14.3.3 getNormal()

```
const Math::Vector & HitInfo::getNormal () const
```

Get the normal vector at the hit point.

Returns

The normal vector at the hit point.

Note

This method returns the normal vector of the hit.

6.14.3.4 getPosition()

```
const Math::Vector & HitInfo::getPosition () const
```

Get the position of the hit.

Returns

The position of the hit.

Note

This method returns the position vector of the hit.

The documentation for this class was generated from the following file:

- include/utilitaries/HitInfo.hpp

6.15 ILight Class Reference

An interface for light sources in the raytracer project.

```
#include <ILight.hpp>
```

Inheritance diagram for ILight:

Public Member Functions

- virtual ~ILight ()=default

  Virtual destructor for ILight class.
- virtual const std::string & getType () const =0

  Get the type of the light source.
- virtual const Math::Point & getPosition () const =0

  Get the position of the light source.
- virtual const Math::Vector & getDirection () const =0

  getDirection
- virtual float getIntensity () const =0

  Get the intensity of the light source.
- virtual const Color & getColor () const =0

  Get the color of the light source.
- virtual const Color & getAmbientComponent () const =0

  Get the ambient component of the light source.
- virtual const Color & getDiffuseComponent () const =0

  Get the diffuse component of the light source.
- virtual const Color & getSpecularComponent () const =0

  Get the specular component of the light source.

6.15.1 Detailed Description

An interface for light sources in the raytracer project.

6.15.2 Constructor & Destructor Documentation

6.15.2.1 ~ILight()

```
virtual ILight::~ILight () [virtual], [default]
```

Virtual destructor for ILight class.

Note

This destructor is virtual to ensure proper cleanup of derived classes.

6.15.3 Member Function Documentation

6.15.3.1 getAmbientComponent()

```
virtual const Color & ILight::getAmbientComponent () const [pure virtual]
```

Get the ambient component of the light source.

Returns

The ambient component of the light source.

Implemented in AmbientLight, DirectionalLight, and PointLight.

6.15.3.2 getColor()

```
virtual const Color & ILight::getColor () const [pure virtual]
```

Get the color of the light source.

Returns

The color of the light source.

Implemented in AmbientLight, DirectionalLight, and PointLight.

6.15.3.3 getDiffuseComponent()

```
virtual const Color & ILight::getDiffuseComponent () const [pure virtual]
```

Get the diffuse component of the light source.

Returns

The diffuse component of the light source.

Implemented in AmbientLight, DirectionalLight, and PointLight.

6.15.3.4 getDirection()

```
virtual const Math::Vector & ILight::getDirection () const  [pure virtual]
```

getDirection

Get the direction of the light source.

Returns

The direction of the light source.

Implemented in AmbientLight, DirectionalLight, and PointLight.

6.15.3.5 getIntensity()

```
virtual float ILight::getIntensity () const  [pure virtual]
```

Get the intensity of the light source.

Returns

The intensity of the light source.

Implemented in AmbientLight, DirectionalLight, and PointLight.

6.15.3.6 getPosition()

```
virtual const Math::Point & ILight::getPosition () const  [pure virtual]
```

Get the position of the light source.

Returns

The position of the light source.

Implemented in AmbientLight, DirectionalLight, and PointLight.

6.15.3.7 getSpecularComponent()

```
virtual const Color & ILight::getSpecularComponent () const  [pure virtual]
```

Get the specular component of the light source.

Returns

The specular component of the light source.

Implemented in AmbientLight, DirectionalLight, and PointLight.

6.15.3.8 getType()

```
virtual const std::string & ILight::getType () const  [pure virtual]
```

Get the type of the light source.

Returns

The type of the light source.

Implemented in AmbientLight, DirectionalLight, and PointLight.

The documentation for this class was generated from the following file:

- include/interfaces/ILight.hpp

6.16 Image Class Reference

A class to represent an image in the raytracer project.

```
#include <Image.hpp>
```

Public Member Functions

- Image (int width, int height, std::vector< Color > &pixels)

  Default constructor for Image class. Initializes the image with default settings.
- ~Image ()

  Destructor for Image class. Cleans up resources used by the image.
- std::filesystem::path createDirAndFileName (const std::string &filename)

  Create the dir and the file name.
- void save (const std::string &filename)

  Save the image to a file.

6.16.1 Detailed Description

A class to represent an image in the raytracer project.

6.16.2 Constructor & Destructor Documentation

6.16.2.1 Image()

```
Image::Image (
            int width,
            int height,
            std::vector< Color > & pixels)
```

Default constructor for Image class. Initializes the image with default settings.

Note

This constructor can be used to create an image with default values.

Parameters

width	The width of the image.
height	The height of the image.
pixels	The vector of pixels to use for the image.

6.16.2.2 ~Image()

```
Image::~Image ()
```

Destructor for Image class. Cleans up resources used by the image.

Note

This destructor is called when the image object is destroyed.

6.16.3 Member Function Documentation

6.16.3.1 createDirAndFileName()

```
std::filesystem::path Image::createDirAndFileName (
            const std::string & filename)
```

Create the dir and the file name.

Create the directory and the file name.

Parameters

filename	The name of the file to save the image to.

Note

This function creates a directory for screenshots and generates a unique file name.

Returns

The path to the created file.

6.16.3.2 save()

```
void Image::save (
            const std::string & filename)
```

Save the image to a file.

Parameters

filename	The name of the file to save the image to.
filename	The name of the file to save the image to.

Note

This function saves the image to a file in PNG format.

The documentation for this class was generated from the following files:

- include/images/Image.hpp
- src/images/Image.cpp

6.17 ImageException Class Reference

Exception class for the raytracer project.

```
#include <Exception.hpp>
```

Inheritance diagram for ImageException:

Collaboration diagram for ImageException:

Public Member Functions

- ImageException (const std::string &message)

  Handle the exception in image.

6.17.1 Detailed Description

Exception class for the raytracer project.

This class is used to handle exceptions in the image system of the raytracer project.

6.17.2 Constructor & Destructor Documentation

6.17.2.1 ImageException()

```
ImageException::ImageException (
            const std::string & message)  [explicit]
```

Handle the exception in image.

Handle the exception in image generation.

Parameters

message	The message to display.

The documentation for this class was generated from the following files:

- include/exceptions/Exception.hpp
- src/exceptions/Exception.cpp

6.18 InfinitCone Class Reference

```
#include <InfinitCone.hpp>
```

Inheritance diagram for InfinitCone:

Collaboration diagram for InfinitCone:

Public Member Functions

- InfinitCone ()

  Default constructor for the InfinitCone shape. By default tip is at (0, 0, 0), height at (0, 0, 0) and angle at 0.
- InfinitCone (Math::Point tip, Math::Vector height, double angle, Color &color, const std::shared_ptr< AMaterial > &mat)

  Constructor of the InfinitCone shape.
- ~InfinitCone ()

  basic destructor of the InfinitCone class
- const std::string & getType () const override

  Get the type of the primitive shape.
- bool intersect (const Ray &ray) const override

  Check if a ray intersects with the primitive shape.
- Math::Point getIntersection (const Ray &ray) const override

  This function does exactly what you think it does.
- Math::Vector normal_at (const Math::Point &point) const override

  Create a normal vector to the shape.
- Color getColor () const override

  Get the color of the primitive shape.
- std::shared_ptr< AMaterial > getMaterial () const override

  Get the material of the primitive shape.

Public Member Functions inherited from IPrimitive

- virtual ~IPrimitive ()=default

  Virtual destructor for IPrimitive class.

6.18.1 Constructor & Destructor Documentation

6.18.1.1 InfinitCone() [1/2]

```
InfinitCone::InfinitCone ()
```

Default constructor for the InfinitCone shape. By default tip is at (0, 0, 0), height at (0, 0, 0) and angle at 0.

6.18.1.2 InfinitCone() [2/2]

```
InfinitCone::InfinitCone (
            Math::Point tip,
            Math::Vector height,
            double angle,
            Color & color,
            const std::shared_ptr< AMaterial > & mat)
```

Constructor of the InfinitCone shape.

Parameters

tip	Point that represent the tip of the InfinitCone
height	height vector, the vector's origin is the tip of the InfinitCone
angle	the angle in degree represented between the height vector and the surface of the shape
color	The color of the InfinitCone.

Note

If you want a straight, x and z align axis InfinitCone a basic height vector would looks like height = (0, -1, 0)

Parameters

tip	Point that represent the tip of the InfinitCone
height	height vector, the vector's origin is the tip of the InfinitCone
angle	the angle in degree represented between the height vector and the surface of the shape

Note

If you want a straight, x and z align axis InfinitCone a basic height vector would looks like height = (0, -1, 0)

6.18.1.3 ~InfinitCone()

```
InfinitCone::~InfinitCone ()
```

basic destructor of the InfinitCone class

6.18.2 Member Function Documentation

6.18.2.1 getColor()

```
Color InfinitCone::getColor () const  [override], [virtual]
```

Get the color of the primitive shape.

Returns

The color of the shape.

Implements IPrimitive.

6.18.2.2 getIntersection()

```
Math::Point InfinitCone::getIntersection (
            const Ray & ray) const  [override], [virtual]
```

This function does exactly what you think it does.

Parameters

ray	The ray to check for intersection.

Returns

the intersection point between a Ray and a shape

Implements IPrimitive.

6.18.2.3 getMaterial()

```
std::shared_ptr< AMaterial > InfinitCone::getMaterial () const  [override], [virtual]
```

Get the material of the primitive shape.

Returns

The material of the shape.

Implements IPrimitive.

6.18.2.4 getType()

```
const std::string & InfinitCone::getType () const  [override], [virtual]
```

Get the type of the primitive shape.

Returns

The type of the shape as a string.

Implements IPrimitive.

6.18.2.5 intersect()

```
bool InfinitCone::intersect (
            const Ray & ray) const  [override], [virtual]
```

Check if a ray intersects with the primitive shape.

Parameters

ray	The ray to check for intersection.

Returns

True if the ray intersects with the shape, false otherwise.

Implements IPrimitive.

6.18.2.6 normal_at()

```
Math::Vector InfinitCone::normal_at (
            const Math::Point & point) const  [override], [virtual]
```

Create a normal vector to the shape.

Parameters

point	origin of the normal vector

Returns

normal vector to the shape

Implements IPrimitive.

The documentation for this class was generated from the following files:

- include/plugins/primitives/InfinitCone.hpp
- src/plugins/primitives/InfinitCone.cpp

6.19 InfiniteCylindre Class Reference

```
#include <InfiniteCylindre.hpp>
```

Inheritance diagram for InfiniteCylindre:

Collaboration diagram for InfiniteCylindre:

Public Member Functions

- InfiniteCylindre ()

  Default constructor for the InfiniteCylindre shape. By default tip is at (0, 0, 0), height at (0, 0, 0) and angle at 0.
- InfiniteCylindre (Math::Point center, double radius, double height, Color &color, const std::shared_ptr< AMaterial > &mat)

  Constructor of the InfiniteCylindre shape.
- ~InfiniteCylindre ()

  basic destructor of the InfiniteCylindre class
- const std::string & getType () const override

  Get the type of the primitive shape.
- bool intersect (const Ray &ray) const override

  Check if a ray intersects with the primitive shape.
- Math::Point getIntersection (const Ray &ray) const override

  This function does exactly what you think it does.
- Math::Vector normal_at (const Math::Point &point) const override

  Create a normal vector to the shape.
- Color getColor () const override

  Get the color of the primitive shape.
- std::shared_ptr< AMaterial > getMaterial () const override

  Get the material of the primitive shape.

Public Member Functions inherited from IPrimitive

- virtual ~IPrimitive ()=default

  Virtual destructor for IPrimitive class.

6.19.1 Constructor & Destructor Documentation

6.19.1.1 InfiniteCylindre() [1/2]

```
InfiniteCylindre::InfiniteCylindre ()
```

Default constructor for the InfiniteCylindre shape. By default tip is at (0, 0, 0), height at (0, 0, 0) and angle at 0.

6.19.1.2 InfiniteCylindre() [2/2]

```
InfiniteCylindre::InfiniteCylindre (
            Math::Point center,
            double radius,
            double height,
            Color & color,
            const std::shared_ptr< AMaterial > & mat)
```

Constructor of the InfiniteCylindre shape.

Parameters

center	Point that represents the center of the bottom cap of the InfiniteCylindre
radius	The radius of both cap of the InfiniteCylindre
height	height of the InfiniteCylindre

6.19.1.3 ~InfiniteCylindre()

```
InfiniteCylindre::~InfiniteCylindre ()
```

basic destructor of the InfiniteCylindre class

6.19.2 Member Function Documentation

6.19.2.1 getColor()

```
Color InfiniteCylindre::getColor () const  [override], [virtual]
```

Get the color of the primitive shape.

Returns

The color of the shape.

Implements IPrimitive.

6.19.2.2 getIntersection()

```
Math::Point InfiniteCylindre::getIntersection (
            const Ray & ray) const  [override], [virtual]
```

This function does exactly what you think it does.

Parameters

ray	The ray to check for intersection.

Returns

the intersection point between a Ray and a shape

Implements IPrimitive.

6.19.2.3 getMaterial()

```
std::shared_ptr< AMaterial > InfiniteCylindre::getMaterial () const  [override], [virtual]
```

Get the material of the primitive shape.

Returns

The material of the shape.

Implements IPrimitive.

6.19.2.4 getType()

```
const std::string & InfiniteCylindre::getType () const  [override], [virtual]
```

Get the type of the primitive shape.

Returns

The type of the shape as a string.

Implements IPrimitive.

6.19.2.5 intersect()

```
bool InfiniteCylindre::intersect (
            const Ray & ray) const  [override], [virtual]
```

Check if a ray intersects with the primitive shape.

Parameters

ray	The ray to check for intersection.

Returns

True if the ray intersects with the shape, false otherwise.

Implements IPrimitive.

6.19.2.6 normal_at()

```
Math::Vector InfiniteCylindre::normal_at (
            const Math::Point & point) const  [override], [virtual]
```

Create a normal vector to the shape.

Parameters

point	origin of the normal vector

Returns

normal vector to the shape

Implements IPrimitive.

The documentation for this class was generated from the following files:

- include/plugins/primitives/InfiniteCylindre.hpp
- src/plugins/primitives/InfiniteCylindre.cpp

6.20 IPrimitive Class Reference

An interface for primitive shapes in the raytracer project.

```
#include <IPrimitive.hpp>
```

Inheritance diagram for IPrimitive:

Public Member Functions

- virtual ~IPrimitive ()=default

  Virtual destructor for IPrimitive class.
- virtual const std::string & getType () const =0

  Get the type of the primitive shape.
- virtual bool intersect (const Ray &ray) const =0

  Check if a ray intersects with the primitive shape.
- virtual Math::Point getIntersection (const Ray &ray) const =0

  This function does exactly what you think it does.
- virtual Math::Vector normal_at (const Math::Point &point) const =0

  Create a normal vector to the shape.
- virtual Color getColor () const =0

  Get the color of the primitive shape.
- virtual std::shared_ptr< AMaterial > getMaterial () const =0

  Get the material of the primitive shape.

6.20.1 Detailed Description

An interface for primitive shapes in the raytracer project.

6.20.2 Constructor & Destructor Documentation

6.20.2.1 ~IPrimitive()

```
virtual IPrimitive::~IPrimitive () [virtual], [default]
```

Virtual destructor for IPrimitive class.

Note

This destructor is virtual to ensure proper cleanup of derived classes.

6.20.3 Member Function Documentation

6.20.3.1 getColor()

```
virtual Color IPrimitive::getColor () const [pure virtual]
```

Get the color of the primitive shape.

Returns

The color of the shape.

Implemented in Cone, Cylindre, Face, InfinitCone, InfiniteCylindre, Plane, Sphere, and Triangle.

6.20.3.2 getIntersection()

```
virtual Math::Point IPrimitive::getIntersection (
            const Ray & ray) const  [pure virtual]
```

This function does exactly what you think it does.

Parameters

ray	The ray to check for intersection.

Returns

the intersection point between a Ray and a shape

Implemented in Cone, Cylindre, Face, InfinitCone, InfiniteCylindre, Plane, Sphere, and Triangle.

6.20.3.3 getMaterial()

```
virtual std::shared_ptr< AMaterial > IPrimitive::getMaterial () const  [pure virtual]
```

Get the material of the primitive shape.

Returns

The material of the shape.

Implemented in Cone, Cylindre, Face, InfinitCone, InfiniteCylindre, Plane, Sphere, and Triangle.

6.20.3.4 getType()

```
virtual const std::string & IPrimitive::getType () const  [pure virtual]
```

Get the type of the primitive shape.

Returns

The type of the shape as a string.

Implemented in Cone, Cylindre, Face, InfinitCone, InfiniteCylindre, Plane, Sphere, and Triangle.

6.20.3.5 intersect()

```
virtual bool IPrimitive::intersect (
            const Ray & ray) const  [pure virtual]
```

Check if a ray intersects with the primitive shape.

Parameters

ray	The ray to check for intersection.

Returns

True if the ray intersects with the shape, false otherwise.

Implemented in Cone, Cylindre, Face, InfinitCone, InfiniteCylindre, Plane, Sphere, and Triangle.

6.20.3.6 normal_at()

```
virtual Math::Vector IPrimitive::normal_at (
            const Math::Point & point) const  [pure virtual]
```

Create a normal vector to the shape.

Parameters

point	origin of the normal vector

Returns

normal vector to the shape

Implemented in Cone, Cylindre, Face, InfinitCone, InfiniteCylindre, Plane, Sphere, and Triangle.

The documentation for this class was generated from the following file:

- include/interfaces/IPrimitive.hpp

6.21 ITransformations Class Reference

An interface for transformations in the raytracer project.

```
#include <ITransformations.hpp>
```

Inheritance diagram for ITransformations:

Public Member Functions

- virtual ~ITransformations ()=default

  Virtual destructor for ITransformations class.
- virtual void apply (Math::Point &origin, Math::Vector translation)=0

  Apply the transformation to the given vector.

6.21.1 Detailed Description

An interface for transformations in the raytracer project.

This interface is used to define the transformations of the objects in the raytracer project.

Note

All transformations must implement this interface.

6.21.2 Constructor & Destructor Documentation

6.21.2.1 ~ITransformations()

```
virtual ITransformations::~ITransformations () [virtual], [default]
```

Virtual destructor for ITransformations class.

Note

This destructor is virtual to ensure proper cleanup of derived classes.

6.21.3 Member Function Documentation

6.21.3.1 apply()

```
virtual void ITransformations::apply (
            Math::Point & origin,
            Math::Vector translation) [pure virtual]
```

Apply the transformation to the given vector.

Parameters

inverted	A boolean indicating whether to apply the transformation in reverse.

Note

This function applies the transformation to the given vector.

Implemented in Rotation, Shear, and Translation.

The documentation for this class was generated from the following file:

- include/interfaces/ITransformations.hpp

6.22 Math::Matrix< X, Y > Class Template Reference

A class to use Matrix for calculus.

```
#include <Matrix.hpp>
```

Public Member Functions

- Matrix ()

  Default constructor of the Matrix class.
- Matrix (double matrix[X][Y])

  Constructor of the Matrix class.
- ~Matrix ()

  Default destructor of the Matrix class.
- double & operator() (unsigned int row, unsigned int col)

  Overload of the () operator: allow us to acces the value inside the matrix.
- const double & operator() (unsigned int row, unsigned int col) const

  Overload of the () operator: allow us to acces the value inside the matrix.
- template<unsigned int x, unsigned int y>
  Matrix< X, y > operator* (Matrix< x, y > const &m2)

  *Overload of the * operator: allow us to multiply two matrices.*
- unsigned int getRow () const

  get the number of row in the current matrix
- unsigned int getCol () const

  get the number of columns in the current matrix

Public Attributes

- unsigned int _row
- unsigned int _col
- double _matrix [X][Y]

6.22.1 Detailed Description

template<unsigned int X, unsigned int Y>
class Math::Matrix< X, Y >

A class to use Matrix for calculus.

Parameters

X	X represents the number of lines in the matrix
Y	Y represents the number of columns in the matrix

6.22.2 Constructor & Destructor Documentation

6.22.2.1 Matrix() [1/2]

```
template<unsigned int X, unsigned int Y>
Math::Matrix< X, Y >::Matrix ()  [inline]
```

Default constructor of the Matrix class.

Note

By default the Matrix is set to 0.0 for each value

6.22.2.2 Matrix() [2/2]

```
template<unsigned int X, unsigned int Y>
Math::Matrix< X, Y >::Matrix (
            double matrix[X][Y])  [inline]
```

Constructor of the Matrix class.

Parameters

matrix	the matrix to copy

6.22.2.3 ~Matrix()

```
template<unsigned int X, unsigned int Y>
Math::Matrix< X, Y >::~Matrix ()  [inline]
```

Default destructor of the Matrix class.

6.22.3 Member Function Documentation

6.22.3.1 getCol()

```
template<unsigned int X, unsigned int Y>
unsigned int Math::Matrix< X, Y >::getCol () const  [inline]
```

get the number of columns in the current matrix

6.22.3.2 getRow()

```
template<unsigned int X, unsigned int Y>
unsigned int Math::Matrix< X, Y >::getRow () const  [inline]
```

get the number of row in the current matrix

6.22.3.3 operator()() [1/2]

```
template<unsigned int X, unsigned int Y>
double & Math::Matrix< X, Y >::operator() (
            unsigned int row,
            unsigned int col)  [inline]
```

Overload of the () operator: allow us to acces the value inside the matrix.

Parameters

row	row represents the row of the matrix you want to access
col	col represents the col of the matrix you want to access

Returns

the value at the row and the col passed as parameter

6.22.3.4 operator()() [2/2]

```
template<unsigned int X, unsigned int Y>
const double & Math::Matrix< X, Y >::operator() (
            unsigned int row,
            unsigned int col) const  [inline]
```

Overload of the () operator: allow us to acces the value inside the matrix.

Parameters

row	row represents the row of the matrix you want to access
col	col represents the col of the matrix you want to access

Returns

a copy of the value at the row and the col passed as parameter

6.22.3.5 operator∗()

```
template<unsigned int X, unsigned int Y>
template<unsigned int x, unsigned int y>
Matrix< X, y > Math::Matrix< X, Y >::operator* (
            Matrix< x, y > const & m2)  [inline]
```

Overload of the ∗ operator: allow us to multiply two matrices.

Parameters

m2	the matrix that the current ones will be multiply by

Returns

an out of range exception if the multiply isn't possible

the result of the multiplication

Note

the x and y param represent the row and the cols of the matrix to be multiply

the X and Y represent the row and the cols of the current matrix

in the case where Y != x then the multiplication cannot be performed

6.22.4 Member Data Documentation

6.22.4.1 _col

```
template<unsigned int X, unsigned int Y>
unsigned int Math::Matrix< X, Y >::_col
```

6.22.4.2 _matrix

```
template<unsigned int X, unsigned int Y>
double Math::Matrix< X, Y >::_matrix[X][Y]
```

6.22.4.3 _row

```
template<unsigned int X, unsigned int Y>
unsigned int Math::Matrix< X, Y >::_row
```

The documentation for this class was generated from the following file:

- include/utilitaries/Matrix.hpp

6.23 Metal Class Reference

```
#include <Metal.hpp>
```

Inheritance diagram for Metal:

Collaboration diagram for Metal:

Public Member Functions

- Metal ()

Public Member Functions inherited from AMaterial

- AMaterial (const std::string &name, float ambient, float diffuse, float specular, float shininess)

  Construct a new AMaterial object with given coefficients.
- virtual ~AMaterial ()=default
- const std::string & getName () const
- float getAmbient () const
- float getDiffuse () const
- float getSpecular () const
- float getShininess () const
- float getTransparency () const
- float getRefractionIndex () const
- void setDiffuseColor (const Color &color)

  Set the diffuse color of the material.
- const Color & getDiffuseColor () const

  Get the diffuse color of the material.
- void setSpecularColor (const Color &color)

  Set the specular color of the material.
- const Color & getSpecularColor () const

  Get the specular color of the material.
- void setShininess (float shininess)

  Set the shininess of the material.
- virtual Color getColorAt (const Math::Point &point) const

  Get the color at a specific point on the surface (for procedural textures)
- void setTransparency (float transparency)

  Set the transparency of the material.
- void setRefractionIndex (float refractionIndex)

  Set the refraction index of the material.
- void loadFromMtlLine (const std::string &line)

  Load material properties from a .mtl file line.

6.23.1 Constructor & Destructor Documentation

6.23.1.1 Metal()

```
Metal::Metal ()
```

The documentation for this class was generated from the following files:

- include/materials/Metal.hpp
- src/materials/Metal.cpp

6.24 Mtl Class Reference

```
#include <Mtl.hpp>
```

Inheritance diagram for Mtl:

Collaboration diagram for Mtl:

Public Member Functions

- Mtl (const std::string &name, float ambient, float diffuse, float specular, float shininess)

  Construct a new Mtl object.
- ~Mtl () override=default

  Destroy the Mtl object.

Public Member Functions inherited from AMaterial

- AMaterial (const std::string &name, float ambient, float diffuse, float specular, float shininess)

  Construct a new AMaterial object with given coefficients.
- virtual ~AMaterial ()=default
- const std::string & getName () const
- float getAmbient () const
- float getDiffuse () const
- float getSpecular () const
- float getShininess () const
- float getTransparency () const
- float getRefractionIndex () const
- void setDiffuseColor (const Color &color)

  Set the diffuse color of the material.
- const Color & getDiffuseColor () const

  Get the diffuse color of the material.
- void setSpecularColor (const Color &color)

  Set the specular color of the material.
- const Color & getSpecularColor () const

  Get the specular color of the material.
- void setShininess (float shininess)

  Set the shininess of the material.
- virtual Color getColorAt (const Math::Point &point) const

  Get the color at a specific point on the surface (for procedural textures)
- void setTransparency (float transparency)

  Set the transparency of the material.
- void setRefractionIndex (float refractionIndex)

  Set the refraction index of the material.
- void loadFromMtlLine (const std::string &line)

  Load material properties from a .mtl file line.

6.24.1 Constructor & Destructor Documentation

6.24.1.1 Mtl()

```
Mtl::Mtl (
            const std::string & name,
            float ambient,
            float diffuse,
            float specular,
            float shininess)  [inline]
```

Construct a new Mtl object.

6.24.1.2 ~Mtl()

```
Mtl::~Mtl () [override], [default]
```

Destroy the Mtl object.

The documentation for this class was generated from the following file:

- include/materials/Mtl.hpp

6.25 ObjParser Class Reference

A class to parse .obj files and create shapes for the raytracer.

```
#include <ObjParser.hpp>
```

Static Public Member Functions

- static std::vector< std::shared_ptr< IPrimitive > > parse (const std::string &filepath)

  Parse a .obj file and create a list of primitives.
- static std::unordered_map< std::string, AMaterial > loadMaterials (const std::string &mtlFilePath)

  Load materials from a .mtl file.
- static std::unordered_map< std::string, Color > parseMaterial (const std::string &line)

  Parse a material line from a .mtl file.

6.25.1 Detailed Description

A class to parse .obj files and create shapes for the raytracer.

6.25.2 Member Function Documentation

6.25.2.1 loadMaterials()

```
static std::unordered_map< std::string, AMaterial > ObjParser::loadMaterials (
            const std::string & mtlFilePath) [static]
```

Load materials from a .mtl file.

Parameters

mtlFilePath	The path to the .mtl file.

Returns

A map of material names to AMaterial objects.

6.25.2.2 parse()

```
std::vector< std::shared_ptr< IPrimitive > > ObjParser::parse (
            const std::string & filepath) [static]
```

Parse a .obj file and create a list of primitives.

Parameters

filepath	The path to the .obj file.

Returns

A vector of shared pointers to IPrimitive objects.

6.25.2.3 parseMaterial()

```
std::unordered_map< std::string, Color > ObjParser::parseMaterial (
            const std::string & line)  [static]
```

Parse a material line from a .mtl file.

Parameters

line	The line to parse.
material	The material to update.

Returns

The updated material.

The documentation for this class was generated from the following files:

- include/loader/ObjParser.hpp
- src/loader/ObjParser.cpp

6.26 PerlinNoiseMaterial Class Reference

```
#include <PerlinNoise.hpp>
```

Inheritance diagram for PerlinNoiseMaterial:

Collaboration diagram for PerlinNoiseMaterial:

Public Member Functions

- PerlinNoiseMaterial ()

  Default constructor for the PerlinNoiseMaterial class.
- PerlinNoiseMaterial (const Color &color1, const Color &color2, double scale=1.0)

  Constructor for the PerlinNoiseMaterial class.
- ~PerlinNoiseMaterial () override=default

  Destructor for the PerlinNoiseMaterial class.
- Color getColorAt (const Math::Point &point) const override

  Calculate the color of the material at a given point using Perlin noise.

Public Member Functions inherited from AMaterial

- AMaterial (const std::string &name, float ambient, float diffuse, float specular, float shininess)

  Construct a new AMaterial object with given coefficients.
- virtual ~AMaterial ()=default
- const std::string & getName () const
- float getAmbient () const
- float getDiffuse () const
- float getSpecular () const
- float getShininess () const
- float getTransparency () const
- float getRefractionIndex () const
- void setDiffuseColor (const Color &color)

  Set the diffuse color of the material.
- const Color & getDiffuseColor () const

  Get the diffuse color of the material.
- void setSpecularColor (const Color &color)

  Set the specular color of the material.
- const Color & getSpecularColor () const

  Get the specular color of the material.
- void setShininess (float shininess)

  Set the shininess of the material.
- void setTransparency (float transparency)

  Set the transparency of the material.
- void setRefractionIndex (float refractionIndex)

  Set the refraction index of the material.
- void loadFromMtlLine (const std::string &line)

  Load material properties from a .mtl file line.

6.26.1 Constructor & Destructor Documentation

6.26.1.1 PerlinNoiseMaterial() [1/2]

```
PerlinNoiseMaterial::PerlinNoiseMaterial ()
```

Default constructor for the PerlinNoiseMaterial class.

6.26.1.2 PerlinNoiseMaterial() [2/2]

```
PerlinNoiseMaterial::PerlinNoiseMaterial (
            const Color & color1,
            const Color & color2,
            double scale = 1.0)
```

Constructor for the PerlinNoiseMaterial class.

Parameters

color1	First color of the Perlin noise
color2	Second color of the Perlin noise
scale	Scale of the Perlin noise

6.26.1.3 ~PerlinNoiseMaterial()

```
PerlinNoiseMaterial::~PerlinNoiseMaterial () [override], [default]
```

Destructor for the PerlinNoiseMaterial class.

6.26.2 Member Function Documentation

6.26.2.1 getColorAt()

```
Color PerlinNoiseMaterial::getColorAt (
            const Math::Point & point) const [override], [virtual]
```

Calculate the color of the material at a given point using Perlin noise.

Parameters

point	Intersection point where to calculate the color

Returns

Color at this point

Reimplemented from AMaterial.

The documentation for this class was generated from the following files:

- include/materials/PerlinNoise.hpp
- src/materials/PerlinNoise.cpp

6.27 Plane Class Reference

A class to represent a plane in 3D space.

```
#include <Plane.hpp>
```

Inheritance diagram for Plane:

Collaboration diagram for Plane:

Public Member Functions

- Plane (Math::Point p0, Math::Point p1, Math::Point p2, const Color &color, const std::shared_ptr< AMaterial > &mat)

  Constructor for the Plane class.
- ~Plane () override=default

  Destructor for the Plane class.
- const std::string & getType () const override

  Get the type of the plane.
- bool intersect (const Ray &ray) const override

  Check if a ray intersects with the plane.
- Math::Point getIntersection (const Ray &ray) const override

  This function does exactly what you think it does.
- Math::Vector normal_at (const Math::Point &point) const override

  Create a normal vector to the shape.
- Color getColor () const override

  Get the color of the plane.
- std::shared_ptr< AMaterial > getMaterial () const override

  Get the material of the primitive shape.

Public Member Functions inherited from IPrimitive

- virtual ~IPrimitive ()=default

  Virtual destructor for IPrimitive class.

6.27.1 Detailed Description

A class to represent a plane in 3D space.

6.27.2 Constructor & Destructor Documentation

6.27.2.1 Plane()

```
Plane::Plane (
            Math::Point p0,
            Math::Point p1,
            Math::Point p2,
            const Color & color,
            const std::shared_ptr< AMaterial > & mat)
```

Constructor for the Plane class.

Parameters

axis	The axis of the plane (x, y, or z).
position	The position of the plane along the specified axis.
color	The color of the plane.

6.27.2.2 ~Plane()

```
Plane::~Plane () [override], [default]
```

Destructor for the Plane class.

Note

This destructor is virtual to ensure proper cleanup of derived classes.

6.27.3 Member Function Documentation

6.27.3.1 getColor()

```
Color Plane::getColor () const [override], [virtual]
```

Get the color of the plane.

Returns

The color of the plane.

Implements IPrimitive.

6.27.3.2 getIntersection()

```
Math::Point Plane::getIntersection (
            const Ray & ray) const [override], [virtual]
```

This function does exactly what you think it does.

Parameters

ray	The ray to check for intersection.

Returns

the intersection point between a Ray and a shape

Implements IPrimitive.

6.27.3.3 getMaterial()

```
std::shared_ptr< AMaterial > Plane::getMaterial () const [override], [virtual]
```

Get the material of the primitive shape.

Returns

The material of the shape.

Implements IPrimitive.

6.27.3.4 getType()

```
const std::string & Plane::getType () const  [override], [virtual]
```

Get the type of the plane.

Returns

The type of the plane as a string.

Implements IPrimitive.

6.27.3.5 intersect()

```
bool Plane::intersect (
            const Ray & ray) const  [override], [virtual]
```

Check if a ray intersects with the plane.

Parameters

ray	The ray to check for intersection.

Returns

True if the ray intersects with the plane, false otherwise.

Implements IPrimitive.

6.27.3.6 normal_at()

```
Math::Vector Plane::normal_at (
            const Math::Point & point) const  [override], [virtual]
```

Create a normal vector to the shape.

Parameters

point	origin of the normal vector

Returns

normal vector to the shape

Implements IPrimitive.

The documentation for this class was generated from the following files:

- include/plugins/primitives/Plane.hpp
- src/plugins/primitives/Plane.cpp

6.28 PlugInException Class Reference

Exception class for the raytracer project.

```
#include <Exception.hpp>
```

6.28.1 Detailed Description

Exception class for the raytracer project.

This class is used to handle exceptions in the plugin system of the raytracer project.

The documentation for this class was generated from the following file:

- include/exceptions/Exception.hpp

6.29 PluginsException Class Reference

```
#include <Exception.hpp>
```

Inheritance diagram for PluginsException:

Collaboration diagram for PluginsException:

Public Member Functions

- PluginsException (const std::string &message)
  Handle the exception in plugin.

6.29.1 Constructor & Destructor Documentation

6.29.1.1 PluginsException()

```
PluginsException::PluginsException (
            const std::string & message)  [explicit]
```

Handle the exception in plugin.

Handle the exception in plugin loader.

Parameters

message	The message to display.

The documentation for this class was generated from the following files:

- include/exceptions/Exception.hpp
- src/exceptions/Exception.cpp

6.30 PluginsHandle Class Reference

A class to manage plugins in the raytracer project.

```
#include <PluginsHandle.hpp>
```

Public Member Functions

- PluginsHandle (const std::string &path)

  Constructor for PluginsHandle.
- ~PluginsHandle ()

  Destructor for PluginsHandle.
- PluginsHandle (const PluginsHandle &)=delete

  Deleted copy constructor for the PluginsHandle class.
- PluginsHandle & operator= (const PluginsHandle &)=delete

  Deleted copy assignment operator for the PluginsHandle class.
- PluginsHandle (PluginsHandle &&) noexcept

  Move constructor for PluginsHandle.
- PluginsHandle & operator= (PluginsHandle &&) noexcept

  Move assignment operator for PluginsHandle.
- template<typename T>
  T getSymbol (const std::string &symbolName)

  Get a symbol from the loaded plugin.

6.30.1 Detailed Description

A class to manage plugins in the raytracer project.

6.30.2 Constructor & Destructor Documentation

6.30.2.1 PluginsHandle() [1/3]

```
PluginsHandle::PluginsHandle (
            const std::string & path)  [explicit]
```

Constructor for PluginsHandle.

Parameters

path	The path to the plugin file (.so).

6.30.2.2 ~PluginsHandle()

```
PluginsHandle::~PluginsHandle ()
```

Destructor for PluginsHandle.

Note

Ensures the plugin handle is properly closed.

6.30.2.3 PluginsHandle() [2/3]

```
PluginsHandle::PluginsHandle (
            const PluginsHandle & )  [delete]
```

Deleted copy constructor for the PluginsHandle class.

Parameters

other	The PluginsHandle object to copy from.

6.30.2.4 PluginsHandle() [3/3]

```
PluginsHandle::PluginsHandle (
            PluginsHandle && other)  [noexcept]
```

Move constructor for PluginsHandle.

Parameters

other	The PluginsHandle object to move from.

6.30.3 Member Function Documentation

6.30.3.1 getSymbol()

```
template<typename T>
T PluginsHandle::getSymbol (
            const std::string & symbolName)  [inline]
```

Get a symbol from the loaded plugin.

Template Parameters

T	The type of the symbol.

Parameters

symbolName	The name of the symbol to retrieve.

Returns

A pointer to the symbol.

Exceptions

std::runtime_error	if the symbol cannot be found.

6.30.3.2 operator=() [1/2]

```
PluginsHandle & PluginsHandle::operator= (
            const PluginsHandle & )  [delete]
```

Deleted copy assignment operator for the PluginsHandle class.

Parameters

other	The PluginsHandle object to copy from.

6.30.3.3 operator=() [2/2]

```
PluginsHandle & PluginsHandle::operator= (
            PluginsHandle && other)  [noexcept]
```

Move assignment operator for PluginsHandle.

Parameters

other	The PluginsHandle object to move from.

Returns

A reference to this object.

The documentation for this class was generated from the following files:

- include/plugins/PluginsHandle.hpp
- src/core/PluginsHandle.cpp

6.31 PluginsManager Class Reference

A class to manage plugins in the raytracer project.

```
#include <PluginsManager.hpp>
```

Public Member Functions

- void loadPlugins (const std::string &path)

  Load the plugins from the specified path.
- template<typename T>
  T getSymbol (const std::string &symbol)

  Get a symbol from the plugin handle.

Static Public Member Functions

- static void registerPrimitiveFactory (const std::string &type, PrimitivesFactory factory)

  Register a factory for creating IPrimitive objects.
- static void registerLightFactory (const std::string &type, LightsFactory factory)

  Register a factory for creating ILight objects.
- static std::shared_ptr< IPrimitive > createPrimitive (const std::string &name, const libconfig::Setting &setting)

  Create a primitive object based on the specified name and settings.
- static std::shared_ptr< ILight > createLight (const std::string &name, const libconfig::Setting &setting)

  Create a light object based on the specified name and settings.
- static void unregisterAllFactories ()

  Unergister all factory for creating IPrimitive objects and Ilight objects.

6.31.1 Detailed Description

A class to manage plugins in the raytracer project.

6.31.2 Member Function Documentation

6.31.2.1 createLight()

```
std::shared_ptr< ILight > PluginsManager::createLight (
            const std::string & name,
            const libconfig::Setting & setting)  [static]
```

Create a light object based on the specified name and settings.

Parameters

name	The name of the light type.
setting	The settings for the light.

Returns

A shared pointer to the created ILight object.

6.31.2.2 createPrimitive()

```
std::shared_ptr< IPrimitive > PluginsManager::createPrimitive (
            const std::string & name,
            const libconfig::Setting & setting)  [static]
```

Create a primitive object based on the specified name and settings.

Parameters

name	The name of the primitive type.
setting	The settings for the primitive.

Returns

A shared pointer to the created IPrimitive object.

Parameters

name	The name of the primitive type.
setting	The settings for the primitive

Returns

A shared pointer to the created IPrimitive object.

6.31.2.3 getSymbol()

```
template<typename T>
T PluginsManager::getSymbol (
            const std::string & symbol)  [inline]
```

Get a symbol from the plugin handle.

Parameters

symbol	The name of the symbol to retrieve.

Returns

A pointer to the symbol.

6.31.2.4 loadPlugins()

```
void PluginsManager::loadPlugins (
            const std::string & path)
```

Load the plugins from the specified path.

Parameters

path	The path to the directory containing the plugins.

6.31.2.5 registerLightFactory()

```
void PluginsManager::registerLightFactory (
            const std::string & type,
            LightsFactory factory)  [static]
```

Register a factory for creating ILight objects.

Parameters

type	The type of the light (e.g., "point", "directional").
factory	The factory function to create ILight objects.
type	The type of the light (e.g., "point", "directional").
factory	The factory function to create ILight objects.

Note

This method is used to register a factory function for creating ILight objects.

6.31.2.6 registerPrimitiveFactory()

```
void PluginsManager::registerPrimitiveFactory (
            const std::string & type,
            PrimitivesFactory factory)  [static]
```

Register a factory for creating IPrimitive objects.

Parameters

type	The type of the primitive (e.g., "sphere", "cube").
factory	The factory function to create IPrimitive objects.
type	The type of the primitive (e.g., "sphere", "cube").
factory	The factory function to create IPrimitive objects.

Note

This method is used to register a factory function for creating

6.31.2.7 unregisterAllFactories()

```
void PluginsManager::unregisterAllFactories () [static]
```

Unergister all factory for creating IPrimitive objects and Ilight objects.

Unregister all factory for creating IPrimitive objects and ILight objects.

Note

This method is used to unregister all factory functions for creating IPrimitive and ILight objects.

The documentation for this class was generated from the following files:

- include/plugins/PluginsManager.hpp
- src/core/PluginsManager.cpp

6.32 Math::Point Class Reference

A class to represent a 3D point with basic operations.

```
#include <Point.hpp>
```

Public Member Functions

- Point ()

  Default constructor for Point class. Initializes the point to (0, 0, 0).
- Point (double x, double y, double z)

  Default constructor for Point class. Initializes the point to (0, 0, 0).
- Point operator+ (const Point &point) const

  Overload of the operator + for two Points.
- Point operator+ (const Vector &vector) const

  Overload of the operator + for a Point and a Vector.
- Point operator- (const Vector &vector) const

  Overload of the operator + for a Point and a Vector.
- Point & operator+= (const Vector &vector)

Overload of the operator += for a Point and a Vector.

- bool operator== (const Point &point) const

  Overload of the operator == for a Point and another Point.
- Vector operator- (const Point &other) const

  Generate a vector between two points.
- double getX () const

  This does exactly what you think it does.
- double getY () const

  This does exactly what you think it does.
- double getZ () const

  This does exactly what you think it does.
- void setX (const double x)
- void setY (const double y)
- void setZ (const double z)

  This does exactly what you think it does.
- Math::Matrix< 4, 1 > pointToMatrix ()
- Math::Matrix< 1, 2 > shearToMatrix ()
- double dot (const Vector &other) const

  Dot product of two vectors.

6.32.1 Detailed Description

A class to represent a 3D point with basic operations.

6.32.2 Constructor & Destructor Documentation

6.32.2.1 Point() [1/2]

```
Math::Point::Point ()
```

Default constructor for Point class. Initializes the point to (0, 0, 0).

Note

This constructor can be used to create a "null" point.

6.32.2.2 Point() [2/2]

```
Math::Point::Point (
            double x,
            double y,
            double z)
```

Default constructor for Point class. Initializes the point to (0, 0, 0).

Parameters

x	The x-coordinate of the point.
y	The y-coordinate of the point.
z	The z-coordinate of the point.

Note

This constructor can be used to create a point with specific coordinates.

6.32.3 Member Function Documentation

6.32.3.1 dot()

```
double Math::Point::dot (
            const Vector & other) const
```

Dot product of two vectors.

Parameters

other	The vector to dot with.

Returns

The dot product of this vector and the other vector.

6.32.3.2 getX()

```
double Math::Point::getX () const
```

This does exactly what you think it does.

Returns

the X attribute of the current point

6.32.3.3 getY()

```
double Math::Point::getY () const
```

This does exactly what you think it does.

Returns

the Y attribute of the current point

6.32.3.4 getZ()

```
double Math::Point::getZ () const
```

This does exactly what you think it does.

Returns

the Z attribute of the current point

6.32.3.5 operator+() [1/2]

```
Point Math::Point::operator+ (
            const Point & point) const
```

Overload of the operator + for two Points.

Parameters

point	point to add to the other

Returns

a newly created point represented by the add of two points

6.32.3.6 operator+() [2/2]

```
Point Math::Point::operator+ (
            const Vector & vector) const
```

Overload of the operator + for a Point and a Vector.

Parameters

Vector	vector to add to the other point

Returns

a newly created point represented by the add of a point and a vector

6.32.3.7 operator+=()

```
Point & Math::Point::operator+= (
            const Vector & vector)
```

Overload of the operator += for a Point and a Vector.

Parameters

Vector	vector to add to the other point

Returns

add to the current point coordinate of the vector

6.32.3.8 operator-() [1/2]

```
Vector Math::Point::operator- (
            const Point & other) const
```

Generate a vector between two points.

Parameters

other	other end of the vector represented by a point

Returns

a newly created vector

6.32.3.9 operator-() [2/2]

```
Point Math::Point::operator- (
            const Vector & vector) const
```

Overload of the operator + for a Point and a Vector.

Parameters

Vector	vector to add to the other point

Returns

a newly created point represented by the add of a point and a vector

6.32.3.10 operator==()

```
bool Math::Point::operator== (
            const Point & point) const
```

Overload of the operator == for a Point and another Point.

Parameters

point	point to check the equality

Returns

true if both are the same, false if not

6.32.3.11 pointToMatrix()

```
Math::Matrix< 4, 1 > Math::Point::pointToMatrix () [inline]
```

6.32.3.12 setX()

```
void Math::Point::setX (
            const double x)
```

Returns

set X attribute of the current point

6.32.3.13 setY()

```
void Math::Point::setY (
            const double y)
```

Returns

set Y attribute of the current point

6.32.3.14 setZ()

```
void Math::Point::setZ (
            const double z)
```

This does exactly what you think it does.

6.32.3.15 shearToMatrix()

```
Math::Matrix< 1, 2 > Math::Point::shearToMatrix () [inline]
```

The documentation for this class was generated from the following files:

- include/utilitaries/Point.hpp
- src/utilitaries/Point.cpp

6.33 PointLight Class Reference

A class to represent a point light source in the raytracer project.

```
#include <PointLight.hpp>
```

Inheritance diagram for PointLight:

Collaboration diagram for PointLight:

Public Member Functions

- PointLight (const Math::Vector &position, const Color &color, float intensity)

  Constructor for the PointLight class.
- const std::string & getType () const override

  Get the type of the light source.
- const Math::Point & getPosition () const override

  Get the position of the light source.
- const Math::Vector & getDirection () const override

  Get the direction of the light source.
- float getIntensity () const override

  Get the intensity of the light source.
- const Color & getColor () const override

  Get the color of the light source.
- const Color & getAmbientComponent () const override

  Get the ambient component of the light source.
- const Color & getDiffuseComponent () const override

  Get the diffuse component of the light source.
- const Color & getSpecularComponent () const override

  Get the specular component of the light source.

Public Member Functions inherited from ILight

- virtual ~ILight ()=default

  Virtual destructor for ILight class.

6.33.1 Detailed Description

A class to represent a point light source in the raytracer project.

6.33.2 Constructor & Destructor Documentation

6.33.2.1 PointLight()

```
PointLight::PointLight (
            const Math::Vector & position,
            const Color & color,
            float intensity)
```

Constructor for the PointLight class.

Parameters

position	The position of the light source.
intensity	The intensity of the light source.
color	The color of the light source.

6.33.3 Member Function Documentation

6.33.3.1 getAmbientComponent()

```
const Color & PointLight::getAmbientComponent () const [override], [virtual]
```

Get the ambient component of the light source.

Returns

The ambient component of the light source.

Implements ILight.

6.33.3.2 getColor()

```
const Color & PointLight::getColor () const [override], [virtual]
```

Get the color of the light source.

Returns

The color of the light source.

Implements ILight.

6.33.3.3 getDiffuseComponent()

```
const Color & PointLight::getDiffuseComponent () const [override], [virtual]
```

Get the diffuse component of the light source.

Returns

The diffuse component of the light source.

Implements ILight.

6.33.3.4 getDirection()

```
const Math::Vector & PointLight::getDirection () const [override], [virtual]
```

Get the direction of the light source.

Returns

The color of the light source.

Implements ILight.

6.33.3.5 getIntensity()

```
float PointLight::getIntensity () const  [override], [virtual]
```

Get the intensity of the light source.

Returns

The intensity of the light source.

Implements ILight.

6.33.3.6 getPosition()

```
const Math::Point & PointLight::getPosition () const  [override], [virtual]
```

Get the position of the light source.

Returns

The position of the light source.

Implements ILight.

6.33.3.7 getSpecularComponent()

```
const Color & PointLight::getSpecularComponent () const  [override], [virtual]
```

Get the specular component of the light source.

Returns

The specular component of the light source.

Implements ILight.

6.33.3.8 getType()

```
const std::string & PointLight::getType () const  [override], [virtual]
```

Get the type of the light source.

Returns

The type of the light source.

Implements ILight.

The documentation for this class was generated from the following files:

- include/plugins/lights/PointLight.hpp
- src/plugins/lights/PointLight.cpp

6.34 Ray Class Reference

A class to represent a ray in 3D space.

```
#include <Ray.hpp>
```

Public Member Functions

- Ray (const Math::Point &origin=Math::Point(), const Math::Vector &direction=Math::Vector())

  Default constructor for Ray class. Initializes the ray with a position and direction.
- const Math::Point & getOrigin () const

  Get the origin of the ray.
- const Math::Vector & getDirection () const

  Get the direction of the ray.
- void setOrigin (const Math::Point &origin)

  Set the origin of the ray.
- void setDirection (const Math::Vector &direction)

  Set the direction of the ray.

6.34.1 Detailed Description

A class to represent a ray in 3D space.

6.34.2 Constructor & Destructor Documentation

6.34.2.1 Ray()

```
Ray::Ray (
            const Math::Point & origin = Math::Point(),
            const Math::Vector & direction = Math::Vector())
```

Default constructor for Ray class. Initializes the ray with a position and direction.

Parameters

origin	The origin of the ray.
direction	The direction of the ray.

Note

If no parameters are provided, the ray is initialized to (0, 0, 0).

This constructor can be used to create a ray with specific origin and direction.

6.34.3 Member Function Documentation

6.34.3.1 getDirection()

```
const Math::Vector & Ray::getDirection () const
```

Get the direction of the ray.

Returns

The direction of the ray.

6.34.3.2 getOrigin()

```
const Math::Point & Ray::getOrigin () const
```

Get the origin of the ray.

Returns

The origin of the ray.

6.34.3.3 setDirection()

```
void Ray::setDirection (
            const Math::Vector & direction)
```

Set the direction of the ray.

Parameters

direction	direction of the ray.

6.34.3.4 setOrigin()

```
void Ray::setOrigin (
            const Math::Point & origin)
```

Set the origin of the ray.

Parameters

origin	origin to be set

The documentation for this class was generated from the following files:

- include/utilitaries/Ray.hpp
- src/utilitaries/Ray.cpp

6.35 RaytracerException Class Reference

Exception class for the raytracer project.

```
#include <Exception.hpp>
```

Inheritance diagram for RaytracerException:

Collaboration diagram for RaytracerException:

Public Member Functions

- RaytracerException (const std::string &message)
  Handle the exception in raytracer project.

6.35.1 Detailed Description

Exception class for the raytracer project.

This class is used to handle exceptions in the raytracer project.

6.35.2 Constructor & Destructor Documentation

6.35.2.1 RaytracerException()

```
RaytracerException::RaytracerException (
            const std::string & message)  [explicit]
```

Handle the exception in raytracer project.

Handle the exception.

Parameters

message	The message to display.

The documentation for this class was generated from the following files:

- include/exceptions/Exception.hpp
- src/exceptions/Exception.cpp

6.36 Render Class Reference

A class to render the scene in the raytracer project.

```
#include <Render.hpp>
```

Public Member Functions

- Render (int width, int height, Scene &scene)

  Default constructor for Render class. Initializes the render with default settings.
- ~Render ()

  Destructor for Render class. Cleans up resources used by the render.
- void computeRender (std::vector< Color > &pixelsRender)

  Compute the render of the scene.
- void computeFastRender (std::vector< Color > &pixelsFastRender)

  Compute the fast render of the scene.
- void setProgressCallback (const std::function< void()> &callback)

  Set the progress callback function.
- Color renderPixel (int x, int y, int maxWidth, int maxHeight)

  Compute the color of the pixel based on the ray tracing.
- Color ComputeSpecularLight (Math::Vector &lightSource, Math::Vector normal, const Camera &cam, const std::shared_ptr< ILight > &light, float specularIntensity)
- Color ComputeDiffuseLights (const std::shared_ptr< ILight > &light, float diffuseStrength)
- Color ComputePixelColor (std::shared_ptr< IPrimitive > closest_object, const Camera &cam, const Ray &ray, int ao)
- Math::Vector randomHemisphereDirection (const Math::Vector &normal)
- float AmbiantOcclusion (std::shared_ptr< IPrimitive > closest_object, Ray &ray)
- bool isInShadowOfShape (Math::Point &hit_point, std::shared_ptr< IPrimitive > closest_object, Math::Vector &lightSource)

  Check if a shape is in the shadow of another shape.
- Math::Vector reflect (Math::Vector incident, Math::Vector normal)

  Calculate the reflection of a ray based on the normal vector.
- Color renderReflection (std::shared_ptr< IPrimitive > closest_object, const Ray &ray, Camera &cam, int depthReflection)

  Function to handle reflection rendering.
- Color renderRefraction (std::shared_ptr< IPrimitive > closest_object, const Ray &ray, Camera &cam, int depthReflection)
- void sendRay (Ray &ray, Color &pixel, Camera &cam, int depthReflexion, std::shared_ptr< IPrimitive > closest_object)

  Function to send a ray for rendering.
- Color adaptiveSample (double x, double y, double x1, double y1, int maxWidth, int maxHeight, int depth)

  Function to compute the adaptive sampling of a pixel.

6.36.1 Detailed Description

A class to render the scene in the raytracer project.

6.36.2 Constructor & Destructor Documentation

6.36.2.1 Render()

```
Render::Render (
            int width,
            int height,
            Scene & scene)
```

Default constructor for Render class. Initializes the render with default settings.

Parameters

width	The width of the render.
height	The height of the render.

Note

This constructor can be used to create a render with default values.

6.36.2.2 ~Render()

```
Render::~Render ()
```

Destructor for Render class. Cleans up resources used by the render.

Note

This destructor is called when the render object is destroyed.

6.36.3 Member Function Documentation

6.36.3.1 adaptiveSample()

```
Color Render::adaptiveSample (
            double x,
            double y,
            double x1,
            double y1,
            int maxWidth,
            int maxHeight,
            int depth)
```

Function to compute the adaptive sampling of a pixel.

Parameters

x	The x coordinate of the pixel.
y	The y coordinate of the pixel.
x1	The x coordinate of the pixel in the image.
y1	The y coordinate of the pixel in the image.
maxWidth	The maximum width of the image.
maxHeight	The maximum height of the image.
depth	The depth of the pixel.

Note

This function computes the adaptive sampling of a pixel

Returns

The color of the pixel after adaptive sampling.

6.36.3.2 AmbiantOcclusion()

```
float Render::AmbiantOcclusion (
            std::shared_ptr< IPrimitive > closest_object,
            Ray & ray)
```

6.36.3.3 ComputeDiffuseLights()

```
Color Render::ComputeDiffuseLights (
            const std::shared_ptr< ILight > & light,
            float diffuseStrength)
```

6.36.3.4 computeFastRender()

```
void Render::computeFastRender (
            std::vector< Color > & pixelsFastRender)
```

Compute the fast render of the scene.

Parameters

pixelsFastRender	The vector to store the fast rendered pixels.

Note

This function computes the fast render of the scene and stores the result in the provided vector.

6.36.3.5 ComputePixelColor()

```
Color Render::ComputePixelColor (
            std::shared_ptr< IPrimitive > closest_object,
            const Camera & cam,
            const Ray & ray,
            int ao)
```

6.36.3.6 computeRender()

```
void Render::computeRender (
            std::vector< Color > & pixelsRender)
```

Compute the render of the scene.

Parameters

pixelsRender	The vector to store the rendered pixels.

Note

This function computes the render of the scene and stores the result in the provided vector.

6.36.3.7 ComputeSpecularLight()

```
Color Render::ComputeSpecularLight (
            Math::Vector & lightSource,
            Math::Vector normal,
            const Camera & cam,
            const std::shared_ptr< ILight > & light,
            float specularIntensity)
```

6.36.3.8 isInShadowOfShape()

```
bool Render::isInShadowOfShape (
            Math::Point & hit_point,
            std::shared_ptr< IPrimitive > closest_object,
            Math::Vector & lightSource)
```

Check if a shape is in the shadow of another shape.

Parameters

hit_point	The point hit by the current ray in the closest shape of the current one
closest_object	a pointer to the closest shape of the current one

Returns

true if it is, false if not

6.36.3.9 randomHemisphereDirection()

```
Math::Vector Render::randomHemisphereDirection (
            const Math::Vector & normal)
```

6.36.3.10 reflect()

```
Math::Vector Render::reflect (
            Math::Vector incident,
            Math::Vector normal)
```

Calculate the reflection of a ray based on the normal vector.

Parameters

incident	The incident vector (the direction of the incoming ray).
normal	The normal vector at the point of intersection.

Returns

The reflected vector.

6.36.3.11 renderPixel()

```
Color Render::renderPixel (
            int x,
            int y,
            int maxWidth,
            int maxHeight)
```

Compute the color of the pixel based on the ray tracing.

Parameters

x	The x coordinate of the pixel.
y	The y coordinate of the pixel.
maxWidth	The maximum width of the render.
maxHeight	The maximum height of the render.

Note

the width parameters and the height one are not the same depending on the mode
the fast render will have a smallest size of screen compare to the normal render

6.36.3.12 renderReflection()

```
Color Render::renderReflection (
            std::shared_ptr< IPrimitive > closest_object,
            const Ray & ray,
            Camera & cam,
            int depthReflection)
```

Function to handle reflection rendering.

Parameters

closest_object	The closest object to the ray.
ray	The ray to be rendered.
cam	The camera used for rendering.
depthReflection	The depth of reflection.

Returns

The color of the pixel after reflection rendering.

Note

This function computes the reflection of the ray and returns the color of the pixel.

6.36.3.13 renderRefraction()

```
Color Render::renderRefraction (
            std::shared_ptr< IPrimitive > closest_object,
            const Ray & ray,
            Camera & cam,
            int depthReflection)
```

6.36.3.14 sendRay()

```
void Render::sendRay (
            Ray & ray,
            Color & pixel,
            Camera & cam,
            int depthReflexion,
            std::shared_ptr< IPrimitive > closest_object)
```

Function to send a ray for rendering.

Parameters

ray	The ray to be sent.
pixel	The color of the pixel.
cam	The camera used for rendering.
depthReflexion	The depth of reflection.
closest_object	The closest object to the ray.

Note

This function sends the ray for rendering and computes the color of the pixel.

6.36.3.15 setProgressCallback()

```
void Render::setProgressCallback (
            const std::function< void() > & callback)
```

Set the progress callback function.

Parameters

callback	The callback function to be called on progress.

Note

This function sets the callback function to be called when the rendering progress is updated.

Parameters

callback	The callback function to set.

Note

This function sets the callback function to be called

The documentation for this class was generated from the following files:

- include/core/Render.hpp
- src/core/CalculateRender.cpp
- src/core/Render.cpp

6.37 RenderException Class Reference

Exception class for the raytracer project.

```
#include <Exception.hpp>
```

Inheritance diagram for RenderException:

Collaboration diagram for RenderException:

Public Member Functions

- RenderException (const std::string &message)

  Handle the exception in render.

6.37.1 Detailed Description

Exception class for the raytracer project.

This class is used to handle exceptions in the render system of the raytracer project.

6.37.2 Constructor & Destructor Documentation

6.37.2.1 RenderException()

```
RenderException::RenderException (
            const std::string & message)  [explicit]
```

Handle the exception in render.

Handle the exception in render scene.

Parameters

message	The message to display.

The documentation for this class was generated from the following files:

- include/exceptions/Exception.hpp
- src/exceptions/Exception.cpp

6.38 Rotation Class Reference

A class to represent a Rotation transformation.

```
#include <Rotation.hpp>
```

Inheritance diagram for Rotation:

Collaboration diagram for Rotation:

Public Member Functions

- Rotation ()

  Default constructor for Rotation class. Initializes the Rotation with default values.
- ~Rotation () override=default

  Destructor for Rotation class. Cleans up resources used by the Rotation.
- void apply (Math::Point &origin, Math::Vector Rotation) override

  Apply the Rotation to the given vector.
- void RotateX (bool Ctrl_pressed, Scene &scene)

  Rotate in the X axis.
- void RotateY (bool Ctrl_pressed, Scene &scene)

  Rotate in the Y axis.
- void RotateZ (bool Ctrl_pressed, Scene &scene)

  Rotate in the Y axis.

Public Member Functions inherited from ITransformations

- virtual ~ITransformations ()=default

  Virtual destructor for ITransformations class.

6.38.1 Detailed Description

A class to represent a Rotation transformation.

This class is used to represent a Rotation transformation in the raytracer project. It inherits from the ITransformations interface.

Note

The Rotation class is responsible for applying Rotation transformations

6.38.2 Constructor & Destructor Documentation

6.38.2.1 Rotation()

```
Rotation::Rotation ()
```

Default constructor for Rotation class. Initializes the Rotation with default values.

Note

This constructor can be used to create a Rotation with default values.

6.38.2.2 ~Rotation()

```
Rotation::~Rotation () [override], [default]
```

Destructor for Rotation class. Cleans up resources used by the Rotation.

Note

This destructor is called when the Rotation object is destroyed.

6.38.3 Member Function Documentation

6.38.3.1 apply()

```
void Rotation::apply (
            Math::Point & origin,
            Math::Vector translation)  [override], [virtual]
```

Apply the Rotation to the given vector.

Apply the translation to the given vector.

Parameters

inverted	A boolean indicating whether to apply the transformation in reverse.

Note

This function applies the Rotation to the given vector.

This function applies the translation to the given vector.

Implements ITransformations.

6.38.3.2 RotateX()

```
void Rotation::RotateX (
            bool Ctrl_pressed,
            Scene & scene)
```

Rotate in the X axis.

6.38.3.3 RotateY()

```
void Rotation::RotateY (
            bool Ctrl_pressed,
            Scene & scene)
```

Rotate in the Y axis.

6.38.3.4 RotateZ()

```
void Rotation::RotateZ (
            bool Ctrl_pressed,
            Scene & scene)
```

Rotate in the Y axis.

The documentation for this class was generated from the following files:

- include/transformations/Rotation.hpp
- src/transformation/Rotation.cpp

6.39 Scene Class Reference

A class to represent a scene in the raytracer project.

```
#include <Scene.hpp>
```

Public Member Functions

- Scene ()

  Default constructor for Scene class. Initializes the scene with a camera, lights, and primitives.
- ~Scene ()

  Destructor for Scene class. Cleans up the resources used by the scene.
- void addCamera (const Camera &camera)

  Add a camera to the scene.
- void addLight (const std::shared_ptr< ILight > &light)

  Add a light to the scene.
- void addPrimitive (const std::shared_ptr< IPrimitive > &primitive)

  Add a primitive to the scene.
- Camera getCamera () const

  Get the camera of the scene.
- Camera & getModifableCamera ()

  Get the camera of the scene.
- const std::vector< std::shared_ptr< IPrimitive > > & getPrimitives () const

  Get the vector of primitives.
- const std::vector< std::shared_ptr< ILight > > & getLights () const

  Get the vector of lights.
- double getDiffuse () const

  Get the diffuse value of the scene.
- void setDiffuse (double diffuse)

  Set the diffuse value of the scene.
- void setTranslateScene (Math::Vector translation)

  Call the translate function of the camera.
- void setRotate (double angle, std::string axis)
- void setShear (double strength)

6.39.1 Detailed Description

A class to represent a scene in the raytracer project.

6.39.2 Constructor & Destructor Documentation

6.39.2.1 Scene()

```
Scene::Scene ()
```

Default constructor for Scene class. Initializes the scene with a camera, lights, and primitives.

Note

This constructor can be used to create a scene with default values.

6.39.2.2 ~Scene()

```
Scene::~Scene ()
```

Destructor for Scene class. Cleans up the resources used by the scene.

6.39.3 Member Function Documentation

6.39.3.1 addCamera()

```
void Scene::addCamera (
            const Camera & camera)
```

Add a camera to the scene.

Parameters

camera	The camera to add to the scene.

6.39.3.2 addLight()

```
void Scene::addLight (
            const std::shared_ptr< ILight > & light)
```

Add a light to the scene.

Parameters

light	The light to add to the scene.

6.39.3.3 addPrimitive()

```
void Scene::addPrimitive (
            const std::shared_ptr< IPrimitive > & primitive)
```

Add a primitive to the scene.

Parameters

primitive	The primitive to add to the scene.

6.39.3.4 getCamera()

```
Camera Scene::getCamera () const
```

Get the camera of the scene.

Returns

The camera of the scene.

6.39.3.5 getDiffuse()

```
double Scene::getDiffuse () const
```

Get the diffuse value of the scene.

Returns

The diffuse value of the scene.

6.39.3.6 getLights()

```
const std::vector< std::shared_ptr< ILight > > & Scene::getLights () const
```

Get the vector of lights.

Returns

The vector of lights in the scene.

6.39.3.7 getModifableCamera()

```
Camera & Scene::getModifableCamera ()
```

Get the camera of the scene.

Returns

The camera of the scene.

6.39.3.8 getPrimitives()

```
const std::vector< std::shared_ptr< IPrimitive > > & Scene::getPrimitives () const
```

Get the vector of primitives.

Returns

The vector of primitives in the scene.

6.39.3.9 setDiffuse()

```
void Scene::setDiffuse (
            double diffuse)
```

Set the diffuse value of the scene.

Parameters

diffuse	The diffuse value to set.

6.39.3.10 setRotate()

```
void Scene::setRotate (
            double angle,
            std::string axis)
```

6.39.3.11 setShear()

```
void Scene::setShear (
            double strength)
```

6.39.3.12 setTranslateScene()

```
void Scene::setTranslateScene (
            Math::Vector translation)
```

Call the translate function of the camera.

Parameters

translation	Vector that define the translation

The documentation for this class was generated from the following files:

- include/core/Scene.hpp
- src/core/Scene.cpp

6.40 SceneException Class Reference

Exception class for the raytracer project.

```
#include <Exception.hpp>
```

Inheritance diagram for SceneException:

Collaboration diagram for SceneException:

Public Member Functions

- SceneException (const std::string &message)

  Handle the exception in scene.

6.40.1 Detailed Description

Exception class for the raytracer project.

This class is used to handle exceptions in the scene system of the raytracer project.

6.40.2 Constructor & Destructor Documentation

6.40.2.1 SceneException()

```
SceneException::SceneException (
            const std::string & message)  [explicit]
```

Handle the exception in scene.

Handle the exception in scene parsing.

Parameters

message	The message to display.

The documentation for this class was generated from the following files:

- include/exceptions/Exception.hpp
- src/exceptions/Exception.cpp

6.41 SceneLoader Class Reference

A class to load scenes from files in the raytracer project.

```
#include <SceneLoader.hpp>
```

Static Public Member Functions

- static Scene parseSceneFile (const std::string &filename)

  Load a scene from a file.
- static void parseSceneCamera (const libconfig::Setting &file, Scene &scene)

  Parse the scene file and create the camera.
- static void parseScenePrimitive (const libconfig::Setting &file, Scene &scene)

  Parse the scene file and create the primitive.
- static void parseSceneLight (const libconfig::Setting &file, Scene &scene)

  Parse the scene file and create the light.

6.41.1 Detailed Description

A class to load scenes from files in the raytracer project.

6.41.2 Member Function Documentation

6.41.2.1 parseSceneCamera()

```
void SceneLoader::parseSceneCamera (
            const libconfig::Setting & file,
            Scene & scene)  [static]
```

Parse the scene file and create the camera.

Parameters

file	The file to parse.
scene	The scene object to fill.

6.41.2.2 parseSceneFile()

```
Scene SceneLoader::parseSceneFile (
            const std::string & filename)  [static]
```

Load a scene from a file.

Parameters

filename	The name of the file to load the scene from.

Returns

A pointer to the loaded scene.

Parameters

filename	The name of the file to load the scene from.

Returns

A pointer to the loaded Scene object.

6.41.2.3 parseSceneLight()

```
void SceneLoader::parseSceneLight (
            const libconfig::Setting & file,
            Scene & scene)  [static]
```

Parse the scene file and create the light.

Parameters

file	The file to parse.
scene	The scene object to fill.

6.41.2.4 parseScenePrimitive()

```
void SceneLoader::parseScenePrimitive (
            const libconfig::Setting & file,
            Scene & scene)  [static]
```

Parse the scene file and create the primitive.

Parameters

file	The file to parse.
scene	The scene object to fill.

The documentation for this class was generated from the following files:

- include/loader/SceneLoader.hpp
- src/loader/SceneLoader.cpp

6.42 Screen Class Reference

```
#include <Screen.hpp>
```

Public Member Functions

- Screen ()
- Screen (Math::Point origin, Math::Vector bottom_side, Math::Vector left_side)
- ~Screen ()
- Math::Point pointAt (double u, double v)
- void translate (Math::Vector translation)
- void rotate (double angle, std::string axis)
- void shear (double strength)
- Math::Point getOrigin () const

6.42.1 Constructor & Destructor Documentation

6.42.1.1 Screen() [1/2]

```
Screen::Screen ()
```

6.42.1.2 Screen() [2/2]

```
Screen::Screen (
            Math::Point origin,
            Math::Vector bottom_side,
            Math::Vector left_side)
```

6.42.1.3 ~Screen()

```
Screen::~Screen ()
```

6.42.2 Member Function Documentation

6.42.2.1 getOrigin()

```
Math::Point Screen::getOrigin () const
```

6.42.2.2 pointAt()

```
Math::Point Screen::pointAt (
            double u,
            double v)
```

6.42.2.3 rotate()

```
void Screen::rotate (
            double angle,
            std::string axis)
```

6.42.2.4 shear()

```
void Screen::shear (
            double strength)
```

6.42.2.5 translate()

```
void Screen::translate (
            Math::Vector translation)
```

The documentation for this class was generated from the following files:

- include/core/Screen.hpp
- src/core/Screen.cpp

6.43 Shear Class Reference

A class to represent a Shear transformation.

```
#include <Shear.hpp>
```

Inheritance diagram for Shear:

Collaboration diagram for Shear:

Public Member Functions

- Shear ()

  Default constructor for Shear class. Initializes the Shear with default values.
- ~Shear () override=default

  Destructor for Shear class. Cleans up resources used by the Shear.
- void apply (Math::Point &origin, Math::Vector Shear) override

  Apply the Shear to the given vector.
- void shear (Scene &scene)

  Rotate in the X axis.

Public Member Functions inherited from ITransformations

- virtual ~ITransformations ()=default

  Virtual destructor for ITransformations class.

6.43.1 Detailed Description

A class to represent a Shear transformation.

This class is used to represent a Shear transformation in the raytracer project. It inherits from the ITransformations interface.

Note

The Shear class is responsible for applying Shear transformations

6.43.2 Constructor & Destructor Documentation

6.43.2.1 Shear()

```
Shear::Shear ()
```

Default constructor for Shear class. Initializes the Shear with default values.

Note

This constructor can be used to create a Shear with default values.

6.43.2.2 ~Shear()

```
Shear::~Shear () [override], [default]
```

Destructor for Shear class. Cleans up resources used by the Shear.

Note

This destructor is called when the Shear object is destroyed.

6.43.3 Member Function Documentation

6.43.3.1 apply()

```
void Shear::apply (
            Math::Point & origin,
            Math::Vector translation) [override], [virtual]
```

Apply the Shear to the given vector.

Apply the translation to the given vector.

Parameters

inverted	A boolean indicating whether to apply the transformation in reverse.

Note

This function applies the Shear to the given vector.

This function applies the translation to the given vector.

Implements ITransformations.

6.43.3.2 shear()

```
void Shear::shear (
            Scene & scene)
```

Rotate in the X axis.

Rotate in the Y axis.

The documentation for this class was generated from the following files:

- include/transformations/Shear.hpp
- src/transformation/Shear.cpp

6.44 Sphere Class Reference

A class to represent a Sphere in 3D space.

```
#include <Sphere.hpp>
```

Inheritance diagram for Sphere:

Collaboration diagram for Sphere:

Public Member Functions

- Sphere (Math::Vector center, float radius, const Color &color, const std::shared_ptr< AMaterial > &mat)

  Constructor for the Sphere class.
- ~Sphere () override=default

  Destructor for the Sphere class.
- const std::string & getType () const override

  Get the type of the Sphere.
- bool intersect (const Ray &ray) const override

  Check if a ray intersects with the Sphere.
- Math::Point getIntersection (const Ray &ray) const override

  This function does exactly what you think it does.
- Math::Vector normal_at (const Math::Point &point) const override

  Create a normal vector to the shape.
- Color getColor () const override

  Get the color of the Sphere.
- std::shared_ptr< AMaterial > getMaterial () const override

  Get the material of the primitive shape.

Public Member Functions inherited from IPrimitive

- virtual ~IPrimitive ()=default

  Virtual destructor for IPrimitive class.

6.44.1 Detailed Description

A class to represent a Sphere in 3D space.

6.44.2 Constructor & Destructor Documentation

6.44.2.1 Sphere()

```
Sphere::Sphere (
            Math::Vector center,
            float radius,
            const Color & color,
            const std::shared_ptr< AMaterial > & mat)
```

Constructor for the Sphere class.

Parameters

center	the center of the sphere (x, y, or z).
radius	the radius of the sphere
color	The color of the sphere.

6.44.2.2 ~Sphere()

```
Sphere::~Sphere () [override], [default]
```

Destructor for the Sphere class.

Note

This destructor is virtual to ensure proper cleanup of derived classes.

6.44.3 Member Function Documentation

6.44.3.1 getColor()

```
Color Sphere::getColor () const [override], [virtual]
```

Get the color of the Sphere.

Get the color of the sphere.

Returns

The color of the Sphere.

The color of the sphere.

Implements IPrimitive.

6.44.3.2 getIntersection()

```
Math::Point Sphere::getIntersection (
            const Ray & ray) const  [override], [virtual]
```

This function does exactly what you think it does.

Parameters

ray	The ray to check for intersection.

Returns

the intersection point between a Ray and a shape

Implements IPrimitive.

6.44.3.3 getMaterial()

```
std::shared_ptr< AMaterial > Sphere::getMaterial () const  [override], [virtual]
```

Get the material of the primitive shape.

Returns

The material of the shape.

Implements IPrimitive.

6.44.3.4 getType()

```
const std::string & Sphere::getType () const  [override], [virtual]
```

Get the type of the Sphere.

Get the type of the sphere.

Returns

The type of the Sphere as a string.

The type of the sphere as a string.

Implements IPrimitive.

6.44.3.5 intersect()

```
bool Sphere::intersect (
            const Ray & ray) const  [override], [virtual]
```

Check if a ray intersects with the Sphere.

Check if a ray intersects with the sphere.

Parameters

ray	The ray to check for intersection.

Returns

True if the ray intersects with the Sphere, false otherwise.

Parameters

ray	The ray to check for intersection.

Returns

True if the ray intersects with the sphere, false otherwise.

Implements IPrimitive.

6.44.3.6 normal_at()

```
Math::Vector Sphere::normal_at (
            const Math::Point & point) const  [override], [virtual]
```

Create a normal vector to the shape.

Parameters

point	origin of the normal vector

Returns

normal vector to the shape

Implements IPrimitive.

The documentation for this class was generated from the following files:

- include/plugins/primitives/Sphere.hpp
- src/plugins/primitives/Sphere.cpp

6.45 Translation Class Reference

A class to represent a translation transformation.

```
#include <Translation.hpp>
```

Inheritance diagram for Translation:

Collaboration diagram for Translation:

Public Member Functions

- Translation ()

  Default constructor for Translation class. Initializes the translation with default values.
- ~Translation () override=default

  Destructor for Translation class. Cleans up resources used by the translation.
- void apply (Math::Point &origin, Math::Vector translation) override

  Apply the translation to the given vector.
- void translateUp (Scene &scene)

  translate all the scene up
- void translateDown (Scene &scene)

  translate all the scene down
- void translateLeft (Scene &scene)

  translate all the scene left
- void translateRight (Scene &scene)

  translate all the scene right
- void translateIn (Scene &scene)

  translate all the scene up
- void translateOut (Scene &scene)

  translate all the scene down

Public Member Functions inherited from ITransformations

- virtual ~ITransformations ()=default

  Virtual destructor for ITransformations class.

6.45.1 Detailed Description

A class to represent a translation transformation.

This class is used to represent a translation transformation in the raytracer project. It inherits from the ITransformations interface.

Note

The Translation class is responsible for applying translation transformations

6.45.2 Constructor & Destructor Documentation

6.45.2.1 Translation()

```
Translation::Translation ()
```

Default constructor for Translation class. Initializes the translation with default values.

Note

This constructor can be used to create a translation with default values.

Parameters

translation	The translation

Note

This constructor can be used to create a translation with default values.

6.45.2.2 ~Translation()

```
Translation::~Translation () [override], [default]
```

Destructor for Translation class. Cleans up resources used by the translation.

Note

This destructor is called when the translation object is destroyed.

6.45.3 Member Function Documentation

6.45.3.1 apply()

```
void Translation::apply (
            Math::Point & origin,
            Math::Vector translation) [override], [virtual]
```

Apply the translation to the given vector.

Parameters

inverted	A boolean indicating whether to apply the transformation in reverse.

Note

This function applies the translation to the given vector.

This function applies the translation to the given vector.

Implements ITransformations.

6.45.3.2 translateDown()

```
void Translation::translateDown (
            Scene & scene)
```

translate all the scene down

6.45.3.3 translateIn()

```
void Translation::translateIn (
            Scene & scene)
```

translate all the scene up

translate all the scene left

6.45.3.4 translateLeft()

```
void Translation::translateLeft (
            Scene & scene)
```

translate all the scene left

6.45.3.5 translateOut()

```
void Translation::translateOut (
            Scene & scene)
```

translate all the scene down

translate all the scene right

6.45.3.6 translateRight()

```
void Translation::translateRight (
            Scene & scene)
```

translate all the scene right

6.45.3.7 translateUp()

```
void Translation::translateUp (
            Scene & scene)
```

translate all the scene up

The documentation for this class was generated from the following files:

- include/transformations/Translation.hpp
- src/transformation/Translation.cpp

6.46 Transparent Class Reference

```
#include <Transparent.hpp>
```

Inheritance diagram for Transparent:

Collaboration diagram for Transparent:

Public Member Functions

- Transparent ()

Public Member Functions inherited from AMaterial

- AMaterial (const std::string &name, float ambient, float diffuse, float specular, float shininess)

  Construct a new AMaterial object with given coefficients.
- virtual ~AMaterial ()=default
- const std::string & getName () const
- float getAmbient () const
- float getDiffuse () const
- float getSpecular () const
- float getShininess () const
- float getTransparency () const
- float getRefractionIndex () const
- void setDiffuseColor (const Color &color)

  Set the diffuse color of the material.
- const Color & getDiffuseColor () const

  Get the diffuse color of the material.
- void setSpecularColor (const Color &color)

  Set the specular color of the material.
- const Color & getSpecularColor () const

  Get the specular color of the material.
- void setShininess (float shininess)

  Set the shininess of the material.
- virtual Color getColorAt (const Math::Point &point) const

  Get the color at a specific point on the surface (for procedural textures)
- void setTransparency (float transparency)

  Set the transparency of the material.
- void setRefractionIndex (float refractionIndex)

  Set the refraction index of the material.
- void loadFromMtlLine (const std::string &line)

  Load material properties from a .mtl file line.

6.46.1 Constructor & Destructor Documentation

6.46.1.1 Transparent()

```
Transparent::Transparent ()
```

The documentation for this class was generated from the following files:

- include/materials/Transparent.hpp
- src/materials/Transparent.cpp

6.47 Triangle Class Reference

A class to represent a Triangle in 3D space.

```
#include <Triangle.hpp>
```

Inheritance diagram for Triangle:

Collaboration diagram for Triangle:

Public Member Functions

- Triangle (Math::Point v0, Math::Point v1, Math::Point v2, const Color &color, const std::shared_ptr< AMaterial > &mat)

  Constructor for the Triangle class.
- ~Triangle () override=default

  Destructor for the Triangle class.
- const std::string & getType () const override

  Get the type of the Triangle.
- bool intersect (const Ray &ray) const override

  Check if a ray intersects with the Triangle.
- Math::Point getIntersection (const Ray &ray) const override

  This function does exactly what you think it does.
- Math::Vector normal_at (const Math::Point &point) const override

  Create a normal vector to the shape.
- Color getColor () const override

  Get the color of the Triangle.
- std::shared_ptr< AMaterial > getMaterial () const override

  Get the material of the primitive shape.

Public Member Functions inherited from IPrimitive

- virtual ~IPrimitive ()=default

  Virtual destructor for IPrimitive class.

6.47.1 Detailed Description

A class to represent a Triangle in 3D space.

6.47.2 Constructor & Destructor Documentation

6.47.2.1 Triangle()

```
Triangle::Triangle (
            Math::Point v0,
            Math::Point v1,
            Math::Point v2,
            const Color & color,
            const std::shared_ptr< AMaterial > & mat)
```

Constructor for the Triangle class.

Parameters

axis	The axis of the Triangle (x, y, or z).
v0	The position of the first vertex
v1	The position of the second vertex
v2	The position of the third vertex
color	The color of the Triangle.

6.47.2.2 ~Triangle()

```
Triangle::~Triangle () [override], [default]
```

Destructor for the Triangle class.

Note

This destructor is virtual to ensure proper cleanup of derived classes.

6.47.3 Member Function Documentation

6.47.3.1 getColor()

```
Color Triangle::getColor () const [override], [virtual]
```

Get the color of the Triangle.

Returns

The color of the Triangle.

Implements IPrimitive.

6.47.3.2 getIntersection()

```
Math::Point Triangle::getIntersection (
            const Ray & ray) const [override], [virtual]
```

This function does exactly what you think it does.

Parameters

ray	The ray to check for intersection.

Returns

the intersection point between a Ray and a shape

Implements IPrimitive.

6.47.3.3 getMaterial()

```
std::shared_ptr< AMaterial > Triangle::getMaterial () const  [override], [virtual]
```

Get the material of the primitive shape.

Returns

The material of the shape.

Implements IPrimitive.

6.47.3.4 getType()

```
const std::string & Triangle::getType () const  [override], [virtual]
```

Get the type of the Triangle.

Returns

The type of the Triangle as a string.

Implements IPrimitive.

6.47.3.5 intersect()

```
bool Triangle::intersect (
            const Ray & ray) const  [override], [virtual]
```

Check if a ray intersects with the Triangle.

Parameters

ray	The ray to check for intersection.

Returns

True if the ray intersects with the Triangle, false otherwise.

Implements IPrimitive.

6.47.3.6 normal_at()

```
Math::Vector Triangle::normal_at (
            const Math::Point & point) const  [override], [virtual]
```

Create a normal vector to the shape.

Parameters

point	origin of the normal vector

Returns

normal vector to the shape

Implements IPrimitive.

The documentation for this class was generated from the following files:

- include/plugins/primitives/Triangle.hpp
- src/plugins/primitives/Triangle.cpp

6.48 Math::Vector Class Reference

A class to represent a 3D vector with basic operations.

```
#include <Vector.hpp>
```

Public Member Functions

- Vector (double x=0, double y=0, double z=0)

  Default constructor for Vector class. Initializes the vector to (0, 0, 0).
- double length () const

  Length of the vector.
- double dot (const Vector &other) const

  Dot product of two vectors.
- Vector normalize () const

  Normalize the vector.
- Vector invert () const
- Vector operator+ (const Vector &other) const

  Addition operator overload.
- Vector & operator+= (const Vector &other)

  Addition operator overload.
- Vector operator- (const Vector &other) const

  Subtraction operator overload.
- Vector & operator-= (const Vector &other)

  Subtraction operator overload.
- Vector operator∗ (const Vector &other) const

  Scalar multiplication operator overload.
- Vector & operator∗= (const Vector &other)

  Scalar multiplication operator overload.
- Vector operator∗ (double scalar) const

  Scalar multiplication operator overload.
- Vector & operator∗= (double scalar)

  Scalar multiplication operator overload.

- Vector operator/ (const Vector &other) const

  Scalar division operator overload.
- Vector & operator/= (const Vector &other)

  Scalar division operator overload.
- Vector operator/ (double scalar) const

  Scalar division operator overload.
- Vector & operator/= (double scalar)

  Scalar division operator overload.
- bool operator== (const Vector &other) const

  Check equality operator overload.
- bool operator== (std::string axis) const

  Check equality operator overload.
- double getX () const

  Get the x-coordinate of the vector.
- double getY () const

  Get the y-coordinate of the vector.
- double getZ () const

  Get the z-coordinate of the vector.
- Vector crossProduct (const Vector &other) const

  Cross product of two vector.
- Math::Matrix< 4, 1 > vectorToMatrix ()
- Math::Matrix< 1, 2 > shearToMatrix ()
- double random_double ()
- double random_double (double min, double max)
- Vector random ()
- Vector random (double min, double max)
- bool near_zero () const
- Vector reflect (const Vector &v, const Vector &n)
- Vector operator- () const

  Unary negation operator overload.

6.48.1 Detailed Description

A class to represent a 3D vector with basic operations.

6.48.2 Constructor & Destructor Documentation

6.48.2.1 Vector()

```
Math::Vector::Vector (
            double x = 0,
            double y = 0,
            double z = 0)
```

Default constructor for Vector class. Initializes the vector to (0, 0, 0).

Parameters

x	The x-coordinate of the vector.
y	The y-coordinate of the vector.
z	The z-coordinate of the vector.

Note

If no parameters are provided, the vector is initialized to (0, 0, 0).

This constructor can be used to create a vector with specific coordinates.

6.48.3 Member Function Documentation

6.48.3.1 crossProduct()

```
Vector Math::Vector::crossProduct (
            const Vector & other) const
```

Cross product of two vector.

Parameters

other	The vector to calculate the cross product

Returns

A new vector that is the result of the cross product between two vectors

Note

This function can be used to calculate a normal vector of two other ones

6.48.3.2 dot()

```
double Math::Vector::dot (
            const Vector & other) const
```

Dot product of two vectors.

Parameters

other	The vector to dot with.

Returns

The dot product of this vector and the other vector.

6.48.3.3 getX()

```
double Math::Vector::getX () const
```

Get the x-coordinate of the vector.

Returns

The x-coordinate of the vector.

6.48.3.4 getY()

```
double Math::Vector::getY () const
```

Get the y-coordinate of the vector.

Returns

The y-coordinate of the vector.

@brif Get the y-coordinate of the vector.

Returns

The y-coordinate of the vector.

6.48.3.5 getZ()

```
double Math::Vector::getZ () const
```

Get the z-coordinate of the vector.

Returns

The z-coordinate of the vector.

@brif Get the z-coordinate of the vector.

Returns

The z-coordinate of the vector.

6.48.3.6 invert()

```
Vector Math::Vector::invert () const
```

6.48.3.7 length()

```
double Math::Vector::length () const
```

Length of the vector.

Returns

The length of the vector.

6.48.3.8 near_zero()

```
bool Math::Vector::near_zero () const
```

6.48.3.9 normalize()

```
Vector Math::Vector::normalize () const
```

Normalize the vector.

Returns

A new vector that is the normalized version of this vector.

6.48.3.10 operator∗() [1/2]

```
Vector Math::Vector::operator* (
            const Vector & other) const
```

Scalar multiplication operator overload.

Parameters

other	The vector to multiply with.

Returns

A new vector that is the product of this vector and the scalar.

6.48.3.11 operator∗() [2/2]

```
Vector Math::Vector::operator* (
            double scalar) const
```

Scalar multiplication operator overload.

Parameters

scalar	The scalar to multiply with.

Returns

A new vector that is the product of this vector and the scalar.

6.48.3.12 operator∗=() [1/2]

```
Vector & Math::Vector::operator*= (
            const Vector & other)
```

Scalar multiplication operator overload.

Parameters

other	The vector to multiply with.

Returns

A new vector that is the product of this vector and the scalar.

6.48.3.13 operator*=() [2/2]

```
Vector & Math::Vector::operator*= (
            double scalar)
```

Scalar multiplication operator overload.

Parameters

scalar	The scalar to multiply with.

Returns

A new vector that is the product of this vector and the scalar.

6.48.3.14 operator+()

```
Vector Math::Vector::operator+ (
            const Vector & other) const
```

Addition operator overload.

Parameters

other	The vector to add.

Returns

A new vector that is the sum of this vector and the other vector.

6.48.3.15 operator+=()

```
Vector & Math::Vector::operator+= (
            const Vector & other)
```

Addition operator overload.

Parameters

other	The vector to add.

Returns

A new vector that is the sum of this vector and the other vector.

6.48.3.16 operator-() [1/2]

```
Vector Math::Vector::operator- () const [inline]
```

Unary negation operator overload.

Returns

A new vector that is the negation of this vector.

6.48.3.17 operator-() [2/2]

```
Vector Math::Vector::operator- (
            const Vector & other) const
```

Subtraction operator overload.

Parameters

other	The vector to subtract.

Returns

A new vector that is the difference of this vector and the other vector.

6.48.3.18 operator-=()

```
Vector & Math::Vector::operator-= (
            const Vector & other)
```

Subtraction operator overload.

Parameters

other	The vector to subtract.

Returns

A new vector that is the difference of this vector and the other vector.

6.48.3.19 operator/() [1/2]

```
Vector Math::Vector::operator/ (
            const Vector & other) const
```

Scalar division operator overload.

Parameters

other	The vector to divide with.

Returns

A new vector that is the quotient of this vector and the scalar.

6.48.3.20 operator/() [2/2]

```
Vector Math::Vector::operator/ (
            double scalar) const
```

Scalar division operator overload.

Parameters

scalar	The scalar to divide by.

Returns

A new vector that is the quotient of this vector and the scalar.

6.48.3.21 operator/=() [1/2]

```
Vector & Math::Vector::operator/= (
            const Vector & other)
```

Scalar division operator overload.

Parameters

other	The vector to divide with.

Returns

A new vector that is the quotient of this vector and the scalar.

6.48.3.22 operator/=() [2/2]

```
Vector & Math::Vector::operator/= (
            double scalar)
```

Scalar division operator overload.

Parameters

scalar	The scalar to divide by.

Returns

A new vector that is the quotient of this vector and the scalar.

6.48.3.23 operator==() [1/2]

```
bool Math::Vector::operator== (
            const Vector & other) const
```

Check equality operator overload.

Parameters

other	The vector to compare with.

Returns

True if the vectors are equal, false otherwise.

6.48.3.24 operator==() [2/2]

```
bool Math::Vector::operator== (
            std::string axis) const
```

Check equality operator overload.

Parameters

other	The vector to compare with.

Returns

True if the vectors are equal, false otherwise.

6.48.3.25 random() [1/2]

```
Vector Math::Vector::random ()
```

6.48.3.26 random() [2/2]

```
Vector Math::Vector::random (
            double min,
            double max)
```

6.48.3.27 random_double() [1/2]

```
double Math::Vector::random_double () [inline]
```

6.48.3.28 random_double() [2/2]

```
double Math::Vector::random_double (
            double min,
            double max) [inline]
```

6.48.3.29 reflect()

```
Vector Math::Vector::reflect (
            const Vector & v,
            const Vector & n)
```

6.48.3.30 shearToMatrix()

```
Math::Matrix< 1, 2 > Math::Vector::shearToMatrix () [inline]
```

6.48.3.31 vectorToMatrix()

```
Math::Matrix< 4, 1 > Math::Vector::vectorToMatrix () [inline]
```

The documentation for this class was generated from the following files:

- include/utilitaries/Vector.hpp
- src/utilitaries/Vector.cpp

Chapter 7

File Documentation

7.1 include/core/Camera.hpp File Reference

```
#include "Vector.hpp"
#include "Matrix.hpp"
#include "Ray.hpp"
#include "Screen.hpp"
```

Include dependency graph for Camera.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class Camera

  A class to represent a camera in the raytracer project.

7.2 Camera.hpp

Go to the documentation of this file.

```
/*
** EPITECH PROJECT, 2025
** Sans titre(Espace de travail)
** File description:
** Camera
*/


#pragma once

#include "Vector.hpp"
#include "Matrix.hpp"
#include "Ray.hpp"
#include "Screen.hpp"
class Camera {
    public:
        Camera(const Math::Vector &position = Math::Vector(), const Math::Vector &rotation = Math::Vector(),
               double fov = 90.0f, int width = 800, int height = 600);

        const Math::Point &getPosition() const;

        double getFov() const;

        int getWidth() const;

        int getHeight() const;

        void setRay(double x, double y);
```

```
        Ray getRay() const;

        void setPosition(const Math::Point &position);

        void translate(Math::Vector translation);

        void rotate(double angle, std::string axis);

        void shear(double strength);

    private:
        Math::Point _position;
        Math::Vector _rotation;
        Ray _ray;
        Screen _screen;
        double _fov;
        int _width;
        int _height;
};
```

7.3 include/core/Core.hpp File Reference

Core class header file.

```
#include "Scene.hpp"
#include "SceneLoader.hpp"
#include "Gui.hpp"
#include "Render.hpp"
#include "Image.hpp"
#include "Define.hpp"
#include "Translation.hpp"
#include "Rotation.hpp"
#include "Shear.hpp"
```

Include dependency graph for Core.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class Core

  A class to manage the raytracer project.

7.3.1 Detailed Description

Core class header file.

This file contains the definition of the Core class, which is used to manage the raytracer project.

Note

The Core class is responsible for initializing and managing the raytracer project.

7.4 Core.hpp

Go to the documentation of this file.

```
/*
** EPITECH PROJECT, 2025
** Sans titre(Espace de travail)
** File description:
** Core
*/

#pragma once

#include "Scene.hpp"
#include "SceneLoader.hpp"
#include "Gui.hpp"
#include "Render.hpp"
#include "Image.hpp"
#include "Define.hpp"
#include "Translation.hpp"
#include "Rotation.hpp"
#include "Shear.hpp"

class Core {
    public:
        Core();

        ~Core();

        void init(const std::string &filename);

        void loop();

        void createGuiThread();

        void createRenderThread();

        void setCtrl();

        void setHandleTranslation();

        void setHandleRotation();

        void notifyRenderThread();

        void checkSceneFile();

        void reloadScene();

    private:
        bool _guiReady = false;

        std::string _scenePath;
        std::filesystem::file_time_type _lastModificationTime;

        Scene _scene;
        PluginsManager _pluginsManager;
        std::shared_ptr<Gui> _gui;
        std::shared_ptr<Render> _render;

        std::mutex _guiMutex;
        std::mutex _guiReadyMutex;
        std::mutex _renderMutex;

        std::condition_variable _renderCondition;
        std::condition_variable _guiReadyCond;

        std::atomic<bool> _needRenderUpdate;

        std::thread _guiThread;
        std::thread _renderThread;

        std::vector<Color> _pixelsRender;
        std::vector<Color> _pixelsFastRender;

        std::unique_ptr<Translation>_translate;
        std::unique_ptr<Rotation>_rotation;
        std::unique_ptr<Shear>_shear;
        bool _Ctrl_pressed;
        int _eventKey;
        std::map<int, void (Translation::*)(Scene&)> _handle_translation;
        std::map<int, void (Rotation::*)(bool, Scene&)> _handle_rotation;
};
```

7.5 include/core/Gui.hpp File Reference

GUI class header file.

```
#include "Color.hpp"
#include "Define.hpp"
#include "Camera.hpp"
#include "Cursor.hpp"
```

Include dependency graph for Gui.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class Gui

7.5.1 Detailed Description

GUI class header file.

This file contains the definition of the GUI class, which is used to manage the graphical user interface in the raytracer project.

Note

The GUI class is responsible for rendering the scene in SFML and handling user interactions.

7.6 Gui.hpp

Go to the documentation of this file.

```
00001 /*
00002 ** EPITECH PROJECT, 2025
00003 ** Sans titre(Espace de travail)
00004 ** File description:
00005 ** Gui
00006 */
00007
00015 #pragma once
00016
00017 #include "Color.hpp"
00018 #include "Define.hpp"
00019 #include "Camera.hpp"
00020 #include "Cursor.hpp"
00021
00022 class Gui {
00023     public:
00024
00030         Gui() = default;
00031
00038         Gui(int width, int height);
00039
00044         ~Gui();
00045
00050         void handleInput(int& eventKey);
00051
00056         void updateScene(std::vector<Color> &pixelsRender, std::vector<Color> &pixelsFastRender);
00057
00066         void createImage(sf::Image &image, const std::vector<Color> &pixels, int width, int height);
00067
00072         void renderScene(int& eventKey);
00073
00078         bool isOpen() const;
00079
00080     private:
00081
```

```
        int _width;
        int _height;
        int _fastWidth;
        int _fastHeight;

        bool _isOpen;
        sf::RenderWindow _window;
        sf::Event _event;

        std::vector<Color> _pixelsRender;
        std::vector<Color> _pixelsFastRender;

        sf::Image _renderImage;
        sf::Texture _renderTexture;
        sf::Sprite _renderSprite;
        sf::Image _fastRenderImage;
        sf::Texture _fastRenderTexture;
        sf::Sprite _fastRenderSprite;

        sf::Vector2i _prevMousPos;
        sf::Font _font;

        // TRANSFORMATION
        std::vector<std::shared_ptr<Cursor>> _transfomationCursors;
        std::shared_ptr<Cursor> _translateX;
        std::shared_ptr<Cursor> _translateY;
        std::shared_ptr<Cursor> _translateZ;
        std::shared_ptr<Cursor> _rotateX;
        std::shared_ptr<Cursor> _rotateY;
        std::shared_ptr<Cursor> _rotateZ;
};
```

7.7 include/core/Render.hpp File Reference

Render class header file.

```
#include "Color.hpp"
#include "Define.hpp"
#include "IPrimitive.hpp"
#include "PluginsManager.hpp"
#include "Scene.hpp"
```

Include dependency graph for Render.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class Render

  A class to render the scene in the raytracer project.

7.7.1 Detailed Description

Render class header file.

This file contains the definition of the Render class, which is used to render the scene in the raytracer project.

Note

The Render class is responsible for rendering the scene and generating the final image.

The Render class calculates the color of each pixel in the image

7.8 Render.hpp

Go to the documentation of this file.

```
/*
** EPITECH PROJECT, 2025
** Sans titre(Espace de travail)
** File description:
** Render
*/

#pragma once

#include "Color.hpp"
#include "Define.hpp"
#include "IPrimitive.hpp"
#include "PluginsManager.hpp"
#include "Scene.hpp"

class Render {
    public:
        Render(int width, int height, Scene &scene);

        ~Render();

        void computeRender(std::vector<Color> &pixelsRender);

        void computeFastRender(std::vector<Color> &pixelsFastRender);

        void setProgressCallback(const std::function<void()> &callback);

        Color renderPixel(int x, int y, int maxWidth, int maxHeight);

        Color ComputeSpecularLight(Math::Vector &lightSource,
            Math::Vector normal, const Camera &cam, const std::shared_ptr<ILight> &light,
            float specularIntensity);

        Color ComputeDiffuseLights(const std::shared_ptr<ILight> &light,
            float diffuseStrength);

        Color ComputePixelColor(std::shared_ptr<IPrimitive> closest_object, const Camera& cam,
            const Ray& ray, int ao);

        Math::Vector randomHemisphereDirection(const Math::Vector& normal);

        float AmbiantOcclusion(std::shared_ptr<IPrimitive> closest_object, Ray &ray);

        bool isInShadowOfShape(Math::Point &hit_point, std::shared_ptr<IPrimitive> closest_object,
            Math::Vector &lightSource);

        Math::Vector reflect(Math::Vector incident, Math::Vector normal);

        Color renderReflection(std::shared_ptr<IPrimitive> closest_object,
            const Ray &ray, Camera &cam, int depthReflection);

        Color renderRefraction(std::shared_ptr<IPrimitive> closest_object,
            const Ray &ray, Camera &cam, int depthReflection);

        void sendRay(Ray &ray, Color &pixel, Camera &cam,
            int depthReflexion, std::shared_ptr<IPrimitive> closest_object);

        Color adaptiveSample(double x, double y, double x1, double y1,
            int maxWidth, int maxHeight, int depth);

    private:

        void workerThread(std::vector<Color> &pixelsRender);

        void computeRenderColumns(std::vector<Color> &pixelsRender, int start, int end);

        int _width;
        int _height;

        bool _stopThread = false;

        std::atomic<int> _tasksRemaining{0};

        std::vector<std::thread> _threads;

        std::mutex _queueMutex;
        std::mutex renderMutex;

        std::condition_variable _queueCondition;
        std::queue<std::pair<int, int» _queue;

        std::function<void()> _progressCallback;
```

```
00196
00197         Scene &_scene;
00198 };
```

7.9 include/core/Scene.hpp File Reference

Scene class header file.

```
#include "Camera.hpp"
#include "Vector.hpp"
#include "ILight.hpp"
#include "IPrimitive.hpp"
#include "Define.hpp"
```

Include dependency graph for Scene.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class Scene

  A class to represent a scene in the raytracer project.

7.9.1 Detailed Description

Scene class header file.

This file contains the definition of the Scene class, which is used to manage the scene in the raytracer project.

Note

The Scene class is responsible for managing the objects in the scene. It stores the objects all the entities in the scene like lights, cameras, primitives, etc.

7.10 Scene.hpp

Go to the documentation of this file.

```
00001 /*
00002 ** EPITECH PROJECT, 2025
00003 ** Sans titre(Espace de travail)
00004 ** File description:
00005 ** Scene
00006 */
00007
00016 #pragma once
00017
00018 #include "Camera.hpp"
00019 #include "Vector.hpp"
00020 #include "ILight.hpp"
00021 #include "IPrimitive.hpp"
00022 #include "Define.hpp"
00023
00028 class Scene {
00029     public:
00035         Scene();
00036
00041         ~Scene();
00042
00047         void addCamera(const Camera &camera);
00048
00053         void addLight(const std::shared_ptr<ILight> &light);
```

```
        void addPrimitive(const std::shared_ptr<IPrimitive> &primitive);

        Camera getCamera() const;

        Camera& getModifableCamera();

        const std::vector<std::shared_ptr<IPrimitive»& getPrimitives() const;

        const std::vector<std::shared_ptr<ILight»& getLights() const;

        double getDiffuse() const;

        void setDiffuse(double diffuse);

        void setTranslateScene(Math::Vector translation);

        void setRotate(double angle, std::string axis);

        void setShear(double strength);

    private:
        Camera _camera;
        std::vector<std::shared_ptr <ILight» _lights;
        std::vector<std::shared_ptr <IPrimitive» _primitives;

        double _diffuse = 0.0f;
};
```

7.11 include/core/Screen.hpp File Reference

```
#include "Point.hpp"
#include "Vector.hpp"
```

Include dependency graph for Screen.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class Screen

7.12 Screen.hpp

Go to the documentation of this file.

```
/*
** EPITECH PROJECT, 2025
** raytracer
** File description:
** Screen
*/

#pragma once

#include "Point.hpp"
#include "Vector.hpp"

class Screen {
    public:
        Screen();
        Screen(Math::Point origin, Math::Vector bottom_side, Math::Vector left_side);
        ~Screen();

        Math::Point pointAt(double u, double v);

        void translate(Math::Vector translation);

        void rotate(double angle, std::string axis);

        void shear(double strength);

        Math::Point getOrigin() const;

```

```
    protected:
    private:
        Math::Point _origin;
        Math::Vector _bottom_side;
        Math::Vector _left_side;
};
```

7.13 include/Define.hpp File Reference

```
#include <iostream>
#include <string>
#include <cstring>
#include <vector>
#include <map>
#include <cmath>
#include <stdexcept>
#include <memory>
#include <functional>
#include <filesystem>
#include <dlfcn.h>
#include <queue>
#include <random>
#include <chrono>
#include <fstream>
#include <array>
#include <thread>
#include <mutex>
#include <atomic>
#include <condition_variable>
#include <SFML/Graphics.hpp>
#include <libconfig.h++>
```

Include dependency graph for Define.hpp: This graph shows which files directly or indirectly include this file:

Macros

- #define FAST_RENDER_RATIO 0.15
- #define THREADS_NUMBER 36
- #define RENDER_COLOMNS_DIVISION_PIXEL 20
- #define FONT "./ressources/BebasNeue-Regular.ttf"
- #define BACKGROUNDCOLOR Color(0, 0, 0)
- #define OCCLUSION_SAMPLE_PX 8
- #define TRANSLATE_X "Translate X"
- #define TRANSLATE_Y "Translate Y"
- #define TRANSLATE_Z "Translate Z"
- #define ROTATE_X "Rotate X"
- #define ROTATE_Y "Rotate Y"
- #define ROTATE_Z "Rotate Z"
- #define MAX_DEPTH 3
- #define COLOR_THRESHOLD 0.10

7.13.1 Macro Definition Documentation

7.13.1.1 BACKGROUNDCOLOR

```
#define BACKGROUNDCOLOR Color(0, 0, 0)
```

7.13.1.2 COLOR_THRESHOLD

```
#define COLOR_THRESHOLD 0.10
```

7.13.1.3 FAST_RENDER_RATIO

```
#define FAST_RENDER_RATIO 0.15
```

7.13.1.4 FONT

```
#define FONT "./ressources/BebasNeue-Regular.ttf"
```

7.13.1.5 MAX_DEPTH

```
#define MAX_DEPTH 3
```

7.13.1.6 OCCLUSION_SAMPLE_PX

```
#define OCCLUSION_SAMPLE_PX 8
```

7.13.1.7 RENDER_COLOMNS_DIVISION_PIXEL

```
#define RENDER_COLOMNS_DIVISION_PIXEL 20
```

7.13.1.8 ROTATE_X

```
#define ROTATE_X "Rotate X"
```

7.13.1.9 ROTATE_Y

```
#define ROTATE_Y "Rotate Y"
```

7.13.1.10 ROTATE_Z

```
#define ROTATE_Z "Rotate Z"
```

7.13.1.11 THREADS_NUMBER

```
#define THREADS_NUMBER 36
```

7.13.1.12 TRANSLATE_X

```
#define TRANSLATE_X "Translate X"
```

7.13.1.13 TRANSLATE_Y

```
#define TRANSLATE_Y "Translate Y"
```

7.13.1.14 TRANSLATE_Z

```
#define TRANSLATE_Z "Translate Z"
```

7.14 Define.hpp

Go to the documentation of this file.

```
/*
** EPITECH PROJECT, 2025
** Sans titre(Espace de travail)
** File description:
** Define
*/

#define FAST_RENDER_RATIO 0.15
#define THREADS_NUMBER 36
#define RENDER_COLOMNS_DIVISION_PIXEL 20
#define FONT "./ressources/BebasNeue-Regular.ttf"
#define BACKGROUNDCOLOR Color(0, 0, 0)
#define OCCLUSION_SAMPLE_PX 8

#define TRANSLATE_X "Translate X"
#define TRANSLATE_Y "Translate Y"
#define TRANSLATE_Z "Translate Z"
#define ROTATE_X "Rotate X"
#define ROTATE_Y "Rotate Y"
#define ROTATE_Z "Rotate Z"

#define MAX_DEPTH 3
#define COLOR_THRESHOLD 0.10

#include <iostream>
#include <string>
#include <cstring>
#include <vector>
#include <map>
#include <cmath>
#include <stdexcept>
#include <memory>
#include <functional>
#include <filesystem>
#include <dlfcn.h>
#include <queue>
#include <random>
#include <chrono>
#include <fstream>
#include <array>

#include <thread>
#include <mutex>
#include <atomic>
#include <condition_variable>

#include <SFML/Graphics.hpp>

#include <libconfig.h++>
```

7.15 include/exceptions/Exception.hpp File Reference

Exception class for the raytracer project.

`#include "Define.hpp"`
Include dependency graph for Exception.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class RaytracerException

  Exception class for the raytracer project.
- class PluginsException
- class SceneException

  Exception class for the raytracer project.
- class RenderException

  Exception class for the raytracer project.
- class ImageException

  Exception class for the raytracer project.

7.15.1 Detailed Description

Exception class for the raytracer project.

This class is used to handle exceptions in the raytracer project.

Note

This class is used to handle exceptions in the raytracer project.

This class is used to handle exceptions in the raytracer project.

7.16 Exception.hpp

Go to the documentation of this file.

```
00001 /*
00002 ** EPITECH PROJECT, 2025
00003 ** Sans titre(Espace de travail)
00004 ** File description:
00005 ** Exception
00006 */
00007
00014 #pragma once
00015
00016 #include "Define.hpp"
00017
00023 class RaytracerException : public std::runtime_error {
00024     public:
00025
00029         explicit RaytracerException(const std::string &message);
00030 };
00031
00037 class PluginsException : public std::runtime_error {
00038     public:
00039
00043         explicit PluginsException(const std::string &message);
00044 };
00045
```

```
class SceneException : public std::runtime_error {
    public:

        explicit SceneException(const std::string &message);
};

class RenderException : public std::runtime_error {
    public:

        explicit RenderException(const std::string &message);
};

class ImageException : public std::runtime_error {
    public:

        explicit ImageException(const std::string &message);
};
```

7.17 include/images/Image.hpp File Reference

Image class header file.

```
#include "Color.hpp"
#include "Define.hpp"
```

Include dependency graph for Image.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class Image

  A class to represent an image in the raytracer project.

7.17.1 Detailed Description

Image class header file.

This file contains the definition of the Image class, which is used to manage images in the raytracer project.

Note

The Image class is responsible for generation and saving images.

7.18 Image.hpp

Go to the documentation of this file.

```
/*
** EPITECH PROJECT, 2025
** Sans titre(Espace de travail)
** File description:
** Image
*/

#pragma once

#include "Color.hpp"
#include "Define.hpp"

class Image {
    public:
        Image(int width, int height, std::vector<Color> &pixels);

        ~Image();

        std::filesystem::path createDirAndFileName(const std::string &filename);

        void save(const std::string &filename);

    private:
        int _width;
        int _height;
        std::vector<Color> &_pixels;
};
```

7.19 include/interfaces/ILight.hpp File Reference

ILight class header file.

```
#include "Vector.hpp"
#include "Color.hpp"
#include "Define.hpp"
#include "Point.hpp"
```

Include dependency graph for ILight.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class ILight

  An interface for light sources in the raytracer project.

7.19.1 Detailed Description

ILight class header file.

This file contains the definition of the ILight class, which is an interface for light sources in the raytracer project.

Note

The ILight class is used to define the interface for all light sources.

All light sources must implement this interface.

7.20 ILight.hpp

Go to the documentation of this file.

```
00001 /*
00002 ** EPITECH PROJECT, 2025
00003 ** Sans titre(Espace de travail)
00004 ** File description:
00005 ** ILight
00006 */
00007
00016 #pragma once
00017
00018 #include "Vector.hpp"
00019 #include "Color.hpp"
00020 #include "Define.hpp"
00021 #include "Point.hpp"
00022
00027 class ILight {
00028     public:
00033         virtual ~ILight() = default;
00034
00039         virtual const std::string &getType() const = 0;
00040
00045         virtual const Math::Point &getPosition() const = 0;
00046
00052         virtual const Math::Vector &getDirection() const = 0;
00053
00058         virtual float getIntensity() const = 0;
00059
00064         virtual const Color &getColor() const = 0;
00065
00070         virtual const Color &getAmbientComponent() const = 0;
00071
00076         virtual const Color &getDiffuseComponent() const = 0;
00077
00082         virtual const Color &getSpecularComponent() const = 0;
00083 };
```

7.21 include/interfaces/IPrimitive.hpp File Reference

IPrimitive class header file.

```
#include "Ray.hpp"
#include "Color.hpp"
#include "Define.hpp"
#include "Point.hpp"
#include "Vector.hpp"
#include "AMaterial.hpp"
```

Include dependency graph for IPrimitive.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class IPrimitive

  An interface for primitive shapes in the raytracer project.

7.21.1 Detailed Description

IPrimitive class header file.

This file contains the definition of the IPrimitive class, which is an interface for primitive shapes in the raytracer project.

Note

The IPrimitive class is used to define the interface for all primitive shapes.

7.22 IPrimitive.hpp

Go to the documentation of this file.

```
00001 /*
00002 ** EPITECH PROJECT, 2025
00003 ** Sans titre(Espace de travail)
00004 ** File description:
00005 ** IPrimitive
00006 */
00007
00015 #pragma once
00016
00017 #include "Ray.hpp"
00018 #include "Color.hpp"
00019 #include "Define.hpp"
00020 #include "Point.hpp"
00021 #include "Vector.hpp"
00022 #include "AMaterial.hpp"
00023
00028 class IPrimitive {
00029     public:
00034         virtual ~IPrimitive() = default;
00035
00040         virtual const std::string &getType() const = 0;
00041
00047         virtual bool intersect(const Ray &ray) const = 0;
00048
00054         virtual Math::Point getIntersection(const Ray &ray) const = 0;
00055
00061         virtual Math::Vector normal_at(const Math::Point &point) const = 0;
00062
00067         virtual Color getColor() const = 0;
00068
00073         virtual std::shared_ptr<AMaterial> getMaterial() const = 0;
00074 };
```

7.23 include/interfaces/ITransformations.hpp File Reference

ITransformations interface header file.

```
#include "Define.hpp"
#include "Point.hpp"
#include "Vector.hpp"
```

Include dependency graph for ITransformations.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class ITransformations

  An interface for transformations in the raytracer project.

7.23.1 Detailed Description

ITransformations interface header file.

This file contains the definition of the ITransformations interface, which is used to define the transformations of the objects in the raytracer project.

7.24 ITransformations.hpp

Go to the documentation of this file.

```
00001 /*
00002 ** EPITECH PROJECT, 2025
00003 ** Sans titre(Espace de travail)
00004 ** File description:
00005 ** ITransformations
00006 */
00007
00014 #pragma once
00015
00016 #include "Define.hpp"
00017 #include "Point.hpp"
00018 #include "Vector.hpp"
00019
00026 class ITransformations {
00027     public:
00032         virtual ~ITransformations() = default;
00033
00039         virtual void apply(Math::Point &origin, Math::Vector translation) = 0;
00040 };
```

7.25 include/loader/ObjParser.hpp File Reference

```
#include <string>
#include <vector>
#include <fstream>
#include <sstream>
#include <stdexcept>
#include <unordered_map>
#include "Point.hpp"
#include "IPrimitive.hpp"
#include "AMaterial.hpp"
```

Include dependency graph for ObjParser.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class ObjParser

  A class to parse .obj files and create shapes for the raytracer.

7.26 ObjParser.hpp

Go to the documentation of this file.

```
/*
** EPITECH PROJECT, 2025
** raytracer
** File description:
** ObjParser
*/

#pragma once

#include <string>
#include <vector>
#include <fstream>
#include <sstream>
#include <stdexcept>
#include <unordered_map>
#include "Point.hpp"
#include "IPrimitive.hpp"
#include "AMaterial.hpp"

class ObjParser {
    public:
        static std::vector<std::shared_ptr<IPrimitive» parse(const std::string &filepath);

        static std::unordered_map<std::string, AMaterial> loadMaterials(const std::string
&mtlFilePath);

        static std::unordered_map<std::string, Color> parseMaterial(const std::string &line);

    private:
        static Math::Point parseVertex(const std::string &line);
        static std::vector<int> parseFace(const std::string &line);
};
```

7.27 include/loader/SceneLoader.hpp File Reference

SceneLoader class header file.

```
#include "PluginsManager.hpp"
#include "Scene.hpp"
#include "Camera.hpp"
#include "Vector.hpp"
#include "Define.hpp"
```

Include dependency graph for SceneLoader.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class SceneLoader

  A class to load scenes from files in the raytracer project.

7.27.1 Detailed Description

SceneLoader class header file.

This file contains the definition of the SceneLoader class, which is used to load scenes from files in the raytracer project.

Note

The SceneLoader class is responsible for parsing scene files and creating the corresponding objects in the scene.

7.28 SceneLoader.hpp

Go to the documentation of this file.

```
00001 /*
00002 ** EPITECH PROJECT, 2025
00003 ** Sans titre(Espace de travail)
00004 ** File description:
00005 ** SceneLoader
00006 */
00007
00016 #pragma once
00017
00018 #include "PluginsManager.hpp"
00019 #include "Scene.hpp"
00020 #include "Camera.hpp"
00021 #include "Vector.hpp"
00022 #include "Define.hpp"
00023
00028 class SceneLoader {
00029     public:
00030
00036     static Scene parseSceneFile(const std::string &filename);
00037
00043     static void parseSceneCamera(const libconfig::Setting &file, Scene &scene);
00044
00050     static void parseScenePrimitive(const libconfig::Setting &file, Scene &scene);
00051
00057     static void parseSceneLight(const libconfig::Setting &file, Scene &scene);
00058
00059 };
```

7.29 include/materials/AMaterial.hpp File Reference

```
#include "Ray.hpp"
#include "Color.hpp"
```

Include dependency graph for AMaterial.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class AMaterial

7.30 AMaterial.hpp

Go to the documentation of this file.

```
/*
** EPITECH PROJECT, 2025
** raytracer
** File description:
** AMaterial
*/

#ifndef AMATERIAL_HPP_
#define AMATERIAL_HPP_

#include "Ray.hpp"
#include "Color.hpp"

class AMaterial {
    public:
        AMaterial(const std::string &name, float ambient, float diffuse, float specular, float shininess);

        virtual ~AMaterial() = default;

        // Getters
        const std::string &getName() const;
        float getAmbient() const;
        float getDiffuse() const;
        float getSpecular() const;
        float getShininess() const;
        float getTransparency() const;
        float getRefractionIndex() const;

        void setDiffuseColor(const Color &color);

        const Color &getDiffuseColor() const;

        void setSpecularColor(const Color &color);

        const Color &getSpecularColor() const;

        void setShininess(float shininess);

        virtual Color getColorAt(const Math::Point &point) const;

        void setTransparency(float transparency);

        void setRefractionIndex(float refractionIndex);

        void loadFromMtlLine(const std::string &line);

    private:
        std::string _name;
        float _ka;
        float _kd;
        float _ks;
        float _shininess;
        float _transparency = 0.0f; // 0 = opaque, 1 = fully transparent
        float _refractionIndex = 1.0f; // Default is air (1.0)
        Color _diffuseColor = Color(255, 255, 255); // Default white color
        Color _specularColor = Color(255, 255, 255); // Default white color
};

#endif // AMATERIAL_HPP_
```

7.31 include/materials/Chessboard.hpp File Reference

```
#include "AMaterial.hpp"
#include "Color.hpp"
#include "Define.hpp"
```

Include dependency graph for Chessboard.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class ChessboardMaterial

  Une classe pour représenter un matériau avec un pattern d'échiquier procédural.

7.32 Chessboard.hpp

Go to the documentation of this file.

```
/*
** EPITECH PROJECT, 2025
** raytracer
** File description:
** ChessboardMaterial
*/

#pragma once

#include "AMaterial.hpp"
#include "Color.hpp"
#include "Define.hpp"

class ChessboardMaterial : public AMaterial {
    public:
        ChessboardMaterial();

        ChessboardMaterial(const Color &color1, const Color &color2, double scale = 1.0);

        ~ChessboardMaterial() override = default;

        Color getColorAt(const Math::Point &point) const;

    private:
        Color _color1;
        Color _color2;
        double _scale;
};
```

7.33 include/materials/Default.hpp File Reference

`#include "AMaterial.hpp"`

Include dependency graph for Default.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class Default

7.34 Default.hpp

Go to the documentation of this file.

```
/*
** EPITECH PROJECT, 2025
** raytracer
** File description:
** Default
*/

#ifndef DEFAULT_HPP_
#define DEFAULT_HPP_

#include "AMaterial.hpp"

class Default : public AMaterial {
    public:
        Default();

};

#endif /* !DEFAULT_HPP_ */
```

7.35 include/materials/Metal.hpp File Reference

`#include "AMaterial.hpp"`
Include dependency graph for Metal.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class Metal

7.36 Metal.hpp

Go to the documentation of this file.

```
00001 /*
00002 ** EPITECH PROJECT, 2025
00003 ** raytracer
00004 ** File description:
00005 ** Metal
00006 */
00007
00008 #pragma once
00009
00010 #include "AMaterial.hpp"
00011
00012 class Metal : public AMaterial {
00013     public:
00014         Metal();
00015
00016 };
```

7.37 include/materials/Mtl.hpp File Reference

`#include "AMaterial.hpp"`
Include dependency graph for Mtl.hpp:

Classes

- class Mtl

7.38 Mtl.hpp

Go to the documentation of this file.

```
00001 /*
00002 ** EPITECH PROJECT, 2025
00003 ** raytracer
00004 ** File description:
00005 ** Mtl
00006 */
00007
00008 #include "AMaterial.hpp"
00009
00010 #ifndef MTL_HPP_
00011 #define MTL_HPP_
00012
00013 class Mtl : public AMaterial {
00014     public:
00018         Mtl(const std::string &name, float ambient, float diffuse, float specular, float shininess)
00019             : AMaterial(name, ambient, diffuse, specular, shininess) {}
00020
00024         ~Mtl() override = default;
00025 };
00026
00027 #endif /* !MTL_HPP_ */
```

7.39 include/materials/PerlinNoise.hpp File Reference

```
#include "AMaterial.hpp"
#include "Color.hpp"
#include "Define.hpp"
```

Include dependency graph for PerlinNoise.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class PerlinNoiseMaterial

7.40 PerlinNoise.hpp

Go to the documentation of this file.

```
/*
** EPITECH PROJECT, 2025
** B-OOP-400-LYN-4-1-raytracer-yanis.dolivet
** File description:
** PerlinNoise
*/

#pragma once

#include "AMaterial.hpp"
#include "Color.hpp"
#include "Define.hpp"

class PerlinNoiseMaterial : public AMaterial {
    public:
        PerlinNoiseMaterial();

        PerlinNoiseMaterial(const Color &color1, const Color &color2, double scale = 1.0);

        ~PerlinNoiseMaterial() override = default;

        Color getColorAt(const Math::Point &point) const override;

    private:
        Color _color1;
        Color _color2;
        double _scale;

        double noise(double x, double y) const;
        double fade(double t) const;
        double lerp(double t, double a, double b) const;
        double grad(int hash, double x, double y) const;
        std::vector<int> _p;
};
```

7.41 include/materials/Transparent.hpp File Reference

```
#include "AMaterial.hpp"
```

Include dependency graph for Transparent.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class Transparent

7.42 Transparent.hpp

Go to the documentation of this file.

```
/*
** EPITECH PROJECT, 2025
** raytracer
** File description:
** Transparent
*/

#include "AMaterial.hpp"

#ifndef TRANSPARENT_HPP_
#define TRANSPARENT_HPP_

class Transparent : public AMaterial {
    public:
        Transparent();
};

#endif /* !TRANSPARENT_HPP_ */
```

7.43 include/plugins/lights/AmbientLight.hpp File Reference

```
#include "PluginsManager.hpp"
#include "ILight.hpp"
```

Include dependency graph for AmbientLight.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class AmbientLight

  A class to represent an ambient light source in the raytracer project.

Functions

- std::shared_ptr< ILight > createLight (const libconfig::Setting &setting)

  Create an AmbientLight object.
- void registerPlugin ()

  Register the AmbientLight plugin.

7.43.1 Function Documentation

7.43.1.1 createLight()

```
std::shared_ptr< ILight > createLight (
            const libconfig::Setting & setting)
```

Create an AmbientLight object.

This function creates a new AmbientLight object.

Parameters

setting	The settings for the AmbientLight object.

Returns

A pointer to the created AmbientLight object.

7.43.1.2 registerPlugin()

```
void registerPlugin ()
```

Register the AmbientLight plugin.

This function registers the AmbientLight plugin with the PluginsManager.

7.44 AmbientLight.hpp

Go to the documentation of this file.

```
00001 /*
00002 ** EPITECH PROJECT, 2025
00003 ** Sans titre(Espace de travail)
00004 ** File description:
00005 ** AmbientLight
00006 */
00007
00015 #pragma once
00016
00017 #include "PluginsManager.hpp"
00018 #include "ILight.hpp"
00019
00024 class AmbientLight : public ILight {
00025     public:
00026
00031         AmbientLight(float intensity, const Color &color);
00032
00037         const std::string &getType() const override;
00038
00043         const Math::Point &getPosition() const override;
00044
00049         const Math::Vector &getDirection() const override;
00050
00055         float getIntensity() const override;
00056
00061         const Color &getColor() const override;
00062
00067         const Color &getAmbientComponent() const override;
00068
00073         const Color &getDiffuseComponent() const override;
00074
00079         const Color &getSpecularComponent() const override;
00080
00081     private:
00082         float _intensity;
00083         Color _ambient;
00084 };
00085
00086 extern "C" {
00087
00094     std::shared_ptr<ILight> createLight(const libconfig::Setting &setting)
00095     {
00096         try {
00097             if (!setting.exists("intensity") || !setting.exists("color"))
00098                 throw std::runtime_error("Missing required fields (intensity, color) in AmbientLight
      settings.");
00099             if (!setting["color"].isGroup())
00100                 throw std::runtime_error("Invalid types for field in AmbientLight settings");
00101
00102             const libconfig::Setting &colorSetting = setting["color"];
00103
00104             if (!colorSetting.exists("r") || !colorSetting.exists("g") ||
00105                 !colorSetting.exists("b") || !colorSetting["r"].isNumber() ||
00106                 !colorSetting["g"].isNumber() || !colorSetting["b"].isNumber())
00107                 throw std::runtime_error("Invalid or missing color fields (r, g, b) in AmbientLight
      settings.");
00108
00109             auto getFloatOrInt = [](const libconfig::Setting &s) -> float {
00110                 if (s.getType() == libconfig::Setting::TypeFloat)
00111                     return static_cast<float>(s);
00112                 if (s.getType() == libconfig::Setting::TypeInt)
00113                     return static_cast<float>(static_cast<int>(s));
00114                 throw std::runtime_error("AmbientLight: value is not a number");
00115             };
00116
00117             float intensity = getFloatOrInt(setting["intensity"]);
```

```
            Color color(
                static_cast<int>(colorSetting["r"]),
                static_cast<int>(colorSetting["g"]),
                static_cast<int>(colorSetting["b"])
            );

            return std::make_shared<AmbientLight>(intensity, color);
        } catch (const std::exception &e) {
            std::cerr « "[AmbientLight] Error: " « e.what() « std::endl;
            return nullptr;
        }
    }

    void registerPlugin()
    {
        PluginsManager::registerLightFactory("ambient", &createLight);
    }
};
```

7.45 include/plugins/lights/DirectionalLight.hpp File Reference

```
#include "PluginsManager.hpp"
#include "ILight.hpp"
```

Include dependency graph for DirectionalLight.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class DirectionalLight

  A class to represent a directional light source in the raytracer project.

Functions

- std::shared_ptr< ILight > createLight (const libconfig::Setting &setting)

  Create a DirectionalLight object.
- void registerPlugin ()

  Register the DirectionalLight plugin.

7.45.1 Function Documentation

7.45.1.1 createLight()

```
std::shared_ptr< ILight > createLight (
            const libconfig::Setting & setting)
```

Create a DirectionalLight object.

This function creates a new DirectionalLight object.

Parameters

setting	The settings for the DirectionalLight object.

Returns

A pointer to the created DirectionalLight object.

7.45.1.2 registerPlugin()

```
void registerPlugin ()
```

Register the DirectionalLight plugin.

This function registers the DirectionalLight plugin with the PluginsManager.

7.46 DirectionalLight.hpp

Go to the documentation of this file.

```
/*
** EPITECH PROJECT, 2025
** Sans titre(Espace de travail)
** File description:
** DirectionalLight
*/

#pragma once

#include "PluginsManager.hpp"
#include "ILight.hpp"

class DirectionalLight : public ILight {
    public:
        DirectionalLight(const Math::Vector &direction, const Color &color, float intensity);

        const std::string &getType() const override;

        const Math::Point &getPosition() const override;

        const Math::Vector &getDirection() const override;

        float getIntensity() const override;

        const Color &getColor() const override;

        const Color &getAmbientComponent() const override;

        const Color &getDiffuseComponent() const override;

        const Color &getSpecularComponent() const override;

    private:
        Math::Vector _direction;
        float _intensity;
        Color _color;
        Color _ambient;
        Color _diffuse;
        Color _specular;
};

extern "C" {

    std::shared_ptr<ILight> createLight(const libconfig::Setting &setting)
    {
        try {
            if (!setting.exists("x") || !setting.exists("y") ||
                !setting.exists("z") || !setting.exists("intensity") ||
                !setting.exists("color"))
                throw std::runtime_error("Missing required fields (x, y, z, intensity, color) in DirectionalLight settings.");
            if (!setting["color"].isGroup())
                throw std::runtime_error("Invalid types for field in DirectionalLight settings");

            const libconfig::Setting &colorSetting = setting["color"];

            if (!colorSetting.exists("r") || !colorSetting.exists("g") ||
                !colorSetting.exists("b") || !colorSetting["r"].isNumber() ||
                !colorSetting["g"].isNumber() || !colorSetting["b"].isNumber())
                throw std::runtime_error("Invalid or missing color fields (r, g, b) in DirectionalLight settings.");

            auto getFloatOrInt = [](const libconfig::Setting &s) -> float {
                if (s.getType() == libconfig::Setting::TypeFloat)
                    return static_cast<float>(s);
                if (s.getType() == libconfig::Setting::TypeInt)
```

```
                    return static_cast<float>(static_cast<int>(s));
                throw std::runtime_error("DirectionalLight: value is not a number");
            };

            Math::Vector direction(
                getFloatOrInt(setting["x"]),
                getFloatOrInt(setting["y"]),
                getFloatOrInt(setting["z"])
            );
            float intensity = getFloatOrInt(setting["intensity"]);
            Color color(
                static_cast<int>(colorSetting["r"]),
                static_cast<int>(colorSetting["g"]),
                static_cast<int>(colorSetting["b"])
            );

            return std::make_shared<DirectionalLight>(direction, color, intensity);
        } catch (const std::exception &e) {
            std::cerr « "[DirectionalLight] Error: " « e.what() « std::endl;
            return nullptr;
        }
    }

    void registerPlugin()
    {
        PluginsManager::registerLightFactory("directional", &createLight);
    }
}
```

7.47 include/plugins/lights/PointLight.hpp File Reference

```
#include "PluginsManager.hpp"
#include "ILight.hpp"
```

Include dependency graph for PointLight.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class PointLight

  A class to represent a point light source in the raytracer project.

Functions

- std::shared_ptr< ILight > createLight (const libconfig::Setting &setting)

  Create a PointLight object.
- void registerPlugin ()

  Register the PointLight plugin.

7.47.1 Function Documentation

7.47.1.1 createLight()

```
std::shared_ptr< ILight > createLight (
            const libconfig::Setting & setting)
```

Create a PointLight object.

This function creates a new PointLight object.

Parameters

file	The file to parse.

Returns

A pointer to the created PointLight object.

7.47.1.2 registerPlugin()

```
void registerPlugin ()
```

Register the PointLight plugin.

This function is called to register the PointLight plugin.

7.48 PointLight.hpp

Go to the documentation of this file.

```
00001 /*
00002 ** EPITECH PROJECT, 2025
00003 ** Sans titre(Espace de travail)
00004 ** File description:
00005 ** PointLight
00006 */
00007
00015 #pragma once
00016
00017 #include "PluginsManager.hpp"
00018 #include "ILight.hpp"
00019
00024 class PointLight : public ILight {
00025     public:
00032         PointLight(const Math::Vector &position, const Color &color, float intensity);
00033
00038         const std::string &getType() const override;
00039
00044         const Math::Point &getPosition() const override;
00045
00050         const Math::Vector &getDirection() const override;
00051
00056         float getIntensity() const override;
00057
00062         const Color &getColor() const override;
00063
00068         const Color &getAmbientComponent() const override;
00069
00074         const Color &getDiffuseComponent() const override;
00075
00080         const Color &getSpecularComponent() const override;
00081
00082     private:
00083         Math::Point _position;
00084         float _intensity;
00085         Color _color;
00086         Color _ambient;
00087         Color _diffuse;
00088         Color _specular;
00089 };
00090
00091 extern "C" {
00092
00099     std::shared_ptr<ILight> createLight(const libconfig::Setting &setting)
00100     {
00101         try {
00102             if (!setting.exists("x") || !setting.exists("y") ||
00103                 !setting.exists("z") || !setting.exists("intensity") ||
00104                 !setting.exists("color"))
```

```
                throw std::runtime_error("Missing required fields (x, y, z, intensity, color) in PointLight settings.");
            if (!setting["color"].isGroup())
                throw std::runtime_error("PointLight: color is not a group");

            const libconfig::Setting &colorSetting = setting["color"];

            if (!colorSetting.exists("r") || !colorSetting.exists("g") ||
                !colorSetting.exists("b") || !colorSetting["r"].isNumber() ||
                !colorSetting["g"].isNumber() || !colorSetting["b"].isNumber())
                throw std::runtime_error("PointLight: r, g, b is not a number");

            auto getFloatOrInt = [](const libconfig::Setting &s) -> float {
                if (s.getType() == libconfig::Setting::TypeFloat)
                    return static_cast<float>(s);
                if (s.getType() == libconfig::Setting::TypeInt)
                    return static_cast<float>(static_cast<int>(s));
                throw std::runtime_error("PointLight: value is not a number");
            };

            Math::Vector position(
                getFloatOrInt(setting["x"]),
                getFloatOrInt(setting["y"]),
                getFloatOrInt(setting["z"])
            );
            float intensity = getFloatOrInt(setting["intensity"]);
            Color color(
                static_cast<int>(colorSetting["r"]),
                static_cast<int>(colorSetting["g"]),
                static_cast<int>(colorSetting["b"])
            );
            return std::make_shared<PointLight>(position, color, intensity);
        } catch (const std::exception &e) {
            std::cerr « "[PointLight] Error: " « e.what() « std::endl;
            return nullptr;
        }
    }

    void registerPlugin()
    {
        PluginsManager::registerLightFactory("point", &createLight);
    }
};
```

7.49 include/plugins/PluginsFactory.hpp File Reference

File that declares the factory type for the plugins.

```
#include "ILight.hpp"
#include "IPrimitive.hpp"
#include "Define.hpp"
```

Include dependency graph for PluginsFactory.hpp: This graph shows which files directly or indirectly include this file:

Typedefs

- using PrimitivesFactory = std::function<std::shared_ptr<IPrimitive>(const libconfig::Setting &)>

  Type definition for a factory that creates IPrimitive objects.
- using LightsFactory = std::function<std::shared_ptr<ILight>(const libconfig::Setting &)>

  Type definition for a factory that creates ILight objects.

7.49.1 Detailed Description

File that declares the factory type for the plugins.

Note

This file is part of the Raytracer project.

7.49.2 Typedef Documentation

7.49.2.1 LightsFactory

```
using LightsFactory = std::function<std::shared_ptr<ILight>(const libconfig::Setting &)>
```

Type definition for a factory that creates ILight objects.

This using is a definition for a function that takes a libconfig::Setting object and returns a pointer to an ILight object.

Note

The function is expected to be used as a factory for creating ILight objects.

7.49.2.2 PrimitivesFactory

```
using PrimitivesFactory = std::function<std::shared_ptr<IPrimitive>(const libconfig::Setting
&)>
```

Type definition for a factory that creates IPrimitive objects.

This using is a definition for a function that takes a libconfig::Setting object and returns a pointer to an IPrimitive object.

Note

The function is expected to be used as a factory for creating IPrimitive objects.

7.50 PluginsFactory.hpp

Go to the documentation of this file.

```
/*
** EPITECH PROJECT, 2025
** Sans titre(Espace de travail)
** File description:
** PluginsFactory
*/

#pragma once

#include "ILight.hpp"
#include "IPrimitive.hpp"
#include "Define.hpp"

using PrimitivesFactory = std::function<std::shared_ptr<IPrimitive>(const libconfig::Setting &)>;
using LightsFactory = std::function<std::shared_ptr<ILight>(const libconfig::Setting &)>;
```

7.51 include/plugins/PluginsHandle.hpp File Reference

Header file for the PluginsHandle class.

`#include "Define.hpp"`

Include dependency graph for PluginsHandle.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class PluginsHandle

  A class to manage plugins in the raytracer project.

7.51.1 Detailed Description

Header file for the PluginsHandle class.

This file contains the definition of the PluginsHandle class, which is used to manage plugins in the raytracer project.

Note

The PluginsHandle class is responsible for loading and managing plugins that extend the functionality of the raytracer.

7.52 PluginsHandle.hpp

Go to the documentation of this file.

```
00001 /*
00002 ** EPITECH PROJECT, 2025
00003 ** Sans titre(Espace de travail)
00004 ** File description:
00005 ** PluginsHandle
00006 */
00007
00016 #pragma once
00017
00018 #include "Define.hpp"
00019
00024 class PluginsHandle {
00025     public:
00030         explicit PluginsHandle(const std::string &path);
00031
00036         ~PluginsHandle();
00037
00042         PluginsHandle(const PluginsHandle &) = delete;
00043
00048         PluginsHandle &operator=(const PluginsHandle &) = delete;
00049
00053         PluginsHandle(PluginsHandle &&) noexcept;
00054
00060         PluginsHandle &operator=(PluginsHandle &&) noexcept;
00061
00069         template <typename T>
00070         T getSymbol(const std::string &symbolName)
00071         {
00072             T symbol = reinterpret_cast<T>(dlsym(_handle.get(), symbolName.c_str()));
00073             if (!symbol) {
00074                 throw std::runtime_error("Failed to load symbol: " + symbolName);
00075             }
00076             return symbol;
00077         }
00078
00079     private:
00080
00085         static void closeHandle(void* handle);
00086         std::unique_ptr<void, int(*)(void*)> _handle{nullptr, dlclose};
00087 };
```

7.53 include/plugins/PluginsManager.hpp File Reference

PluginsManager class header file.

```
#include "Exception.hpp"
#include "PluginsFactory.hpp"
#include "PluginsHandle.hpp"
#include "Define.hpp"
```

Include dependency graph for PluginsManager.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class PluginsManager

  A class to manage plugins in the raytracer project.

7.53.1 Detailed Description

PluginsManager class header file.

This file contains the definition of the PluginsManager class, which is used to manage plugins in the raytracer project and load the .so.

Note

The PluginsManager class is responsible for loading and managing plugins that extend the functionality of the raytracer.

7.54 PluginsManager.hpp

Go to the documentation of this file.

```
00001 /*
00002 ** EPITECH PROJECT, 2025
00003 ** Sans titre(Espace de travail)
00004 ** File description:
00005 ** PluginsManager
00006 */
00007
00016 #pragma once
00017
00018 #include "Exception.hpp"
00019 #include "PluginsFactory.hpp"
00020 #include "PluginsHandle.hpp"
00021 #include "Define.hpp"
00022
00027 class PluginsManager {
00028     public:
00029
00034         void loadPlugins(const std::string &path);
00035
00041         template<typename T>
00042         T getSymbol(const std::string &symbol)
00043         {
00044             auto raw = _plugins.back().getSymbol<T>(symbol);
00045             if (!raw)
00046                 throw PluginsException("Failed to load symbol: " + symbol);
00047             return raw;
00048         }
00049
00055         static void registerPrimitiveFactory(const std::string &type, PrimitivesFactory factory);
00056
00062         static void registerLightFactory(const std::string &type, LightsFactory factory);
00063
00070         static std::shared_ptr<IPrimitive> createPrimitive(const std::string &name, const
      libconfig::Setting &setting);
00071
00078         static std::shared_ptr<ILight> createLight(const std::string &name, const libconfig::Setting
      &setting);
00079
00083         static void unregisterAllFactories();
00084
00085     private:
00086         static inline std::map<std::string, PrimitivesFactory> _primitiveFactories;
00087         static inline std::map<std::string, LightsFactory> _lightFactories;
00088
00089         static inline std::vector <PluginsHandle> _plugins;
00090 };
```

7.55 include/plugins/primitives/Cone.hpp File Reference

```
#include "PluginsManager.hpp"
#include "IPrimitive.hpp"
#include "AMaterial.hpp"
#include "Default.hpp"
#include "Metal.hpp"
#include "Chessboard.hpp"
#include "PerlinNoise.hpp"
```

Include dependency graph for Cone.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class Cone

Functions

- std::shared_ptr< IPrimitive > createPrimitive (const libconfig::Setting &setting)

  Create a Cone object.
- void registerPlugin ()

  Register the Plane plugin.

7.55.1 Function Documentation

7.55.1.1 createPrimitive()

```
std::shared_ptr< IPrimitive > createPrimitive (
            const libconfig::Setting & setting)
```

Create a Cone object.

This function creates a new Cone object.

Parameters

setting	The settings for the Cone object.

Returns

A pointer to the created Cone object.

7.55.1.2 registerPlugin()

```
void registerPlugin ()
```

Register the Plane plugin.

This function is called to register the Plane plugin.

7.56 Cone.hpp

Go to the documentation of this file.

```
/*
** EPITECH PROJECT, 2025
** raytracer
** File description:
** Cone
*/


#pragma once

#include "PluginsManager.hpp"
#include "IPrimitive.hpp"
#include "AMaterial.hpp"
#include "Default.hpp"
#include "Metal.hpp"
#include "Chessboard.hpp"
#include "PerlinNoise.hpp"


class Cone : public IPrimitive {
    public:
        Cone();

        Cone(Math::Point tip, Math::Vector height, double angle, Color &color,
            const std::shared_ptr<AMaterial> &mat);

        ~Cone();

        const std::string &getType() const override;

        bool intersect_base(const Ray &ray) const;

        bool intersectCone(const Ray &ray) const;

        bool intersect(const Ray &ray) const override;

        Math::Point getIntersectBase(const Ray &ray) const;

        Math::Point getIntersectCone(const Ray &ray) const;

        Math::Point getIntersection(const Ray &ray) const override;

        Math::Vector normal_at(const Math::Point &point) const override;

        Color getColor() const override;

        std::shared_ptr<AMaterial> getMaterial() const override;

    protected:
    private:
        Math::Point _tip;
        Math::Vector _height;
        double _radiant;
        Color _color;
        std::shared_ptr<AMaterial> _material;

};

extern "C" {

    std::shared_ptr<IPrimitive> createPrimitive(const libconfig::Setting &setting)
    {
        try {
            if (!setting.exists("x") || !setting.exists("y") ||
                !setting.exists("z") || !setting.exists("height") ||
                !setting.exists("angle")  || !setting.exists("color"))
                throw std::runtime_error("Missing required fields (x, y, z, height, angle, color) in Cone settings.");
            if (!setting["color"].isGroup())
                throw std::runtime_error("Cone: color is not a group");

            const libconfig::Setting &heightSetting = setting["height"];
            const libconfig::Setting &colorSetting = setting["color"];

            if (!heightSetting.exists("x") || !heightSetting.exists("y") ||
                !heightSetting.exists("z") || !heightSetting["x"].isNumber() ||
                !heightSetting["y"].isNumber() || !heightSetting["z"].isNumber())
                throw std::runtime_error("Invalid or missing height fields (x, y, z) in Cone settings.");

            if (!colorSetting.exists("r") || !colorSetting.exists("g") ||
                !colorSetting.exists("b") || !colorSetting["r"].isNumber() ||
```

```
                !colorSetting["g"].isNumber() || !colorSetting["b"].isNumber())
                throw std::runtime_error("Invalid or missing color fields (r, g, b) in Cone settings.");

            auto getFloatOrInt = [](const libconfig::Setting &s) -> float {
                if (s.getType() == libconfig::Setting::TypeFloat)
                    return static_cast<float>(s);
                if (s.getType() == libconfig::Setting::TypeInt)
                    return static_cast<float>(static_cast<int>(s));
                throw std::runtime_error("Cone: value is not a number");
            };

            Math::Point position(
                getFloatOrInt(setting["x"]),
                getFloatOrInt(setting["y"]),
                getFloatOrInt(setting["z"])
            );
            Math::Vector height(
                getFloatOrInt(heightSetting["x"]),
                getFloatOrInt(heightSetting["y"]),
                getFloatOrInt(heightSetting["z"])
            );
            double angle = getFloatOrInt(setting["angle"]);
            Color color(
                static_cast<int>(colorSetting["r"]),
                static_cast<int>(colorSetting["g"]),
                static_cast<int>(colorSetting["b"])
            );

            std::string material = setting["material"].c_str();
            std::shared_ptr<AMaterial> mat;
            if (material == "Default") {
                mat = std::make_shared<Default>();
            } else if (material == "Metal") {
                mat = std::make_shared<Metal>();
            } else if (material == "Chessboard") {
                mat = std::make_shared<ChessboardMaterial>();
            } else if (material == "PerlinNoise") {
                mat = std::make_shared<PerlinNoiseMaterial>();
            } else {
                throw std::runtime_error("Cone: Unknown material type");
            }
            return std::make_shared<Cone>(position, height, angle, color, mat);
        } catch (const std::exception &e) {
            std::cerr « e.what() « std::endl;
            return nullptr;
        }
    }


    void registerPlugin()
    {
        PluginsManager::registerPrimitiveFactory("cone", &createPrimitive);
    }
};

```

7.57 include/plugins/primitives/Cylindre.hpp File Reference

```
#include "PluginsManager.hpp"
#include "IPrimitive.hpp"
#include "AMaterial.hpp"
#include "Default.hpp"
#include "Metal.hpp"
#include "Chessboard.hpp"
#include "PerlinNoise.hpp"
```

Include dependency graph for Cylindre.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class Cylindre

Functions

- std::shared_ptr< IPrimitive > createPrimitive (const libconfig::Setting &setting)

  Create a Cylindre object.
- void registerPlugin ()

  Register the Plane plugin.

7.57.1 Function Documentation

7.57.1.1 createPrimitive()

```
std::shared_ptr< IPrimitive > createPrimitive (
            const libconfig::Setting & setting)
```

Create a Cylindre object.

This function creates a new Cylindre object.

Parameters

setting	The settings for the Cylindre object.

Returns

A pointer to the created Cylindre object.

7.57.1.2 registerPlugin()

```
void registerPlugin ()
```

Register the Plane plugin.

This function is called to register the Plane plugin.

7.58 Cylindre.hpp

Go to the documentation of this file.

```
00001 /*
00002 ** EPITECH PROJECT, 2025
00003 ** raytracer
00004 ** File description:
00005 ** Cylindre
00006 */
00007
00015
00016 #pragma once
00017
00018 #include "PluginsManager.hpp"
00019 #include "IPrimitive.hpp"
00020 #include "AMaterial.hpp"
00021 #include "Default.hpp"
00022 #include "Metal.hpp"
00023 #include "Chessboard.hpp"
00024 #include "PerlinNoise.hpp"
00025
```

```
class Cylindre : public IPrimitive {
    public:
        Cylindre();

        Cylindre(Math::Point center, double radius, double height, Color &color,
            const std::shared_ptr<AMaterial> &mat);

        ~Cylindre();

        const std::string &getType() const override;

        bool intersect(const Ray &ray) const override;

        Math::Point intersection_cap(const Ray &ray, const Math::Point &cap_center, const Math::Vector
&normal) const;

        Math::Point getIntersection(const Ray &ray) const override;

        Math::Vector normal_at(const Math::Point &point) const override;

        Color getColor() const override;

        std::shared_ptr<AMaterial> getMaterial() const override;

    protected:
    private:
        Math::Point _center;
        double _radius;
        double _height;
        Color _color;
        std::shared_ptr<AMaterial> _material;

};

extern "C" {

    std::shared_ptr<IPrimitive> createPrimitive(const libconfig::Setting &setting)
    {
        try {
            if (!setting.exists("x") || !setting.exists("y") ||
                !setting.exists("z") || !setting.exists("height") ||
                !setting.exists("angle")  || !setting.exists("color"))
                throw std::runtime_error("Missing required fields (x, y, z, radius, color) in Cylindre
settings.");
            if (!setting["color"].isGroup())
                throw std::runtime_error("Cylindre: color is not a group");

            const libconfig::Setting &colorSetting = setting["color"];

            if (!colorSetting.exists("r") || !colorSetting.exists("g") ||
                !colorSetting.exists("b") || !colorSetting["r"].isNumber() ||
                !colorSetting["g"].isNumber() || !colorSetting["b"].isNumber())
                throw std::runtime_error("Invalid or missing color fields (r, g, b) in Cylindre
settings.");

            auto getFloatOrInt = [](const libconfig::Setting &s) -> float {
                if (s.getType() == libconfig::Setting::TypeFloat)
                    return static_cast<float>(s);
                if (s.getType() == libconfig::Setting::TypeInt)
                    return static_cast<float>(static_cast<int>(s));
                throw std::runtime_error("Cylindre: value is not a number");
            };

            Math::Point position(
                getFloatOrInt(setting["x"]),
                getFloatOrInt(setting["y"]),
                getFloatOrInt(setting["z"])
            );
            double height = getFloatOrInt(setting["height"]);
            double angle = getFloatOrInt(setting["angle"]);
            Color color(
                static_cast<int>(colorSetting["r"]),
                static_cast<int>(colorSetting["g"]),
                static_cast<int>(colorSetting["b"])
            );
            std::string material = setting["material"].c_str();
            std::shared_ptr<AMaterial> mat;
            if (material == "Default") {
                mat = std::make_shared<Default>();
            } else if (material == "Metal") {
                mat = std::make_shared<Metal>();
            } else if (material == "Chessboard") {
                mat = std::make_shared<ChessboardMaterial>();
            } else if (material == "PerlinNoise") {
                mat = std::make_shared<PerlinNoiseMaterial>();
            } else {
                throw std::runtime_error("Cylinder: Unknown material type");
```

```
            }
            return std::make_shared<Cylindre>(position, angle, height, color, mat);
        } catch (const std::exception &e) {
            std::cerr « e.what() « std::endl;
            return nullptr;
        }
    }


    void registerPlugin()
    {
        PluginsManager::registerPrimitiveFactory("cylindre", &createPrimitive);
    }
};

```

7.59 include/plugins/primitives/InfinitCone.hpp File Reference

```
#include "PluginsManager.hpp"
#include "IPrimitive.hpp"
#include "AMaterial.hpp"
#include "Default.hpp"
#include "Metal.hpp"
#include "Chessboard.hpp"
#include "PerlinNoise.hpp"
```

Include dependency graph for InfinitCone.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class InfinitCone

Functions

- std::shared_ptr< IPrimitive > createPrimitive (const libconfig::Setting &setting)

  Create a InfinitCone object.
- void registerPlugin ()

  Register the Plane plugin.

7.59.1 Function Documentation

7.59.1.1 createPrimitive()

```
std::shared_ptr< IPrimitive > createPrimitive (
            const libconfig::Setting & setting)
```

Create a InfinitCone object.

This function creates a new InfinitCone object.

Parameters

setting	The settings for the InfinitCone object.

Returns

A pointer to the created InfinitCone object.

7.59.1.2 registerPlugin()

```
void registerPlugin ()
```

Register the Plane plugin.

This function is called to register the Plane plugin.

7.60 InfinitCone.hpp

Go to the documentation of this file.

```
00001 /*
00002 ** EPITECH PROJECT, 2025
00003 ** raytracer
00004 ** File description:
00005 ** InfinitCone
00006 */
00007
00015
00016 #pragma once
00017
00018 #include "PluginsManager.hpp"
00019 #include "IPrimitive.hpp"
00020 #include "AMaterial.hpp"
00021 #include "Default.hpp"
00022 #include "Metal.hpp"
00023 #include "Chessboard.hpp"
00024 #include "PerlinNoise.hpp"
00025
00026
00031 class InfinitCone : public IPrimitive {
00032     public:
00037         InfinitCone();
00038
00050         InfinitCone(Math::Point tip, Math::Vector height, double angle, Color &color,
00051             const std::shared_ptr<AMaterial> &mat);
00052
00056         ~InfinitCone();
00057
00062         const std::string &getType() const override;
00063
00069         bool intersect(const Ray &ray) const override;
00070
00076         Math::Point getIntersection(const Ray &ray) const override;
00077
00083         Math::Vector normal_at(const Math::Point &point) const override;
00084
00089         Color getColor() const override;
00090
00095         std::shared_ptr<AMaterial> getMaterial() const override;
00096
00097     protected:
00098     private:
00099         Math::Point _tip;
00100         Math::Vector _height;
00101         double _radiant;
00102         Color _color;
00103         std::shared_ptr<AMaterial> _material;
00104
00105 };
00106
00107 extern "C" {
00108
00115     std::shared_ptr<IPrimitive> createPrimitive(const libconfig::Setting &setting)
00116     {
00117         try {
00118             if (!setting.exists("x") || !setting.exists("y") ||
00119                 !setting.exists("z") || !setting.exists("height") ||
00120                 !setting.exists("angle")  || !setting.exists("color"))
00121                 throw std::runtime_error("Missing required fields (x, y, z, height, angle, color) in
      InfinitCone settings.");
00122             if (!setting["color"].isGroup())
00123                 throw std::runtime_error("InfinitCone: color is not a group");
00124
00125             const libconfig::Setting &heightSetting = setting["height"];
00126             const libconfig::Setting &colorSetting = setting["color"];
00127
```

```
            if (!heightSetting.exists("x") || !heightSetting.exists("y") ||
                !heightSetting.exists("z") || !heightSetting["x"].isNumber() ||
                !heightSetting["y"].isNumber() || !heightSetting["z"].isNumber())
                throw std::runtime_error("Invalid or missing height fields (x, y, z) in InfinitCone settings.");

            if (!colorSetting.exists("r") || !colorSetting.exists("g") ||
                !colorSetting.exists("b") || !colorSetting["r"].isNumber() ||
                !colorSetting["g"].isNumber() || !colorSetting["b"].isNumber())
                throw std::runtime_error("Invalid or missing color fields (r, g, b) in InfinitCone settings.");

            auto getFloatOrInt = [](const libconfig::Setting &s) -> float {
                if (s.getType() == libconfig::Setting::TypeFloat)
                    return static_cast<float>(s);
                if (s.getType() == libconfig::Setting::TypeInt)
                    return static_cast<float>(static_cast<int>(s));
                throw std::runtime_error("InfinitCone: value is not a number");
            };

            Math::Point position(
                getFloatOrInt(setting["x"]),
                getFloatOrInt(setting["y"]),
                getFloatOrInt(setting["z"])
            );
            Math::Vector height(
                getFloatOrInt(heightSetting["x"]),
                getFloatOrInt(heightSetting["y"]),
                getFloatOrInt(heightSetting["z"])
            );
            double angle = getFloatOrInt(setting["angle"]);
            Color color(
                static_cast<int>(colorSetting["r"]),
                static_cast<int>(colorSetting["g"]),
                static_cast<int>(colorSetting["b"])
            );

            std::string material = setting["material"].c_str();
            std::shared_ptr<AMaterial> mat;
            if (material == "Default") {
                mat = std::make_shared<Default>();
            } else if (material == "Metal") {
                mat = std::make_shared<Metal>();
            } else if (material == "Chessboard") {
                mat = std::make_shared<ChessboardMaterial>();
            } else if (material == "PerlinNoise") {
                mat = std::make_shared<PerlinNoiseMaterial>();
            } else {
                throw std::runtime_error("InfinitCone: Unknown material type");
            }
            return std::make_shared<InfinitCone>(position, height, angle, color, mat);
        } catch (const std::exception &e) {
            std::cerr « e.what() « std::endl;
            return nullptr;
        }
    }


    void registerPlugin()
    {
        PluginsManager::registerPrimitiveFactory("infinite_cone", &createPrimitive);
    }
};

```

7.61 include/plugins/primitives/InfiniteCylindre.hpp File Reference

```
#include "PluginsManager.hpp"
#include "IPrimitive.hpp"
#include "AMaterial.hpp"
#include "Default.hpp"
#include "Metal.hpp"
#include "Chessboard.hpp"
#include "PerlinNoise.hpp"
```

Include dependency graph for InfiniteCylindre.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class InfiniteCylindre

Functions

- std::shared_ptr< IPrimitive > createPrimitive (const libconfig::Setting &setting)

  Create a InfiniteCylindre object.
- void registerPlugin ()

  Register the Plane plugin.

7.61.1 Function Documentation

7.61.1.1 createPrimitive()

```
std::shared_ptr< IPrimitive > createPrimitive (
            const libconfig::Setting & setting)
```

Create a InfiniteCylindre object.

This function creates a new InfiniteCylindre object.

Parameters

setting	The settings for the InfiniteCylindre object.

Returns

A pointer to the created InfiniteCylindre object.

7.61.1.2 registerPlugin()

```
void registerPlugin ()
```

Register the Plane plugin.

This function is called to register the Plane plugin.

7.62 InfiniteCylindre.hpp

Go to the documentation of this file.

```
/*
** EPITECH PROJECT, 2025
** raytracer
** File description:
** InfiniteCylindre
*/


#pragma once

#include "PluginsManager.hpp"
#include "IPrimitive.hpp"
#include "AMaterial.hpp"
#include "Default.hpp"
#include "Metal.hpp"
#include "Chessboard.hpp"
#include "PerlinNoise.hpp"

class InfiniteCylindre : public IPrimitive {
    public:
        InfiniteCylindre();

        InfiniteCylindre(Math::Point center, double radius, double height, Color &color,
            const std::shared_ptr<AMaterial> &mat);

        ~InfiniteCylindre();

        const std::string &getType() const override;

        bool intersect(const Ray &ray) const override;

        Math::Point getIntersection(const Ray &ray) const override;

        Math::Vector normal_at(const Math::Point &point) const override;

        Color getColor() const override;

        std::shared_ptr<AMaterial> getMaterial() const override;

    protected:
    private:
        Math::Point _center;
        double _radius;
        double _height;
        Color _color;
        std::shared_ptr<AMaterial> _material;

};

extern "C" {

    std::shared_ptr<IPrimitive> createPrimitive(const libconfig::Setting &setting)
    {
        try {
            if (!setting.exists("x") || !setting.exists("y") ||
                !setting.exists("z") || !setting.exists("height") ||
                !setting.exists("angle")  || !setting.exists("color"))
                throw std::runtime_error("Missing required fields (x, y, z, radius, color) in
InfiniteCylindre settings.");
            if (!setting["color"].isGroup())
                throw std::runtime_error("InfiniteCylindre: color is not a group");

            const libconfig::Setting &colorSetting = setting["color"];

            if (!colorSetting.exists("r") || !colorSetting.exists("g") ||
                !colorSetting.exists("b") || !colorSetting["r"].isNumber() ||
                !colorSetting["g"].isNumber() || !colorSetting["b"].isNumber())
                throw std::runtime_error("Invalid or missing color fields (r, g, b) in
InfiniteCylindre settings.");

            auto getFloatOrInt = [](const libconfig::Setting &s) -> float {
                if (s.getType() == libconfig::Setting::TypeFloat)
                    return static_cast<float>(s);
                if (s.getType() == libconfig::Setting::TypeInt)
                    return static_cast<float>(static_cast<int>(s));
                throw std::runtime_error("InfiniteCylindre: value is not a number");
            };

            Math::Point position(
                getFloatOrInt(setting["x"]),
                getFloatOrInt(setting["y"]),
                getFloatOrInt(setting["z"])
```

```
            );
            double height = getFloatOrInt(setting["height"]);
            double angle = getFloatOrInt(setting["angle"]);
            Color color(
                static_cast<int>(colorSetting["r"]),
                static_cast<int>(colorSetting["g"]),
                static_cast<int>(colorSetting["b"])
            );
            std::string material = setting["material"].c_str();
            std::shared_ptr<AMaterial> mat;
            if (material == "Default") {
                mat = std::make_shared<Default>();
            } else if (material == "Metal") {
                mat = std::make_shared<Metal>();
            } else if (material == "Chessboard") {
                mat = std::make_shared<ChessboardMaterial>();
            } else if (material == "PerlinNoise") {
                mat = std::make_shared<PerlinNoiseMaterial>();
            } else {
                throw std::runtime_error("Cylinder: Unknown material type");
            }
            return std::make_shared<InfiniteCylindre>(position, angle, height, color, mat);
        } catch (const std::exception &e) {
            std::cerr « e.what() « std::endl;
            return nullptr;
        }
    }


    void registerPlugin()
    {
        PluginsManager::registerPrimitiveFactory("infinite_cylindre", &createPrimitive);
    }
};

```

7.63 include/plugins/primitives/Plane.hpp File Reference

```
#include "PluginsManager.hpp"
#include "IPrimitive.hpp"
#include "Point.hpp"
#include "AMaterial.hpp"
#include "Default.hpp"
#include "Metal.hpp"
#include "Chessboard.hpp"
#include "PerlinNoise.hpp"
```

Include dependency graph for Plane.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class Plane

  A class to represent a plane in 3D space.

Functions

- std::shared_ptr< IPrimitive > createPrimitive (const libconfig::Setting &setting)

  Create a Plane object.
- void registerPlugin ()

  Register the Plane plugin.

7.63.1 Function Documentation

7.63.1.1 createPrimitive()

```
std::shared_ptr< IPrimitive > createPrimitive (
            const libconfig::Setting & setting)
```

Create a Plane object.

This function creates a new Plane object.

Parameters

setting	The settings for the Plane object.

Returns

A pointer to the created Plane object.

7.63.1.2 registerPlugin()

```
void registerPlugin ()
```

Register the Plane plugin.

This function is called to register the Plane plugin.

7.64 Plane.hpp

Go to the documentation of this file.

```
00001 /*
00002 ** EPITECH PROJECT, 2025
00003 ** Sans titre(Espace de travail)
00004 ** File description:
00005 ** Plane
00006 */
00007
00015 #pragma once
00016
00017 #include "PluginsManager.hpp"
00018 #include "IPrimitive.hpp"
00019 #include "Point.hpp"
00020 #include "AMaterial.hpp"
00021 #include "Default.hpp"
00022 #include "Metal.hpp"
00023 #include "Chessboard.hpp"
00024 #include "PerlinNoise.hpp"
00025
00030 class Plane : public IPrimitive {
00031     public:
00032
00039         Plane(Math::Point p0, Math::Point p1, Math::Point p2, const Color &color, const
      std::shared_ptr<AMaterial> &mat);
00040
00045         ~Plane() override = default;
00046
00051         const std::string &getType() const override;
00052
00058         bool intersect(const Ray &ray) const override;
00059
00065         Math::Point getIntersection(const Ray &ray) const override;
```

```

        Math::Vector normal_at(const Math::Point &point) const override;

        Color getColor() const override;

        std::shared_ptr<AMaterial> getMaterial() const override;

    private:
        std::string _axis;
        float _position;
        Math::Point _a, _b, _c;
        Math::Vector _normal;
        Color _color;
        std::shared_ptr<AMaterial> _material;
};

extern "C" {

    std::shared_ptr<IPrimitive> createPrimitive(const libconfig::Setting &setting)
    {
        try {
            if (!setting.exists("p0") || !setting.exists("p1") || !setting.exists("p2") ||
                !setting.exists("color")) {
                throw std::runtime_error("[Plane] Missing required fields (p0, p1, p2, color) in Plane settings.");
            }
            if (!setting["color"].isGroup()) {
                throw std::runtime_error("[Plane] Plane: color is not a group");
            }

            const libconfig::Setting &p0Setting = setting["p0"];
            const libconfig::Setting &p1Setting = setting["p1"];
            const libconfig::Setting &p2Setting = setting["p2"];
            const libconfig::Setting &colorSetting = setting["color"];

            if (!p0Setting.exists("x") || !p0Setting.exists("y") ||
                !p0Setting.exists("z") || !p0Setting["x"].isNumber() ||
                !p0Setting["y"].isNumber() || !p0Setting["z"].isNumber()) {
                throw std::runtime_error("[Plane] Invalid or missing p0 fields (x, y, z) in Plane settings.");
            }

            if (!p1Setting.exists("x") || !p1Setting.exists("y") ||
                !p1Setting.exists("z") || !p1Setting["x"].isNumber() ||
                !p1Setting["y"].isNumber() || !p1Setting["z"].isNumber()) {
                throw std::runtime_error("[Plane] Invalid or missing p1 fields (x, y, z) in Plane settings.");
            }

            if (!p2Setting.exists("x") || !p2Setting.exists("y") ||
                !p2Setting.exists("z") || !p2Setting["x"].isNumber() ||
                !p2Setting["y"].isNumber() || !p2Setting["z"].isNumber()) {
                throw std::runtime_error("[Plane] Invalid or missing p2 fields (x, y, z) in Plane settings.");
            }

            if (!colorSetting.exists("r") || !colorSetting.exists("g") ||
                !colorSetting.exists("b") || !colorSetting["r"].isNumber() ||
                !colorSetting["g"].isNumber() || !colorSetting["b"].isNumber()) {
                throw std::runtime_error("[Plane] Invalid or missing color fields (r, g, b) in Plane settings.");
            }

            float p0x = (p0Setting["x"].getType() == libconfig::Setting::TypeFloat) ? (float)p0Setting["x"] : (float)(int)p0Setting["x"];
            float p0y = (p0Setting["y"].getType() == libconfig::Setting::TypeFloat) ? (float)p0Setting["y"] : (float)(int)p0Setting["y"];
            float p0z = (p0Setting["z"].getType() == libconfig::Setting::TypeFloat) ? (float)p0Setting["z"] : (float)(int)p0Setting["z"];
            float p1x = (p1Setting["x"].getType() == libconfig::Setting::TypeFloat) ? (float)p1Setting["x"] : (float)(int)p1Setting["x"];
            float p1y = (p1Setting["y"].getType() == libconfig::Setting::TypeFloat) ? (float)p1Setting["y"] : (float)(int)p1Setting["y"];
            float p1z = (p1Setting["z"].getType() == libconfig::Setting::TypeFloat) ? (float)p1Setting["z"] : (float)(int)p1Setting["z"];
            float p2x = (p2Setting["x"].getType() == libconfig::Setting::TypeFloat) ? (float)p2Setting["x"] : (float)(int)p2Setting["x"];
            float p2y = (p2Setting["y"].getType() == libconfig::Setting::TypeFloat) ? (float)p2Setting["y"] : (float)(int)p2Setting["y"];
            float p2z = (p2Setting["z"].getType() == libconfig::Setting::TypeFloat) ? (float)p2Setting["z"] : (float)(int)p2Setting["z"];

            int r = (int)colorSetting["r"];
            int g = (int)colorSetting["g"];
            int b = (int)colorSetting["b"];

            Math::Point p0(p0x, p0y, p0z);
```

```
            Math::Point p1(p1x, p1y, p1z);
            Math::Point p2(p2x, p2y, p2z);
            Color color(r, g, b);

            std::string material = setting["material"].c_str();
            std::shared_ptr<AMaterial> mat;
            if (material == "Default") {
                mat = std::make_shared<Default>();
            } else if (material == "Metal") {
                mat = std::make_shared<Metal>();
            } else if (material == "Chessboard") {
                mat = std::make_shared<ChessboardMaterial>();
            } else if (material == "PerlinNoise") {
                mat = std::make_shared<PerlinNoiseMaterial>();
            } else {
                throw std::runtime_error("Plane: Unknown material type");
            }

            return std::make_shared<Plane>(p0, p1, p2, color, mat);
        } catch (const std::exception &e) {
            std::cerr « e.what() « std::endl;
            return nullptr;
        }
    }

    void registerPlugin()
    {
        PluginsManager::registerPrimitiveFactory("plane", &createPrimitive);
    }
};
```

7.65 include/plugins/primitives/Sphere.hpp File Reference

```
#include "PluginsManager.hpp"
#include "IPrimitive.hpp"
#include "AMaterial.hpp"
#include "Default.hpp"
#include "Metal.hpp"
#include "Chessboard.hpp"
#include "PerlinNoise.hpp"
#include "Transparent.hpp"
```

Include dependency graph for Sphere.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class Sphere

  A class to represent a Sphere in 3D space.

Functions

- std::shared_ptr< IPrimitive > createPrimitive (const libconfig::Setting &setting)

  Create a Sphere object.
- void registerPlugin ()

  Register the Sphere plugin.

7.65.1 Function Documentation

7.65.1.1 createPrimitive()

```
std::shared_ptr< IPrimitive > createPrimitive (
            const libconfig::Setting & setting)
```

Create a Sphere object.

This function creates a new Sphere object.

Parameters

setting	The settings for the Sphere object.

Returns

A pointer to the created Sphere object.

7.65.1.2 registerPlugin()

```
void registerPlugin ()
```

Register the Sphere plugin.

This function is called to register the Sphere plugin.

7.66 Sphere.hpp

Go to the documentation of this file.

```
00001 /*
00002 ** EPITECH PROJECT, 2025
00003 ** Sans titre(Espace de travail)
00004 ** File description:
00005 ** Sphere
00006 */
00007
00015 #pragma once
00016
00017 #include "PluginsManager.hpp"
00018 #include "IPrimitive.hpp"
00019 #include "AMaterial.hpp"
00020 #include "Default.hpp"
00021 #include "Metal.hpp"
00022 #include "Chessboard.hpp"
00023 #include "PerlinNoise.hpp"
00024 #include "Transparent.hpp"
00025
00030 class Sphere : public IPrimitive {
00031     public:
00038         Sphere(Math::Vector center, float radius, const Color &color, const std::shared_ptr<AMaterial>
      &mat);
00039
00044         ~Sphere() override = default;
00045
00050         const std::string &getType() const override;
00051
00057         bool intersect(const Ray &ray) const override;
00058
00064         Math::Point getIntersection(const Ray &ray) const override;
00065
00071         Math::Vector normal_at(const Math::Point &point) const override;
00072
00077         Color getColor() const override;
00078
00083         std::shared_ptr<AMaterial> getMaterial() const override;
00084
00085     private:
00086         Math::Point _center;
00087         double _radius;
00088         Color _color;
00089         std::shared_ptr<AMaterial> _material;
00090 };
00091
00092 extern "C" {
00093
00100     std::shared_ptr<IPrimitive> createPrimitive(const libconfig::Setting &setting)
00101     {
00102         try {
```

```
            if (!setting.exists("x") || !setting.exists("y") ||
                !setting.exists("z") || !setting.exists("radius") ||
                !setting.exists("color") || !setting.exists("material"))
                throw std::runtime_error("Missing required fields (x, y, z, radius, color, material) in Sphere settings.");
            if (!setting["color"].isGroup() || setting["material"].getType() != libconfig::Setting::TypeString)
                throw std::runtime_error("Sphere: color is not a group");
            const libconfig::Setting &colorSetting = setting["color"];

            if (!colorSetting.exists("r") || !colorSetting.exists("g") ||
                !colorSetting.exists("b") || !colorSetting["r"].isNumber() ||
                !colorSetting["g"].isNumber() || !colorSetting["b"].isNumber())
                throw std::runtime_error("Invalid or missing color fields (r, g, b) in Sphere settings.");

            auto getFloatOrInt = [](const libconfig::Setting &s) -> float {
                if (s.getType() == libconfig::Setting::TypeFloat)
                    return static_cast<float>(s);
                if (s.getType() == libconfig::Setting::TypeInt)
                    return static_cast<float>(static_cast<int>(s));
                throw std::runtime_error("Sphere: value is not a number");
            };

            std::string material = setting["material"].c_str();
            std::shared_ptr<AMaterial> mat;
            if (material == "Default") {
                mat = std::make_shared<Default>();
            } else if (material == "Metal") {
                mat = std::make_shared<Metal>();
            } else if (material == "Chessboard") {
                mat = std::make_shared<ChessboardMaterial>();
            } else if (material == "PerlinNoise") {
                mat = std::make_shared<PerlinNoiseMaterial>();
            } else if (material == "Transparent") {
                mat = std::make_shared<Transparent>();
            } else {
                throw std::runtime_error("Sphere: Unknown material type");
            }
            Math::Vector position(
                getFloatOrInt(setting["x"]),
                getFloatOrInt(setting["y"]),
                getFloatOrInt(setting["z"])
            );
            float radius = getFloatOrInt(setting["radius"]);
            Color color(
                static_cast<int>(colorSetting["r"]),
                static_cast<int>(colorSetting["g"]),
                static_cast<int>(colorSetting["b"])
            );
            return std::make_shared<Sphere>(position, radius, color, mat);
        } catch (const std::exception &e) {
            std::cerr « "[Sphere] Error: " « e.what() « std::endl;
            return nullptr;
        }
    }

    void registerPlugin()
    {
        PluginsManager::registerPrimitiveFactory("sphere", &createPrimitive);
    }
};
```

7.67 include/plugins/primitives/Triangle.hpp File Reference

```
#include "PluginsManager.hpp"
#include "IPrimitive.hpp"
#include "Point.hpp"
#include "AMaterial.hpp"
#include "Default.hpp"
#include "Metal.hpp"
#include "Chessboard.hpp"
#include "PerlinNoise.hpp"
```

Include dependency graph for Triangle.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class Triangle

  A class to represent a Triangle in 3D space.

Functions

- std::shared_ptr< IPrimitive > createPrimitive (const libconfig::Setting &setting)

  Create a Triangle object.
- void registerPlugin ()

  Register the Triangle plugin.

7.67.1 Function Documentation

7.67.1.1 createPrimitive()

```
std::shared_ptr< IPrimitive > createPrimitive (
            const libconfig::Setting & setting)
```

Create a Triangle object.

This function creates a new Triangle object.

Parameters

setting	The settings for the Triangle object.

Returns

A pointer to the created Triangle object.

7.67.1.2 registerPlugin()

```
void registerPlugin ()
```

Register the Triangle plugin.

This function is called to register the Triangle plugin.

7.68 Triangle.hpp

Go to the documentation of this file.

```
/*
** EPITECH PROJECT, 2025
** Sans titre(Espace de travail)
** File description:
** Triangle
*/

#pragma once

#include "PluginsManager.hpp"
#include "IPrimitive.hpp"
#include "Point.hpp"
#include "AMaterial.hpp"
#include "Default.hpp"
#include "Metal.hpp"
#include "Chessboard.hpp"
#include "PerlinNoise.hpp"

class Triangle : public IPrimitive {
    public:

        Triangle(Math::Point v0, Math::Point v1, Math::Point v2, const Color &color,
            const std::shared_ptr<AMaterial> &mat);

        ~Triangle() override = default;

        const std::string &getType() const override;

        bool intersect(const Ray &ray) const override;

        Math::Point getIntersection(const Ray &ray) const override;

        Math::Vector normal_at(const Math::Point &point) const override;

        Color getColor() const override;

        std::shared_ptr<AMaterial> getMaterial() const override;

    private:
        std::string _axis;
        Math::Point _v0, _v1, _v2;
        Color _color;
        Math::Vector _normal;
        std::shared_ptr<AMaterial> _material;
};

extern "C" {

    std::shared_ptr<IPrimitive> createPrimitive(const libconfig::Setting &setting)
    {
        try {
            if (!setting.exists("v0") || !setting.exists("v1") || !setting.exists("v2") ||
                !setting.exists("color"))
                throw std::runtime_error("Missing required fields (v0, v1, v2, color) in Triangle settings.");
            if (!setting["color"].isGroup())
                throw std::runtime_error("Triangle: color is not a group");

            const libconfig::Setting &v0Setting = setting["v0"];
            const libconfig::Setting &v1Setting = setting["v1"];
            const libconfig::Setting &v2Setting = setting["v2"];
            const libconfig::Setting &colorSetting = setting["color"];

            if (!v0Setting.exists("x") || !v0Setting.exists("y") ||
                !v0Setting.exists("z") || !v0Setting["x"].isNumber() ||
                !v0Setting["y"].isNumber() || !v0Setting["z"].isNumber())
                throw std::runtime_error("Invalid or missing v0 fields (x, y, z) in Triangle settings.");

            if (!v1Setting.exists("x") || !v1Setting.exists("y") ||
                !v1Setting.exists("z") || !v1Setting["x"].isNumber() ||
                !v1Setting["y"].isNumber() || !v1Setting["z"].isNumber())
                throw std::runtime_error("Invalid or missing v1 fields (x, y, z) in Triangle settings.");

            if (!v2Setting.exists("x") || !v2Setting.exists("y") ||
                !v2Setting.exists("z") || !v2Setting["x"].isNumber() ||
                !v2Setting["y"].isNumber() || !v2Setting["z"].isNumber())
                throw std::runtime_error("Invalid or missing v2 fields (x, y, z) in Triangle settings.");

            if (!colorSetting.exists("r") || !colorSetting.exists("g") ||
```

```
                !colorSetting.exists("b") || !colorSetting["r"].isNumber() ||
                !colorSetting["g"].isNumber() || !colorSetting["b"].isNumber()) {
                throw std::runtime_error("Invalid or missing color fields (r, g, b) in Triangle settings.");
            }

            auto getFloatOrInt = [](const libconfig::Setting &s) -> float {
                if (s.getType() == libconfig::Setting::TypeFloat)
                    return static_cast<float>(s);
                if (s.getType() == libconfig::Setting::TypeInt)
                    return static_cast<float>(static_cast<int>(s));
                throw std::runtime_error("Triangle: value is not a number");
            };

            Math::Point v0(
                getFloatOrInt(v0Setting["x"]),
                getFloatOrInt(v0Setting["y"]),
                getFloatOrInt(v0Setting["z"])
            );
            Math::Point v1(
                getFloatOrInt(v1Setting["x"]),
                getFloatOrInt(v1Setting["y"]),
                getFloatOrInt(v1Setting["z"])
            );
            Math::Point v2(
                getFloatOrInt(v2Setting["x"]),
                getFloatOrInt(v2Setting["y"]),
                getFloatOrInt(v2Setting["z"])
            );
            Color color(
                static_cast<int>(colorSetting["r"]),
                static_cast<int>(colorSetting["g"]),
                static_cast<int>(colorSetting["b"])
            );
            std::string material = setting["material"].c_str();
            std::shared_ptr<AMaterial> mat;
            if (material == "Default") {
                mat = std::make_shared<Default>();
            } else if (material == "Metal") {
                mat = std::make_shared<Metal>();
            } else if (material == "Chessboard") {
                mat = std::make_shared<ChessboardMaterial>();
            } else if (material == "PerlinNoise") {
                mat = std::make_shared<PerlinNoiseMaterial>();
            } else {
                throw std::runtime_error("Triangle: Unknown material type");
            }
            return std::make_shared<Triangle>(v0, v1, v2, color, mat);
        } catch (const std::exception &e) {
            std::cerr « e.what() « std::endl;
            return nullptr;
        }
    }

    void registerPlugin()
    {
        PluginsManager::registerPrimitiveFactory("triangle", &createPrimitive);
    }
};
```

7.69 include/transformations/Rotation.hpp File Reference

Rotation class header file.

```
#include "ITransformations.hpp"
#include "Define.hpp"
#include "Scene.hpp"
```

Include dependency graph for Rotation.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class Rotation

  A class to represent a Rotation transformation.

7.69.1 Detailed Description

Rotation class header file.

This file contains the definition of the Rotation class, which is used to represent a Rotation transformation in the raytracer project.

7.70 Rotation.hpp

Go to the documentation of this file.

```
00001 /*
00002 ** EPITECH PROJECT, 2025
00003 ** Sans titre(Espace de travail)
00004 ** File description:
00005 ** Rotation
00006 */
00007
00014 #pragma once
00015
00016 #include "ITransformations.hpp"
00017 #include "Define.hpp"
00018 #include "Scene.hpp"
00019
00026 class Rotation : public ITransformations {
00027     public:
00033         Rotation();
00034
00040         ~Rotation() override = default;
00041
00047         void apply(Math::Point &origin, Math::Vector Rotation) override;
00048
00052         void RotateX(bool Ctrl_pressed, Scene &scene);
00053
00057         void RotateY(bool Ctrl_pressed, Scene &scene);
00058
00062         void RotateZ(bool Ctrl_pressed, Scene &scene);
00063
00064 };
```

7.71 include/transformations/Shear.hpp File Reference

Shear class header file.

```
#include "ITransformations.hpp"
#include "Define.hpp"
#include "Scene.hpp"
```

Include dependency graph for Shear.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class Shear

  A class to represent a Shear transformation.

7.71.1 Detailed Description

Shear class header file.

This file contains the definition of the Shear class, which is used to represent a Shear transformation in the raytracer project.

7.72 Shear.hpp

Go to the documentation of this file.

```
/*
** EPITECH PROJECT, 2025
** Sans titre(Espace de travail)
** File description:
** Shear
*/

#pragma once

#include "ITransformations.hpp"
#include "Define.hpp"
#include "Scene.hpp"

class Shear : public ITransformations {
    public:
        Shear();

        ~Shear() override = default;

        void apply(Math::Point &origin, Math::Vector Shear) override;

        void shear(Scene &scene);
};
```

7.73 include/transformations/Translation.hpp File Reference

Translation class header file.

```
#include "ITransformations.hpp"
#include "Define.hpp"
#include "Scene.hpp"
```

Include dependency graph for Translation.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class Translation

  A class to represent a translation transformation.

7.73.1 Detailed Description

Translation class header file.

This file contains the definition of the Translation class, which is used to represent a translation transformation in the raytracer project.

7.74 Translation.hpp

Go to the documentation of this file.

```
/*
** EPITECH PROJECT, 2025
** Sans titre(Espace de travail)
** File description:
** Translation
*/

#pragma once

#include "ITransformations.hpp"
#include "Define.hpp"
#include "Scene.hpp"

class Translation : public ITransformations {
    public:
        Translation();

        ~Translation() override = default;

        void apply(Math::Point &origin, Math::Vector translation) override;

        void translateUp(Scene &scene);

        void translateDown(Scene &scene);

        void translateLeft(Scene &scene);

        void translateRight(Scene &scene);

        void translateIn(Scene &scene);

        void translateOut(Scene &scene);
};
```

7.75 include/utilitaries/Color.hpp File Reference

Color class header file.

`#include "Define.hpp"`

Include dependency graph for Color.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class Color

  A class to represent a color in RGB format.

Functions

- std::ostream & operator<< (std::ostream &os, const Color &color)

7.75.1 Detailed Description

Color class header file.

This file contains the definition of the Color class, which is used for color representation in the raytracer project.

7.75.2 Function Documentation

7.75.2.1 operator<<()

```
std::ostream & operator<< (
            std::ostream & os,
            const Color & color)
```

7.76 Color.hpp

Go to the documentation of this file.

```
00001 /*
00002 ** EPITECH PROJECT, 2025
00003 ** Sans titre(Espace de travail)
00004 ** File description:
00005 ** Color
00006 */
00007
00014 #pragma once
00015
00016 #include "Define.hpp"
00017
00022 class Color {
00023     public:
00024
00034         Color(double r = 0, double g = 0, double b = 0);
00035
00041         Color operator+(const Color &other) const;
00042
00048         Color &operator+=(const Color &other);
00049
00055         Color operator-(const Color &other) const;
00056
00062         Color operator*(double scalar) const;
00063
00069         Color &operator*=(double scalar);
00070
00071         Color &operator/=(int divide);
00072
00078         Color operator*(const Color &other) const;
00079
00085         Color &operator*=(const Color &other);
00086
00092         Color operator/(double scalar) const;
00093
00099         Color applyIntensity(double intensity) const;
00100
00105         Color clamp() const;
00106
00111         double getR() const;
00112
00117         double getG() const;
00118
00123         double getB() const;
00124
00129         void setColor(double r, double g, double b);
00130
00131     private:
00132         double _r;
00133         double _g;
00134         double _b;
00135 };
00136
00137 std::ostream &operator«(std::ostream &os, const Color &color);
```

7.77 include/utilitaries/Cursor.hpp File Reference

`#include "Define.hpp"`

Include dependency graph for Cursor.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class Cursor

Enumerations

- enum CursorType { TRANSLATE , ROTATE }

7.77.1 Enumeration Type Documentation

7.77.1.1 CursorType

```
enum CursorType
```

Enumerator

TRANSLATE	
ROTATE	

7.78 Cursor.hpp

Go to the documentation of this file.

```
00001 /*
00002 ** EPITECH PROJECT, 2025
00003 ** raytracer
00004 ** File description:
00005 ** Cursor
00006 */
00007
00008 #pragma once
00009 #include "Define.hpp"
00010
00011 enum CursorType {
00012     TRANSLATE,
00013     ROTATE
00014 };
00015
00016 class Cursor {
00017     public:
00018         Cursor(std::pair<double, double> percentageStartLine, std::pair<double, double>
      percentageFinishLine,
00019             int width, int height, const sf::Font& font, std::string cursorTitleText);
00020         ~Cursor();
00021
00029         void initText(const sf::Font& font, sf::Vector2f _startLine,
00030             sf::Vector2f _finishLine, std::string cursorTitleText);
00031
00037         void initShape(sf::Vector2f _startLine, sf::Vector2f _finishLine);
00038
00043         sf::Text getText() const;
00044
00049         sf::Text getCursorTitleText() const;
00050
00055         sf::RectangleShape getRectangle() const;
00056
00061         std::array<sf::Vertex, 2> getCursorLine() const;
00062
00067         int getPosCursor() const;
00068
00073         bool isCursorSelected() const;
00074
00079         void setCursorSelected(const bool cursorSelect);
00080
00085         void moveCursor(const int offset);
00086
```

```
        void setCursorText(const std::string cursorsText);

        void moveRectangleShape(const sf::Vector2f offset);

        sf::Vector2i updateCursor(sf::Vector2i mousePos, sf::Vector2i prevMousePos, int& eventKey);

        void mouseClickedOnCursor(sf::Vector2i &prevMousPos);

        CursorType getType() const;

    protected:
    private:
        std::array<sf::Vertex, 2> _cursorLine;
        sf::RectangleShape _cursorShape;
        sf::Text _cursorText;
        sf::Text _cursorTitleText;
        int _posCursor;
        bool _isCursorSelected;
        CursorType _type;

        std::map<std::string, std::pair<sf::Keyboard::Key, sf::Keyboard::Key» _keyToPressed;
};
```

7.79 include/utilitaries/Face.hpp File Reference

```
#include "PluginsManager.hpp"
#include "IPrimitive.hpp"
#include "Point.hpp"
#include "AMaterial.hpp"
```

Include dependency graph for Face.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class Face

  A class to represent a Face in 3D space.

7.80 Face.hpp

Go to the documentation of this file.

```
/*
** EPITECH PROJECT, 2025
** Sans titre(Espace de travail)
** File description:
** Face
*/

#pragma once

#include "PluginsManager.hpp"
#include "IPrimitive.hpp"
#include "Point.hpp"
#include "AMaterial.hpp"

class Face : public IPrimitive {
    public:

        Face(Math::Point v0, Math::Point v1, Math::Point v2, const Color &color);

        ~Face() override = default;

        const std::string &getType() const override;

        bool intersect(const Ray &ray) const override;

        Math::Point getIntersection(const Ray &ray) const override;

        Math::Vector normal_at(const Math::Point &point) const override;

```

```
        Color getColor() const override;

        std::shared_ptr<AMaterial> getMaterial() const override;

    private:
        std::string _axis;
        Math::Point _v0, _v1, _v2;
        Color _color;
        Math::Vector _normal;
        std::shared_ptr<AMaterial> _material;
};
```

7.81 include/utilitaries/HitInfo.hpp File Reference

HitInfo class header file.

```
#include "Vector.hpp"
#include "Color.hpp"
#include "Define.hpp"
```

Include dependency graph for HitInfo.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class HitInfo

  A class to represent hit information in the raytracer project.

7.81.1 Detailed Description

HitInfo class header file.

This file contains the definition of the HitInfo class, which is used to store information about a hit in the raytracer project.

7.82 HitInfo.hpp

Go to the documentation of this file.

```
/*
** EPITECH PROJECT, 2025
** Sans titre(Espace de travail)
** File description:
** HitInfo
*/

#pragma once

#include "Vector.hpp"
#include "Color.hpp"
#include "Define.hpp"

class HitInfo {
    public:

        HitInfo();

        HitInfo(const Math::Vector &position, const Math::Vector &normal, const Color &color, float
    distance);

        const Math::Vector &getPosition() const;

        const Math::Vector &getNormal() const;

        const Color &getColor() const;

        float getDistance() const;

    private:
        Math::Vector _position;
        Math::Vector _normal;
        Color _color;
        float _distance;
};
```

7.83 include/utilitaries/Matrix.hpp File Reference

This graph shows which files directly or indirectly include this file:

Classes

- class Math::Matrix< X, Y >

  A class to use Matrix for calculus.

Namespaces

- namespace Math

7.84 Matrix.hpp

Go to the documentation of this file.

```
00001 /*
00002 ** EPITECH PROJECT, 2025
00003 ** raytracer
00004 ** File description:
00005 ** Matrix
00006 */
00007
00015 #pragma once
00016
00017 namespace Math {
00024     template <unsigned int X, unsigned int Y>
00025     class Matrix {
00026         public:
00027             unsigned int _row;
00028             unsigned int _col;
00029             double _matrix[X][Y];
00030
00035             Matrix()
00036                 : _row(X), _col(Y)
00037             {
00038                 for (unsigned int i = 0; i < X; i++) {
00039                     for (unsigned int j = 0; j < Y; j++) {
00040                         _matrix[i][j] = 0.0;
00041                     }
00042                 }
00043             };
00044
00049             Matrix(double matrix[X][Y])
00050                 : _row(X), _col(Y)
00051             {
00052                 for (unsigned int i = 0; i < X; i++) {
00053                     for (unsigned int j = 0; j < Y; j++) {
00054                         _matrix[i][j] = matrix[i][j];
00055                     }
00056                 }
00057             }
00058
00062             ~Matrix()
00063             {
00064             }
00065
00074             double &operator()(unsigned int row, unsigned int col)
00075             {
00076                 if (row >= X || col >= Y)
00077                     throw std::out_of_range("Out of range");
00078                 return this->_matrix[row][col];
00079             };
00080
00089             const double& operator()(unsigned int row, unsigned int col) const
00090             {
00091                 if (row >= X || col >= Y)
00092                     throw std::out_of_range("Out of range");
00093                 return _matrix[row][col];
00094             }
```

```

            template <unsigned int x, unsigned int y>
            Matrix<X, y> operator*(Matrix<x, y> const &m2)
            {
                Matrix<X, y> res;
                if (Y != x) {
                    throw std::out_of_range("Out of range");
                }
                for (unsigned int i = 0; i < this->getRow(); i++) {
                    for (unsigned int j = 0; j < m2.getCol(); j++) {
                        res(i, j) = 0;
                        for (unsigned int k = 0; k < m2.getRow(); k++) {
                            res._matrix[i][j] += this->_matrix[i][k] * m2._matrix[k][j];
                        }
                    }
                }
                return res;
            }

            unsigned int getRow() const {
                return _row;
            };

            unsigned int getCol() const {
                return _col;
            };
        protected:
        private:
    };
}
```

7.85 include/utilitaries/Point.hpp File Reference

```
#include "Vector.hpp"
#include "Matrix.hpp"
```

Include dependency graph for Point.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class Math::Point

  A class to represent a 3D point with basic operations.

Namespaces

- namespace Math

7.86 Point.hpp

Go to the documentation of this file.

```
/*
** EPITECH PROJECT, 2025
** raytracer
** File description:
** Point
*/

#pragma once

#include "Vector.hpp"
#include "Matrix.hpp"

namespace Math {

    class Point {
        public:
```

```
            Point();

            Point(double x, double y, double z);

            Point operator+(const Point &point) const;

            Point operator+(const Vector &vector) const;

            Point operator-(const Vector &vector) const;

            Point &operator+=(const Vector &vector);

            bool operator==(const Point &point) const;

            Vector operator-(const Point &other) const;

            double getX() const;

            double getY() const;

            double getZ() const;

            void setX(const double x);

            void setY(const double y);

            void setZ(const double z);

            Math::Matrix<4, 1> pointToMatrix()
            {
                Math::Matrix<4, 1> result;
                result._matrix[0][0] = _x;
                result._matrix[1][0] = _y;
                result._matrix[2][0] = _z;
                result._matrix[3][0] = 1.0;
                return result;
            }

            Math::Matrix<1, 2> shearToMatrix()
            {
                Math::Matrix<1, 2> result;
                result._matrix[0][0] = _x;
                result._matrix[0][1] = _y;
                return result;
            }

            double dot(const Vector &other) const;

        protected:
        private:
            double _x;
            double _y;
            double _z;
    };
}
```

7.87 include/utilitaries/Ray.hpp File Reference

Ray class header file.

```
#include "Vector.hpp"
#include "Define.hpp"
#include "Point.hpp"
```

Include dependency graph for Ray.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class Ray

  A class to represent a ray in 3D space.

7.87.1 Detailed Description

Ray class header file.

This file contains the definition of the Ray class, which is used for ray representation in the raytracer project.

Note

The Ray class is used to represent rays in 3D space.

7.88 Ray.hpp

Go to the documentation of this file.

```
00001 /*
00002 ** EPITECH PROJECT, 2025
00003 ** Sans titre(Espace de travail)
00004 ** File description:
00005 ** Ray
00006 */
00007
00015 #pragma once
00016
00017 #include "Vector.hpp"
00018 #include "Define.hpp"
00019 #include "Point.hpp"
00020
00025 class Ray {
00026     public:
00027
00036         Ray(const Math::Point &origin = Math::Point(), const Math::Vector &direction =
      Math::Vector());
00037
00042         const Math::Point &getOrigin() const;
00043
00048         const Math::Vector &getDirection() const;
00049
00054         void setOrigin(const Math::Point &origin);
00055
00060         void setDirection(const Math::Vector &direction);
00061
00062     private:
00063         Math::Point _origin;
00064         Math::Vector _direction;
00065 };
```

7.89 include/utilitaries/Vector.hpp File Reference

Vector class header file.

```
#include "Exception.hpp"
#include "Define.hpp"
#include "Matrix.hpp"
```

Include dependency graph for Vector.hpp: This graph shows which files directly or indirectly include this file:

Classes

- class Math::Vector

  A class to represent a 3D vector with basic operations.

Namespaces

- namespace Math

Functions

- Vector Math::operator* (double scalar, const Vector &vector)

  Scalar multiplication operator overload (commutative).

7.89.1 Detailed Description

Vector class header file.

This file contains the definition of the Vector class, which is used for mathematical operations in the raytracer project.

7.90 Vector.hpp

Go to the documentation of this file.

```
00001 /*
00002 ** EPITECH PROJECT, 2025
00003 ** Sans titre(Espace de travail)
00004 ** File description:
00005 ** Vector
00006 */
00007
00014 #pragma once
00015
00016 #include "Exception.hpp"
00017 #include "Define.hpp"
00018 #include "Matrix.hpp"
00019
00020 namespace Math {
00025     class Vector {
00026         public:
00027
00037             Vector(double x = 0, double y = 0, double z = 0);
00038
00043             double length() const;
00044
00050             double dot(const Vector &other) const;
00051
00056             Vector normalize() const;
00057
00058             Vector invert() const;
00059
00065             Vector operator+(const Vector &other) const;
00066
00072             Vector &operator+=(const Vector &other);
00073
00079             Vector operator-(const Vector &other) const;
00080
00086             Vector &operator-=(const Vector &other);
00087
00093             Vector operator*(const Vector &other) const;
00094
00100             Vector &operator*=(const Vector &other);
00101
00107             Vector operator*(double scalar) const;
00108
00114             Vector &operator*=(double scalar);
00115
00121             Vector operator/(const Vector &other) const;
00122
00128             Vector &operator/=(const Vector &other);
00129
00135             Vector operator/(double scalar) const;
00136
00142             Vector &operator/=(double scalar);
00143
```

```
            bool operator==(const Vector &other) const;

            bool operator==(std::string axis) const;

            double getX() const;

            double getY() const;

            double getZ() const;

            Vector crossProduct(const Vector &other) const;

            Math::Matrix<4, 1> vectorToMatrix()
            {
                Math::Matrix<4, 1> result;
                result._matrix[0][0] = _x;
                result._matrix[1][0] = _y;
                result._matrix[2][0] = _z;
                result._matrix[3][0] = 1.0;
                return result;
            }

            Math::Matrix<1, 2> shearToMatrix()
            {
                Math::Matrix<1, 2> result;
                result._matrix[0][0] = _x;
                result._matrix[0][1] = _y;
                return result;
            }

            inline double random_double();
            inline double random_double(double min, double max);
            Vector random();
            Vector random(double min, double max);
            bool near_zero() const;
            Vector reflect(const Vector& v, const Vector& n);

            Vector operator-() const { return Vector(-_x, -_y, -_z); } // Unary negation
        private:
            double _x;
            double _y;
            double _z;
    };

    Vector operator*(double scalar, const Vector &vector);
}
```

7.91 obj/core/CalculateRender.d File Reference

7.92 obj/core/Camera.d File Reference

7.93 obj/core/Core.d File Reference

7.94 obj/core/Gui.d File Reference

7.95 obj/core/hotreload.d File Reference

7.96 obj/core/Main.d File Reference

7.97 obj/core/PluginsHandle.d File Reference

7.98 obj/core/PluginsManager.d File Reference

7.99 obj/core/Render.d File Reference

7.100 obj/core/Scene.d File Reference

7.101 obj/core/Screen.d File Reference

7.102 obj/exceptions/Exception.d File Reference

7.103 obj/images/Image.d File Reference

7.104 obj/loader/ObjParser.d File Reference

7.105 obj/loader/SceneLoader.d File Reference

7.106 obj/materials/AMaterial.d File Reference

7.107 obj/materials/Chessboard.d File Reference

7.108 obj/materials/Default.d File Reference

7.109 obj/materials/Metal.d File Reference

7.110 obj/materials/PerlinNoise.d File Reference

7.111 obj/materials/Transparent.d File Reference

7.112 obj/transformation/Rotation.d File Reference

7.113 obj/transformation/Shear.d File Reference

7.114 obj/transformation/Translation.d File Reference

```
#include "Render.hpp"
```
Include dependency graph for CalculateRender.cpp:

Functions

- double random_double ()
- Math::Vector orthogonalVector (const Math::Vector &normal)

7.122.1 Detailed Description

Render class implementation file.

This file contains the implementation of the methods used to calculate the render of the scene in the raytracer project.

7.122.2 Function Documentation

7.122.2.1 orthogonalVector()

```
Math::Vector orthogonalVector (
            const Math::Vector & normal)  [inline]
```

7.122.2.2 random_double()

```
double random_double ()  [inline]
```

7.123 src/core/Camera.cpp File Reference

Camera class implementation.

```
#include "Camera.hpp"
```
Include dependency graph for Camera.cpp:

7.123.1 Detailed Description

Camera class implementation.

This file contains the implementation of the Camera class, which is used to represent a camera in the raytracer project.

Note

> The Camera class is used to define the position and direction of the camera. It also provides methods to get the camera's position and direction.

This file contains the implementation of the Camera class, which is used to represent a camera in the raytracer project.

Note

> The Camera class is used to define the camera's position, direction, and field of view in the 3D scene.

7.124 src/core/Core.cpp File Reference

```
#include "Core.hpp"
```
Include dependency graph for Core.cpp:

7.125 src/core/Gui.cpp File Reference

GUI class implementation file.

```
#include "Gui.hpp"
```
Include dependency graph for Gui.cpp:

7.125.1 Detailed Description

GUI class implementation file.

This file contains the implementation of the GUI class, which is used to manage the graphical user interface in the raytracer project.

Note

The GUI class is responsible for rendering the scene in SFML and handling user interactions.

7.126 src/core/hotreload.cpp File Reference

Hot reload implementation for the raytracer project.

```
#include "Core.hpp"
#include <thread>
#include <chrono>
```
Include dependency graph for hotreload.cpp:

7.126.1 Detailed Description

Hot reload implementation for the raytracer project.

This file contains the implementation of the hot reload feature for the raytracer project. It allows for reloading of shared

7.127 src/core/Main.cpp File Reference

Main file for the raytracer project.

```
#include "Core.hpp"
#include "Define.hpp"
#include "Exception.hpp"
```
Include dependency graph for Main.cpp:

Functions

- int help ()

  Display the help message.
- int main (int ac, char **av)

  Main function.

7.127.1 Detailed Description

Main file for the raytracer project.

7.127.2 Function Documentation

7.127.2.1 help()

```
int help ()
```

Display the help message.

Returns

0 on success.

7.127.2.2 main()

```
int main (
            int ac,
            char ** av)
```

Main function.

Parameters

ac	Number of arguments.
av	Array of arguments.

Returns

0 on success.

84 on failure.

Note

This function is the entry point of the program.

7.128 src/core/PluginsHandle.cpp File Reference

`#include "PluginsHandle.hpp"`
Include dependency graph for PluginsHandle.cpp:

7.129 src/core/PluginsManager.cpp File Reference

Implementation of the PluginsManager class.

`#include "PluginsManager.hpp"`
Include dependency graph for PluginsManager.cpp:

7.129.1 Detailed Description

Implementation of the PluginsManager class.

This file contains the implementation of the methods for managing plugins in the raytracer project.

7.130 src/core/Render.cpp File Reference

Render class implementation file.

`#include "Render.hpp"`
Include dependency graph for Render.cpp:

7.130.1 Detailed Description

Render class implementation file.

This file contains the implementation of the Render class, which is used to render the scene in the raytracer project.

7.131 src/core/Scene.cpp File Reference

Scene class implementation.

`#include "Scene.hpp"`
Include dependency graph for Scene.cpp:

7.131.1 Detailed Description

Scene class implementation.

This file contains the implementation of the Scene class, which is used to manage the scene in the raytracer project.

Note

The Scene class is responsible for managing the objects in the scene. It stores the objects all the entities in the scene like lights, cameras, primitives, etc.

7.132 src/core/Screen.cpp File Reference

```
#include "Screen.hpp"
```
Include dependency graph for Screen.cpp:

7.133 src/exceptions/Exception.cpp File Reference

```
#include "Exception.hpp"
```
Include dependency graph for Exception.cpp:

7.134 src/images/Image.cpp File Reference

Image class implementation.

```
#include "Image.hpp"
```
Include dependency graph for Image.cpp:

7.134.1 Detailed Description

Image class implementation.

This file contains the implementation of the Image class, which is used to represent an image in the raytracer project.

Note

The Image class is used to store pixel data and save the image to a file.

7.135 src/loader/ObjParser.cpp File Reference

```
#include "ObjParser.hpp"
#include "Face.hpp"
#include <unordered_map>
#include <sstream>
#include <filesystem>
```
Include dependency graph for ObjParser.cpp:

7.136 src/loader/SceneLoader.cpp File Reference

SceneLoader class implementation.

```
#include "SceneLoader.hpp"
#include "ObjParser.hpp"
```
Include dependency graph for SceneLoader.cpp:

7.136.1 Detailed Description

SceneLoader class implementation.

This file contains the implementation of the SceneLoader class, which is used to load scenes from a configuration file in the raytracer project.

Note

The SceneLoader class is responsible for parsing the scene configuration file and creating the necessary objects for rendering.

7.137 src/materials/AMaterial.cpp File Reference

`#include "AMaterial.hpp"`
Include dependency graph for AMaterial.cpp:

7.138 src/materials/Chessboard.cpp File Reference

`#include "Chessboard.hpp"`
Include dependency graph for Chessboard.cpp:

7.139 src/materials/Default.cpp File Reference

`#include "Default.hpp"`
Include dependency graph for Default.cpp:

7.140 src/materials/Metal.cpp File Reference

`#include "Metal.hpp"`
Include dependency graph for Metal.cpp:

7.141 src/materials/PerlinNoise.cpp File Reference

`#include "PerlinNoise.hpp"`
Include dependency graph for PerlinNoise.cpp:

7.142 src/materials/Transparent.cpp File Reference

`#include "Transparent.hpp"`
Include dependency graph for Transparent.cpp:

7.143 src/plugins/lights/AmbientLight.cpp File Reference

AmbientLight class implementation.

`#include "AmbientLight.hpp"`
Include dependency graph for AmbientLight.cpp:

7.143.1 Detailed Description

AmbientLight class implementation.

This file contains the implementation of the AmbientLight class, which is a plugin for the raytracer project.

Note

The AmbientLight class represents an ambient light source in the scene.

7.144 src/plugins/lights/DirectionalLight.cpp File Reference

DirectionalLight class implementation.

`#include "DirectionalLight.hpp"`
Include dependency graph for DirectionalLight.cpp:

7.144.1 Detailed Description

DirectionalLight class implementation.

This file contains the implementation of the DirectionalLight class, which is a plugin for the raytracer project.

Note

The DirectionalLight class represents a directional light source in the scene.

7.145 src/plugins/lights/PointLight.cpp File Reference

PointLight class implementation.

`#include "PointLight.hpp"`
Include dependency graph for PointLight.cpp:

7.145.1 Detailed Description

PointLight class implementation.

This file contains the implementation of the PointLight class, which is a plugin for the raytracer project.

Note

The PointLight class represents a point light source in the scene.

7.146 src/plugins/primitives/Cone.cpp File Reference

Cone class implementation.

```
#include "Cone.hpp"
#include "Default.hpp"
```

Include dependency graph for Cone.cpp:

7.146.1 Detailed Description

Cone class implementation.

This file contains the implementation of the Cone class, which is a plugin for the raytracer project.

Note

The Cone class represents a Cone in 3D space.

7.147 src/plugins/primitives/Cylindre.cpp File Reference

Cylindre class implementation.

```
#include "Cylindre.hpp"
```

Include dependency graph for Cylindre.cpp:

7.147.1 Detailed Description

Cylindre class implementation.

This file contains the implementation of the Cylindre class, which is a plugin for the raytracer project.

Note

The Cylindre class represents a Cylindre in 3D space.

7.148 src/plugins/primitives/InfinitCone.cpp File Reference

InfinitCone class implementation.

```
#include "InfinitCone.hpp"
#include "Default.hpp"
```

Include dependency graph for InfinitCone.cpp:

7.148.1 Detailed Description

InfinitCone class implementation.

This file contains the implementation of the InfinitCone class, which is a plugin for the raytracer project.

Note

The InfinitCone class represents a InfinitCone in 3D space.

7.149 src/plugins/primitives/InfiniteCylindre.cpp File Reference

InfiniteCylindre class implementation.

`#include "InfiniteCylindre.hpp"`
Include dependency graph for InfiniteCylindre.cpp:

7.149.1 Detailed Description

InfiniteCylindre class implementation.

This file contains the implementation of the InfiniteCylindre class, which is a plugin for the raytracer project.

Note

The InfiniteCylindre class represents a InfiniteCylindre in 3D space.

7.150 src/plugins/primitives/Plane.cpp File Reference

Plane class implementation.

`#include "Plane.hpp"`
Include dependency graph for Plane.cpp:

7.150.1 Detailed Description

Plane class implementation.

This file contains the implementation of the Plane class, which is a plugin for the raytracer project.

Note

The Plane class represents a plane in 3D space.

7.151 src/plugins/primitives/Sphere.cpp File Reference

Sphere class implementation.

```
#include "Sphere.hpp"
```
Include dependency graph for Sphere.cpp:

7.151.1 Detailed Description

Sphere class implementation.

This file contains the implementation of the Sphere class, which is a plugin for the raytracer project.

Note

The Sphere class represents a Sphere in 3D space.

This file contains the implementation of the Sphere class, which is a plugin for the raytracer project.

7.152 src/plugins/primitives/Triangle.cpp File Reference

Triangle class implementation.

```
#include "Triangle.hpp"
#include "Default.hpp"
```
Include dependency graph for Triangle.cpp:

7.152.1 Detailed Description

Triangle class implementation.

This file contains the implementation of the Triangle class, which is a plugin for the raytracer project.

Note

The Triangle class represents a Triangle in 3D space.

7.153 src/transformation/Rotation.cpp File Reference

```
#include "Rotation.hpp"
```
Include dependency graph for Rotation.cpp:

7.154 src/transformation/Shear.cpp File Reference

```
#include "Shear.hpp"
```
Include dependency graph for Shear.cpp:

7.155 src/transformation/Translation.cpp File Reference

Translation class implementation file.

`#include "Translation.hpp"`
Include dependency graph for Translation.cpp:

7.155.1 Detailed Description

Translation class implementation file.

This file contains the implementation of the Translation class, which is used to represent a translation transformation in the raytracer project.

7.156 src/utilitaries/Color.cpp File Reference

Color class implementation.

`#include "Color.hpp"`
Include dependency graph for Color.cpp:

Functions

- std::ostream & operator<< (std::ostream &os, const Color &color)

7.156.1 Detailed Description

Color class implementation.

This file contains the implementation of the Color class, which is used for color representation in the raytracer project.

Note

The Color class is used to represent colors in RGB format.

7.156.2 Function Documentation

7.156.2.1 operator<<()

```
std::ostream & operator<< (
            std::ostream & os,
            const Color & color)
```

7.157 src/utilitaries/Cursor.cpp File Reference

`#include "Cursor.hpp"`
Include dependency graph for Cursor.cpp:

7.158 src/utilitaries/Face.cpp File Reference

Face class implementation.

`#include "Face.hpp"`
Include dependency graph for Face.cpp:

7.158.1 Detailed Description

Face class implementation.

This file contains the implementation of the Face class, which is a plugin for the raytracer project.

Note

The Face class represents a Face in 3D space.

7.159 src/utilitaries/HitInfo.cpp File Reference

HitInfo class implementation.

`#include "HitInfo.hpp"`
Include dependency graph for HitInfo.cpp:

7.159.1 Detailed Description

HitInfo class implementation.

This file contains the implementation of the HitInfo class, which is used to store information about a hit in the raytracer project.

Note

The HitInfo class is used to represent the hit information such as position, normal vector, color, and distance from the ray origin.

7.160 src/utilitaries/Point.cpp File Reference

`#include "Point.hpp"`
Include dependency graph for Point.cpp:

Namespaces

- namespace Math

7.161 src/utilitaries/Ray.cpp File Reference

`#include "Ray.hpp"`
Include dependency graph for Ray.cpp:

7.162 src/utilitaries/Vector.cpp File Reference

Vector class implementation.

`#include "Vector.hpp"`
Include dependency graph for Vector.cpp:

Namespaces

- namespace Math

Functions

- Vector Math::operator∗ (double scalar, const Vector &vector)

  Scalar multiplication operator overload (commutative).

7.162.1 Detailed Description

Vector class implementation.

This file contains the implementation of the Vector class, which is used for vector representation in the raytracer project.

Note

The Vector class is used to represent 3D vectors and perform vector operations.

Index

_col
Math::Matrix< X, Y >, 82
_matrix
Math::Matrix< X, Y >, 82
_row
Math::Matrix< X, Y >, 82
~AMaterial
AMaterial, 14
~ChessboardMaterial
ChessboardMaterial, 26
~Cone
Cone, 34
~Core
Core, 38
~Cursor
Cursor, 42
~Cylindre
Cylindre, 47
~Face
Face, 55
~Gui
Gui, 58
~ILight
ILight, 63
~IPrimitive
IPrimitive, 75
~ITransformations
ITransformations, 78
~Image
Image, 66
~InfinitCone
InfinitCone, 69
~InfiniteCylindre
InfiniteCylindre, 72
~Matrix
Math::Matrix< X, Y >, 80
~Mtl
Mtl, 84
~PerlinNoiseMaterial
PerlinNoiseMaterial, 87
~Plane
Plane, 89
~PluginsHandle
PluginsHandle, 93
~Render
Render, 112
~Rotation
Rotation, 119
~Scene
Scene, 121
~Screen
Screen, 127
~Shear
Shear, 129
~Sphere
Sphere, 131
~Translation
Translation, 136
~Triangle
Triangle, 140

adaptiveSample
Render, 112
addCamera
Scene, 122
addLight
Scene, 122
addPrimitive
Scene, 122
AMaterial, 13
~AMaterial, 14
AMaterial, 14
getAmbient, 14
getColorAt, 14
getDiffuse, 14
getDiffuseColor, 15
getName, 15
getRefractionIndex, 15
getShininess, 15
getSpecular, 15
getSpecularColor, 15
getTransparency, 15
loadFromMtlLine, 15
setDiffuseColor, 16
setRefractionIndex, 16
setShininess, 16
setSpecularColor, 16
setTransparency, 17
AmbiantOcclusion
Render, 112
AmbientLight, 17
AmbientLight, 18
getAmbientComponent, 18
getColor, 18
getDiffuseComponent, 18
getDirection, 19
getIntensity, 19
getPosition, 19
getSpecularComponent, 19

getType, 20
AmbientLight.hpp
createLight, 175
registerPlugin, 175
apply
ITransformations, 78
Rotation, 120
Shear, 129
Translation, 136
applyIntensity
Color, 28

BACKGROUNDCOLOR
Define.hpp, 161

CalculateRender.cpp
orthogonalVector, 219
random_double, 219
Camera, 20
Camera, 21
getFov, 21
getHeight, 21
getPosition, 22
getRay, 22
getWidth, 22
rotate, 23
setPosition, 23
setRay, 23
shear, 24
translate, 24
checkSceneFile
Core, 39
ChessboardMaterial, 24
∼ChessboardMaterial, 26
ChessboardMaterial, 25
getColorAt, 26
clamp
Color, 28
Color, 26
applyIntensity, 28
clamp, 28
Color, 27
getB, 28
getG, 29
getR, 29
operator+, 31
operator+=, 31
operator-, 31
operator/, 32
operator/=, 32
operator∗, 29, 30
operator∗=, 30
setColor, 32
Color.cpp
operator<<, 229
Color.hpp
operator<<, 207
COLOR_THRESHOLD
Define.hpp, 161

ComputeDiffuseLights
Render, 113
computeFastRender
Render, 113
ComputePixelColor
Render, 113
computeRender
Render, 113
ComputeSpecularLight
Render, 113
Cone, 33
∼Cone, 34
Cone, 33
getColor, 34
getIntersectBase, 34
getIntersectCone, 35
getIntersection, 35
getMaterial, 35
getType, 36
intersect, 36
intersect_base, 36
intersectCone, 37
normal_at, 37
Cone.hpp
createPrimitive, 185
registerPlugin, 185
Core, 38
∼Core, 38
checkSceneFile, 39
Core, 38
createGuiThread, 39
createRenderThread, 39
init, 39
loop, 40
notifyRenderThread, 40
reloadScene, 40
setCtrl, 40
setHandleRotation, 40
setHandleTranslation, 41
createDirAndFileName
Image, 66
createGuiThread
Core, 39
createImage
Gui, 58
createLight
AmbientLight.hpp, 175
DirectionalLight.hpp, 177
PluginsManager, 97
PointLight.hpp, 179
createPrimitive
Cone.hpp, 185
Cylindre.hpp, 188
InfinitCone.hpp, 190
InfiniteCylindre.hpp, 193
Plane.hpp, 196
PluginsManager, 97
Sphere.hpp, 198

Triangle.hpp, 201
createRenderThread
Core, 39
crossProduct
Math::Vector, 144
Cursor, 41
~Cursor, 42
Cursor, 42
getCursorLine, 42
getCursorTitleText, 42
getPosCursor, 42
getRectangle, 43
getText, 43
getType, 43
initShape, 43
initText, 43
isCursorSelected, 44
mouseClickedOnCursor, 44
moveCursor, 44
moveRectangleShape, 44
setCursorSelected, 45
setCursorText, 45
updateCursor, 45
Cursor.hpp
CursorType, 208
ROTATE, 208
TRANSLATE, 208
CursorType
Cursor.hpp, 208
Cylindre, 46
~Cylindre, 47
Cylindre, 46
getColor, 47
getIntersection, 47
getMaterial, 47
getType, 48
intersect, 48
intersection_cap, 48
normal_at, 49
Cylindre.hpp
createPrimitive, 188
registerPlugin, 188

Default, 49
Default, 50
Define.hpp
BACKGROUNDCOLOR, 161
COLOR_THRESHOLD, 161
FAST_RENDER_RATIO, 162
FONT, 162
MAX_DEPTH, 162
OCCLUSION_SAMPLE_PX, 162
RENDER_COLOMNS_DIVISION_PIXEL, 162
ROTATE_X, 162
ROTATE_Y, 162
ROTATE_Z, 162
THREADS_NUMBER, 162
TRANSLATE_X, 162
TRANSLATE_Y, 163
TRANSLATE_Z, 163
DirectionalLight, 50
DirectionalLight, 51
getAmbientComponent, 52
getColor, 52
getDiffuseComponent, 52
getDirection, 52
getIntensity, 52
getPosition, 53
getSpecularComponent, 53
getType, 53
DirectionalLight.hpp
createLight, 177
registerPlugin, 177
dot
Math::Point, 101
Math::Vector, 144

Face, 54
~Face, 55
Face, 54
getColor, 55
getIntersection, 55
getMaterial, 55
getType, 56
intersect, 56
normal_at, 56
FAST_RENDER_RATIO
Define.hpp, 162
FONT
Define.hpp, 162

getAmbient
AMaterial, 14
getAmbientComponent
AmbientLight, 18
DirectionalLight, 52
ILight, 63
PointLight, 106
getB
Color, 28
getCamera
Scene, 122
getCol
Math::Matrix< X, Y >, 80
getColor
AmbientLight, 18
Cone, 34
Cylindre, 47
DirectionalLight, 52
Face, 55
HitInfo, 61
ILight, 63
InfinitCone, 70
InfiniteCylindre, 73
IPrimitive, 75
Plane, 90
PointLight, 106
Sphere, 131

Triangle, 140
getColorAt
AMaterial, 14
ChessboardMaterial, 26
PerlinNoiseMaterial, 88
getCursorLine
Cursor, 42
getCursorTitleText
Cursor, 42
getDiffuse
AMaterial, 14
Scene, 122
getDiffuseColor
AMaterial, 15
getDiffuseComponent
AmbientLight, 18
DirectionalLight, 52
ILight, 63
PointLight, 106
getDirection
AmbientLight, 19
DirectionalLight, 52
ILight, 63
PointLight, 106
Ray, 109
getDistance
HitInfo, 61
getFov
Camera, 21
getG
Color, 29
getHeight
Camera, 21
getIntensity
AmbientLight, 19
DirectionalLight, 52
ILight, 64
PointLight, 106
getIntersectBase
Cone, 34
getIntersectCone
Cone, 35
getIntersection
Cone, 35
Cylindre, 47
Face, 55
InfinitCone, 70
InfiniteCylindre, 73
IPrimitive, 75
Plane, 90
Sphere, 131
Triangle, 140
getLights
Scene, 123
getMaterial
Cone, 35
Cylindre, 47
Face, 55
InfinitCone, 70
InfiniteCylindre, 73
IPrimitive, 76
Plane, 90
Sphere, 133
Triangle, 140
getModifableCamera
Scene, 123
getName
AMaterial, 15
getNormal
HitInfo, 61
getOrigin
Ray, 109
Screen, 128
getPosCursor
Cursor, 42
getPosition
AmbientLight, 19
Camera, 22
DirectionalLight, 53
HitInfo, 61
ILight, 64
PointLight, 107
getPrimitives
Scene, 123
getR
Color, 29
getRay
Camera, 22
getRectangle
Cursor, 43
getRefractionIndex
AMaterial, 15
getRow
Math::Matrix< X, Y >, 80
getShininess
AMaterial, 15
getSpecular
AMaterial, 15
getSpecularColor
AMaterial, 15
getSpecularComponent
AmbientLight, 19
DirectionalLight, 53
ILight, 64
PointLight, 107
getSymbol
PluginsHandle, 95
PluginsManager, 97
getText
Cursor, 43
getTransparency
AMaterial, 15
getType
AmbientLight, 20
Cone, 36
Cursor, 43

Cylindre, 48
DirectionalLight, 53
Face, 56
ILight, 64
InfinitCone, 70
InfiniteCylindre, 73
IPrimitive, 76
Plane, 90
PointLight, 107
Sphere, 133
Triangle, 141
getWidth
Camera, 22
getX
Math::Point, 101
Math::Vector, 144
getY
Math::Point, 101
Math::Vector, 144
getZ
Math::Point, 101
Math::Vector, 145
Gui, 57
~Gui, 58
createImage, 58
Gui, 57
handleInput, 58
isOpen, 59
renderScene, 59
updateScene, 59

handleInput
Gui, 58
help
Main.cpp, 221
HitInfo, 59
getColor, 61
getDistance, 61
getNormal, 61
getPosition, 61
HitInfo, 60

ILight, 62
~ILight, 63
getAmbientComponent, 63
getColor, 63
getDiffuseComponent, 63
getDirection, 63
getIntensity, 64
getPosition, 64
getSpecularComponent, 64
getType, 64
Image, 65
~Image, 66
createDirAndFileName, 66
Image, 65
save, 66
ImageException, 67
ImageException, 67
include/core/Camera.hpp, 153
include/core/Core.hpp, 154, 155
include/core/Gui.hpp, 156
include/core/Render.hpp, 157, 158
include/core/Scene.hpp, 159
include/core/Screen.hpp, 160
include/Define.hpp, 161, 163
include/exceptions/Exception.hpp, 164
include/images/Image.hpp, 165
include/interfaces/ILight.hpp, 166
include/interfaces/IPrimitive.hpp, 167
include/interfaces/ITransformations.hpp, 168
include/loader/ObjParser.hpp, 168, 169
include/loader/SceneLoader.hpp, 169, 170
include/materials/AMaterial.hpp, 170, 171
include/materials/Chessboard.hpp, 171, 172
include/materials/Default.hpp, 172
include/materials/Metal.hpp, 173
include/materials/Mtl.hpp, 173
include/materials/PerlinNoise.hpp, 174
include/materials/Transparent.hpp, 174, 175
include/plugins/lights/AmbientLight.hpp, 175, 176
include/plugins/lights/DirectionalLight.hpp, 177, 178
include/plugins/lights/PointLight.hpp, 179, 180
include/plugins/PluginsFactory.hpp, 181, 182
include/plugins/PluginsHandle.hpp, 182, 183
include/plugins/PluginsManager.hpp, 183, 184
include/plugins/primitives/Cone.hpp, 185, 186
include/plugins/primitives/Cylindre.hpp, 187, 188
include/plugins/primitives/InfinitCone.hpp, 190, 191
include/plugins/primitives/InfiniteCylindre.hpp, 192, 194
include/plugins/primitives/Plane.hpp, 195, 196
include/plugins/primitives/Sphere.hpp, 198, 199
include/plugins/primitives/Triangle.hpp, 200, 202
include/transformations/Rotation.hpp, 203, 204
include/transformations/Shear.hpp, 204, 205
include/transformations/Translation.hpp, 205, 206
include/utilitaries/Color.hpp, 206, 207
include/utilitaries/Cursor.hpp, 207, 208
include/utilitaries/Face.hpp, 209
include/utilitaries/HitInfo.hpp, 210
include/utilitaries/Matrix.hpp, 211
include/utilitaries/Point.hpp, 212
include/utilitaries/Ray.hpp, 213, 214
include/utilitaries/Vector.hpp, 214, 215
InfinitCone, 68
~InfinitCone, 69
getColor, 70
getIntersection, 70
getMaterial, 70
getType, 70
InfinitCone, 69
intersect, 70
normal_at, 71
InfinitCone.hpp
createPrimitive, 190
registerPlugin, 190
InfiniteCylindre, 71

~InfiniteCylindre, 72
getColor, 73
getIntersection, 73
getMaterial, 73
getType, 73
InfiniteCylindre, 72
intersect, 74
normal_at, 74
InfiniteCylindre.hpp
createPrimitive, 193
registerPlugin, 193
init
Core, 39
initShape
Cursor, 43
initText
Cursor, 43
intersect
Cone, 36
Cylindre, 48
Face, 56
InfinitCone, 70
InfiniteCylindre, 74
IPrimitive, 76
Plane, 91
Sphere, 133
Triangle, 141
intersect_base
Cone, 36
intersectCone
Cone, 37
intersection_cap
Cylindre, 48
invert
Math::Vector, 145
IPrimitive, 75
~IPrimitive, 75
getColor, 75
getIntersection, 75
getMaterial, 76
getType, 76
intersect, 76
normal_at, 77
isCursorSelected
Cursor, 44
isInShadowOfShape
Render, 114
isOpen
Gui, 59
ITransformations, 77
~ITransformations, 78
apply, 78

length
Math::Vector, 145
LightsFactory
PluginsFactory.hpp, 182
loadFromMtlLine
AMaterial, 15
loadMaterials
ObjParser, 85
loadPlugins
PluginsManager, 98
loop
Core, 40

main
Main.cpp, 221
Main.cpp
help, 221
main, 221
Math, 11
operator∗, 11
Math::Matrix< X, Y >, 79
_col, 82
_matrix, 82
_row, 82
~Matrix, 80
getCol, 80
getRow, 80
Matrix, 80
operator(), 80, 81
operator∗, 81
Math::Point, 99
dot, 101
getX, 101
getY, 101
getZ, 101
operator+, 101, 102
operator+=, 102
operator-, 102, 103
operator==, 103
Point, 100
pointToMatrix, 103
setX, 103
setY, 104
setZ, 104
shearToMatrix, 104
Math::Vector, 142
crossProduct, 144
dot, 144
getX, 144
getY, 144
getZ, 145
invert, 145
length, 145
near_zero, 145
normalize, 146
operator+, 147
operator+=, 147
operator-, 148
operator-=, 148
operator/, 148, 149
operator/=, 149
operator==, 150
operator∗, 146
operator∗=, 146, 147
random, 150

random_double, 150, 151
reflect, 151
shearToMatrix, 151
Vector, 143
vectorToMatrix, 151
Matrix
Math::Matrix< X, Y >, 80
MAX_DEPTH
Define.hpp, 162
Metal, 82
Metal, 83
mouseClickedOnCursor
Cursor, 44
moveCursor
Cursor, 44
moveRectangleShape
Cursor, 44
Mtl, 83
~Mtl, 84
Mtl, 84

near_zero
Math::Vector, 145
normal_at
Cone, 37
Cylindre, 49
Face, 56
InfinitCone, 71
InfiniteCylindre, 74
IPrimitive, 77
Plane, 91
Sphere, 134
Triangle, 141
normalize
Math::Vector, 146
notifyRenderThread
Core, 40

obj/core/CalculateRender.d, 218
obj/core/Camera.d, 218
obj/core/Core.d, 218
obj/core/Gui.d, 218
obj/core/hotreload.d, 218
obj/core/Main.d, 218
obj/core/PluginsHandle.d, 218
obj/core/PluginsManager.d, 218
obj/core/Render.d, 218
obj/core/Scene.d, 218
obj/core/Screen.d, 218
obj/exceptions/Exception.d, 218
obj/images/Image.d, 218
obj/loader/ObjParser.d, 218
obj/loader/SceneLoader.d, 218
obj/materials/AMaterial.d, 218
obj/materials/Chessboard.d, 218
obj/materials/Default.d, 218
obj/materials/Metal.d, 218
obj/materials/PerlinNoise.d, 218
obj/materials/Transparent.d, 218
obj/transformation/Rotation.d, 218
obj/transformation/Shear.d, 218
obj/transformation/Translation.d, 218
obj/utilitaries/Color.d, 218
obj/utilitaries/Cursor.d, 218
obj/utilitaries/Face.d, 218
obj/utilitaries/HitInfo.d, 218
obj/utilitaries/Point.d, 218
obj/utilitaries/Ray.d, 218
obj/utilitaries/Vector.d, 218
ObjParser, 85
loadMaterials, 85
parse, 85
parseMaterial, 86
OCCLUSION_SAMPLE_PX
Define.hpp, 162
operator<<
Color.cpp, 229
Color.hpp, 207
operator()
Math::Matrix< X, Y >, 80, 81
operator+
Color, 31
Math::Point, 101, 102
Math::Vector, 147
operator+=
Color, 31
Math::Point, 102
Math::Vector, 147
operator-
Color, 31
Math::Point, 102, 103
Math::Vector, 148
operator-=
Math::Vector, 148
operator/
Color, 32
Math::Vector, 148, 149
operator/=
Color, 32
Math::Vector, 149
operator=
PluginsHandle, 95, 96
operator==
Math::Point, 103
Math::Vector, 150
operator*
Color, 29, 30
Math, 11
Math::Matrix< X, Y >, 81
Math::Vector, 146
operator*=
Color, 30
Math::Vector, 146, 147
orthogonalVector
CalculateRender.cpp, 219

parse
ObjParser, 85

parseMaterial
ObjParser, 86
parseSceneCamera
SceneLoader, 125
parseSceneFile
SceneLoader, 126
parseSceneLight
SceneLoader, 126
parseScenePrimitive
SceneLoader, 126
PerlinNoiseMaterial, 86
~PerlinNoiseMaterial, 87
getColorAt, 88
PerlinNoiseMaterial, 87
Plane, 88
~Plane, 89
getColor, 90
getIntersection, 90
getMaterial, 90
getType, 90
intersect, 91
normal_at, 91
Plane, 89
Plane.hpp
createPrimitive, 196
registerPlugin, 196
PlugInException, 92
PluginsException, 92
PluginsException, 92
PluginsFactory.hpp
LightsFactory, 182
PrimitivesFactory, 182
PluginsHandle, 93
~PluginsHandle, 93
getSymbol, 95
operator=, 95, 96
PluginsHandle, 93, 95
PluginsManager, 96
createLight, 97
createPrimitive, 97
getSymbol, 97
loadPlugins, 98
registerLightFactory, 98
registerPrimitiveFactory, 98
unregisterAllFactories, 99
Point
Math::Point, 100
pointAt
Screen, 128
PointLight, 104
getAmbientComponent, 106
getColor, 106
getDiffuseComponent, 106
getDirection, 106
getIntensity, 106
getPosition, 107
getSpecularComponent, 107
getType, 107
PointLight, 105
PointLight.hpp
createLight, 179
registerPlugin, 180
pointToMatrix
Math::Point, 103
PrimitivesFactory
PluginsFactory.hpp, 182

random
Math::Vector, 150
random_double
CalculateRender.cpp, 219
Math::Vector, 150, 151
randomHemisphereDirection
Render, 114
Ray, 108
getDirection, 109
getOrigin, 109
Ray, 108
setDirection, 109
setOrigin, 109
RaytracerException, 110
RaytracerException, 110
reflect
Math::Vector, 151
Render, 114
registerLightFactory
PluginsManager, 98
registerPlugin
AmbientLight.hpp, 175
Cone.hpp, 185
Cylindre.hpp, 188
DirectionalLight.hpp, 177
InfinitCone.hpp, 190
InfiniteCylindre.hpp, 193
Plane.hpp, 196
PointLight.hpp, 180
Sphere.hpp, 199
Triangle.hpp, 201
registerPrimitiveFactory
PluginsManager, 98
reloadScene
Core, 40
Render, 110
~Render, 112
adaptiveSample, 112
AmbiantOcclusion, 112
ComputeDiffuseLights, 113
computeFastRender, 113
ComputePixelColor, 113
computeRender, 113
ComputeSpecularLight, 113
isInShadowOfShape, 114
randomHemisphereDirection, 114
reflect, 114
Render, 111
renderPixel, 114
renderReflection, 116

renderRefraction, 116
sendRay, 116
setProgressCallback, 117
RENDER_COLOMNS_DIVISION_PIXEL
Define.hpp, 162
RenderException, 117
RenderException, 118
renderPixel
Render, 114
renderReflection
Render, 116
renderRefraction
Render, 116
renderScene
Gui, 59
ROTATE
Cursor.hpp, 208
rotate
Camera, 23
Screen, 128
ROTATE_X
Define.hpp, 162
ROTATE_Y
Define.hpp, 162
ROTATE_Z
Define.hpp, 162
RotateX
Rotation, 120
RotateY
Rotation, 120
RotateZ
Rotation, 120
Rotation, 118
~Rotation, 119
apply, 120
RotateX, 120
RotateY, 120
RotateZ, 120
Rotation, 119

save
Image, 66
Scene, 121
~Scene, 121
addCamera, 122
addLight, 122
addPrimitive, 122
getCamera, 122
getDiffuse, 122
getLights, 123
getModifableCamera, 123
getPrimitives, 123
Scene, 121
setDiffuse, 123
setRotate, 124
setShear, 124
setTranslateScene, 124
SceneException, 124
SceneException, 125
SceneLoader, 125
parseSceneCamera, 125
parseSceneFile, 126
parseSceneLight, 126
parseScenePrimitive, 126
Screen, 127
~Screen, 127
getOrigin, 128
pointAt, 128
rotate, 128
Screen, 127
shear, 128
translate, 128
sendRay
Render, 116
setColor
Color, 32
setCtrl
Core, 40
setCursorSelected
Cursor, 45
setCursorText
Cursor, 45
setDiffuse
Scene, 123
setDiffuseColor
AMaterial, 16
setDirection
Ray, 109
setHandleRotation
Core, 40
setHandleTranslation
Core, 41
setOrigin
Ray, 109
setPosition
Camera, 23
setProgressCallback
Render, 117
setRay
Camera, 23
setRefractionIndex
AMaterial, 16
setRotate
Scene, 124
setShear
Scene, 124
setShininess
AMaterial, 16
setSpecularColor
AMaterial, 16
setTranslateScene
Scene, 124
setTransparency
AMaterial, 17
setX
Math::Point, 103
setY

Math::Point, 104
setZ
Math::Point, 104
Shear, 128
~Shear, 129
apply, 129
Shear, 129
shear, 130
shear
Camera, 24
Screen, 128
Shear, 130
shearToMatrix
Math::Point, 104
Math::Vector, 151
Sphere, 130
~Sphere, 131
getColor, 131
getIntersection, 131
getMaterial, 133
getType, 133
intersect, 133
normal_at, 134
Sphere, 131
Sphere.hpp
createPrimitive, 198
registerPlugin, 199
src/core/CalculateRender.cpp, 218
src/core/Camera.cpp, 219
src/core/Core.cpp, 220
src/core/Gui.cpp, 220
src/core/hotreload.cpp, 220
src/core/Main.cpp, 220
src/core/PluginsHandle.cpp, 222
src/core/PluginsManager.cpp, 222
src/core/Render.cpp, 222
src/core/Scene.cpp, 222
src/core/Screen.cpp, 223
src/exceptions/Exception.cpp, 223
src/images/Image.cpp, 223
src/loader/ObjParser.cpp, 223
src/loader/SceneLoader.cpp, 223
src/materials/AMaterial.cpp, 224
src/materials/Chessboard.cpp, 224
src/materials/Default.cpp, 224
src/materials/Metal.cpp, 224
src/materials/PerlinNoise.cpp, 224
src/materials/Transparent.cpp, 224
src/plugins/lights/AmbientLight.cpp, 225
src/plugins/lights/DirectionalLight.cpp, 225
src/plugins/lights/PointLight.cpp, 225
src/plugins/primitives/Cone.cpp, 226
src/plugins/primitives/Cylindre.cpp, 226
src/plugins/primitives/InfinitCone.cpp, 226
src/plugins/primitives/InfiniteCylindre.cpp, 227
src/plugins/primitives/Plane.cpp, 227
src/plugins/primitives/Sphere.cpp, 228
src/plugins/primitives/Triangle.cpp, 228
src/transformation/Rotation.cpp, 228
src/transformation/Shear.cpp, 228
src/transformation/Translation.cpp, 229
src/utilitaries/Color.cpp, 229
src/utilitaries/Cursor.cpp, 230
src/utilitaries/Face.cpp, 230
src/utilitaries/HitInfo.cpp, 230
src/utilitaries/Point.cpp, 230
src/utilitaries/Ray.cpp, 231
src/utilitaries/Vector.cpp, 231

THREADS_NUMBER
Define.hpp, 162
TRANSLATE
Cursor.hpp, 208
translate
Camera, 24
Screen, 128
TRANSLATE_X
Define.hpp, 162
TRANSLATE_Y
Define.hpp, 163
TRANSLATE_Z
Define.hpp, 163
translateDown
Translation, 136
translateIn
Translation, 136
translateLeft
Translation, 137
translateOut
Translation, 137
translateRight
Translation, 137
translateUp
Translation, 137
Translation, 134
~Translation, 136
apply, 136
translateDown, 136
translateIn, 136
translateLeft, 137
translateOut, 137
translateRight, 137
translateUp, 137
Translation, 135
Transparent, 137
Transparent, 138
Triangle, 139
~Triangle, 140
getColor, 140
getIntersection, 140
getMaterial, 140
getType, 141
intersect, 141
normal_at, 141
Triangle, 139
Triangle.hpp
createPrimitive, 201

registerPlugin, 201

unregisterAllFactories
PluginsManager, 99
updateCursor
Cursor, 45
updateScene
Gui, 59

Vector
Math::Vector, 143
vectorToMatrix
Math::Vector, 151